

HAWK

version 1.1 (February 5, 2025)

<https://hawk-sign.info>

Joppe W. Bos¹, Olivier Bronchain¹, Léo Ducas^{2,3}, Serge Fehr^{2,3}, Yu-Hsuan Huang², Thomas Pornin⁴, Eamonn W. Postlethwaite², Thomas Prest⁵, Ludo N. Pulles², Wessel van Woerden⁶

¹ NXP Semiconductors,

² Centrum Wiskunde & Informatica,

³ Mathematical Institute, Leiden University,

⁴ NCC Group,

⁵ PQShield,

⁶ Institut de Mathématiques de Bordeaux.

1 Introduction

HAWK is a signature scheme inspired by the introduction of the lattice isomorphism problem (LIP) to signatures [DvW22], and this specification document is based on the article that first described a practical variant [DPPvW22a]. The move from the framework of [DvW22] towards a practical variant in [DPPvW22a] is achieved by introducing module structure and using simpler sampling procedures to create signatures. The HAWK signature scheme specified in this document introduces further simplifications and optimisations compared to the signature scheme introduced in [DPPvW22a], which we will refer to as HAWK-AC22. In particular, the distribution used to sample private keys in HAWK is simplified compared to that in HAWK-AC22.

The practical security of HAWK is determined by extensive lattice reduction experiments along with a detailed cryptanalysis. The theoretical security of HAWK is given by a lattice problem called the one more (approximate) shortest vector problem, or **omSVP**. In particular, violating the **SUF-CMA** security of HAWK allows one to solve **omSVP**. The problem of direct secret key recovery from public information is an instance of the search module lattice isomorphism problem, or **smLIP**. Compared to HAWK-AC22 the experimental cryptanalysis has been extended and the reduction to **omSVP** has been modularised, extended to the **qROM** and its loss has been made explicit. Throughout we work under the premise that our formal reductions and problems give us confidence in the robustness of our design, and our cryptanalysis gives us confidence in the robustness of our parameters.

We provide two parameter sets, HAWK-512 and HAWK-1024, that our analysis suggests satisfy NIST-I and NIST-V security levels respectively. We also provide a challenge parameter set, HAWK-256 to act as a cryptanalytic target.

For didactic purposes we provide an implementation of HAWK in Python 3 that follows the specification, except for in one explicitly commented place. We also provide an optimised C implementation of the exact specification of HAWK. Our C implementation is lightweight, fast, isochronous and without floating points.

We discuss the advantages and limitations of HAWK in Section 1.1.

A note on naming. The name HAWK is similar to another signature scheme: **FALCON**, which was selected by NIST for standardisation. Initially, beyond sharing the hash-and-sign design of **FALCON**, there were no obvious similarities. However, as part of our key generation we need to solve an NTRU equation in a similar setting to **FALCON** and our HAWK-AC22 implementation reused much of the **FALCON** code. We therefore, despite the different underlying hard problems, named this scheme similarly in homage.

1.0.1 Overview of this specification

Technical preliminaries are given in Section 2. The specification of HAWK is given in Section 3. Implementation and performance details are given in Section 4. In Section 5 we detail our cryptanalytic model and experimental results for HAWK, followed by our estimated security strengths. In Section 6 we detail our formal security claims and give the **omSVP** reduction. We also discuss the **omSVP** and **smLIP** assumptions, as well as our design decisions with respect to security. To aid the ease of use of this document the preliminaries of Section 2 should be read by all, then

- implementors need only read Sections 3 and 4,
- cryptanalysts need only read Section 5, as well as possibly Section 6 for a discussion of **omSVP** and **smLIP**,
- theoretical cryptographers need only read Section 2 and Section 6 for the definitions of **omSVP**, **smLIP** and reductions.

Table 1: Speed of HAWK. Bracketed values require more RAM, see Table 3.

	HAWK-512	HAWK-1024
Speed on x86 “Coffee Lake” with AVX2 (clock cycles)		
Key pair generation	8.43×10^6	4.37×10^7
Signature generation	8.54×10^4 (4.37×10^4)	1.81×10^5 (8.54×10^4)
Signature verification	1.48×10^5 (1.24×10^5)	3.03×10^5 (2.55×10^5)
Speed on ARM Cortex M4 (clock cycles)		
Key pair generation	5.23×10^7	2.26×10^8
Signature generation	2.80×10^6 (1.16×10^6)	1.42×10^6 (1.23×10^6)
Signature verification	1.42×10^6 (1.23×10^6)	3.01×10^6 (2.61×10^6)

Table 2: Key and signature sizes for HAWK in bytes.

	HAWK-512	HAWK-1024
Private key size	184	360
Public key size	1024	2440
Signature size	555	1221

1.1 Advantages and Limitations

1.1.1 Advantages

Speed. The procedures for generating and verifying signatures are fast on all devices, including low end devices, see Table 1.

Compactness. The keys and signature sizes are all rather small, see Table 2.

Memory usage. HAWK has a small memory footprint. For example, HAWK-512 requires no more than 14 kiB of RAM for any algorithm, including the faster variants of signing and verification.¹ If the keys can be generated externally and hardcoded onto the device, then HAWK-512 and HAWK-1024 can sign and verify using only 6 kiB and 11 kiB of RAM respectively. This makes HAWK a good candidate for many embedded platforms based on ARM Cortex-M0(+) cores: products in this range include, for example, the LPC800 series by NXP, STM32F0 by ST, or the XMC1000 by Infineon (16 KiB of SRAM).

Well suited for various hardware. HAWK is free of floating-point arithmetic. Therefore, no floating point (double precision) unit is required. This enables one to run HAWK on many (constrained) devices not equipped with such an FPU.

¹A kibibyte (kiB) is 1024 bytes

Table 3: RAM usage of HAWK in bytes. Bracketed values correspond to Table 1.

	HAWK-512	HAWK-1024
Key pair generation	14336	27648
Signature generation	4096 (5272)	7168 (9512)
Signature verification	6144 (8768)	11264 (16512)

Simplicity of distributions. Key generation in HAWK requires samples from a centred binomial distribution, which is simple to sample from using a source of uniform bits. Signing requires discrete Gaussian sampling with a fixed width from two cosets of the integer lattice $\mathbb{Z}$, which is easily achieved using two fixed precomputed tables of sufficient precision.

Isochronous. Any function within HAWK that depends on secret data has a running time independent of said data.

Worst-case running time. With high probability a signature is generated without any internal restarts. In particular, the signing procedures of HAWK-512 and HAWK-1024 restart in approximately 1/200000 and 1/400000 cases respectively.

BUFF transform. Originally HAWK was designed so that applying the BUFF transform [CDF⁺20, Fig. 6]² required implementors to append a single hash digest to the signature and subsequently check its value in verification. This choice meant that if the BUFF security properties were demanded then the required changes would be minimal.

Subsequently it has been shown that HAWK achieves the BUFF security properties *as is* [ADM⁺24], with the technical caveat that the original definition of the non-resignability security property was shown to be unachievable when signatures have high entropy [DFHS24]. As such [ADM⁺24] shows HAWK natively satisfies a weaker, but achievable, non-resignability property.

1.1.2 Limitations

No efficient masking (yet). Despite the simplicity of requiring only two fixed discrete Gaussian distributions to be sampled during signing, it is an open research problem to efficiently mask the table based method we use. On a positive note, besides this component it is known how to efficiently mask the remainder of the design of HAWK.

Recent work [GR24] experimentally demonstrates various power analysis attacks against the Gaussian sampler of HAWK in the reference implementation. They subsequently give a method, based on the approach of [DDGR20], to quantify the cryptanalytic relevance of different types of knowledge about the Gaussian sample. Finally, they give low overhead countermeasures that seem practically robust and that thwart their own attacks, while not representing provable side channel resistance.

Security assumptions (direct key recovery). Recovering the private key from the public key in HAWK is equivalent to solving an instance of rank two **smLIP** that follows a distribution implicit in our key generation. There exists research into the lattice isomorphism problem over the integer lattice [GS02, Szy03, BM21, BGPS23, DPPvW22a, Duc24, BN24], and with a ring structure [LS17, LS19] where the latter pair of works show it is efficiently solvable. However, the use of the problem in constructive cryptography, especially with module structure, is relatively new.

The HAWK team are grateful for the diligent work of many who have much clarified the hardness of **smLIP** in the time since this specification document was originally published. We now summarise these works.

In [MPPW24] a classical algorithm is given to solve rank two **smLIP** when defined over a totally real field that takes time polynomial in the degree of the field extension and the residue at one of the Dedekind zeta function of the field. This algorithm requires a heuristic assumption about the probability that a relevant integral ideal is prime, see [MPPW24,

²Note that the conference version [CDF⁺21] numbers its figures differently.

Ass. 1]. The ‘closest’ case to HAWK is that of the maximal totally real subfield of a cyclotomic field, as in [MPPW24, Cor. 4.7].

Several works subsequently appeared that removed the heuristic assumption required above and gave classical polynomial time algorithms for solving rank two **smLIP** over totally real fields. These works also found computational tasks whose solution is sufficient to solve rank two **smLIP** over CM-fields.

First [CFM⁺24] extends [MPPW24] and shows that to solve rank two **smLIP** it is sufficient to solve a type of principle ideal problem with advice in a (potentially non commutative) extension of the field over which **smLIP** is defined. When restricted to totally real fields the particular principle ideal problem can be efficiently and classically solved. When considering CM-fields, in particular the power of two cyclotomics over which HAWK is built, the principle ideal problem must be solved over a quaternionic extension of the maximal totally real subfield, see [CFM⁺24, Cor. 3.16]. The more geometric argument of [EP24] reaches the same conclusion for HAWK, see the discussion of [EP24, Sec. 2.2.5]. Finally, in [LJPW24] it is shown that the ability to compute a particular symplectic automorphism allows one to instead solve a principle ideal problem in a commutative ring to solve rank two **smLIP**, allowing the use of [LS17, LS19]. If **smLIP** is defined over a totally real field this symplectic automorphism is efficient to compute from public information.

In [APvW25] the algorithm of [MPPW24] that solves rank two **smLIP** over totally real fields is extended to any number field with at least one real embedding, which covers the intermediate cases between totally real and totally complex fields. This algorithm runs in quantum polynomial time under a light heuristic, but can be adapted to a provable algorithm if the defining polynomial of the number field has a 2-transitive Galois group.

In conclusion, one may efficiently solve rank two **smLIP** over a field with at least one real embedding. To solve rank two **smLIP** over a CM-field (as in HAWK) one must be able to either solve a type of principle ideal problem over a quaternionic extension of some maximal totally real subfield or compute some symplectic automorphism. The generalisation and formalisation of the Gentry–Szydło algorithm of [LS17, LS19] is used throughout the above approaches.

Security assumptions (SUF-CMA). In the reprogrammable quantum random oracle model, violating the SUF-CMA security of HAWK allows one to solve **omSVP**. We pick the parameters of HAWK to satisfy our reduction, so that breaking the SUF-CMA security of HAWK breaks the corresponding **omSVP** instance with an explicit loss. This reduction is unidirectional, i.e. we do not know how to break the SUF-CMA security of HAWK by solving **omSVP**.

Like **smLIP**, the use of **omSVP** in cryptography is new, and has benefited from the work of the cryptographic community. In particular, the ‘pseudo automorphism’ of [LJPW24] provided an efficient solution to the original definition of **omSVP**. In the present version of this specification (see Changelog below) we account for this pseudo automorphism in our definition of **omSVP** and calculate that the loss of our reduction is essentially unchanged. A more fulsome study of [LJPW24] that characterises their approach more completely is necessary.

Furthermore, while we set the parameters of HAWK to satisfy our reduction, the parametrisation of **omSVP** this implies can be efficiently solved. To avoid this situation would require us to sample larger keys in our key generation procedure, which we choose not to do. This approach is indicative of our methodology; we use our reduction to **omSVP** to provide confidence in the robustness of the *design* of HAWK and our practical cryptanalysis to provide confidence in the robustness of our *parameters*.

Finally, we do not know how to publicly simulate the signature distribution of HAWK, à la [GPV08]. Instead, **omSVP** provides a well defined target for the applicability of learning style attacks [NR09, DN12] to HAWK, which require further attention.

Deviation from (previous) theoretical guarantees. To sign with a narrower discrete Gaussian and reduce the number of cosets required to be sampled from, HAWK deviates from the EUF-CMA proof structure of the signature scheme given in [DvW22, Sec. 6] and therefore requires the omSVP problem for security, rather than the Δ LIP assumption. Also, the choice to use a centred binomial distribution to simplify the generation of public keys, while having no practical effect on security according to our analysis, does not fit the notion of average case distribution given in [DvW22, Sec. 3].

The final three limitations are discussed in more detail in Sections 5 and 6.

1.2 Changelog

Version 1.0.1.

- Fixed some typos in Algorithms 14 and 15. Thanks to Markku-Juhani O. Saarinen for bringing them to our attention.

Version 1.0.2.

- References to [LS17, LS19, BN24] added in Section 1.1. Thanks to Alice Silverberg for bringing them to our attention.

Version 1.1.

- Updated the omSVP game to include ω as a ‘trivial win’ in Fig. 5 and updated the reduction to the SUF-CMA security of HAWK, see Section 6. The original omSVP game [DPPvW22a] could be trivially won using the results of [LJPW24].
- Updated the introduction to reflect new understanding of the BUFF transform [DFHS24, ADM⁺24], side channel analysis [GR24] and the cryptanalysis of smLIP [MPPW24, CFM⁺24, EP24, LJPW24, APvW25] and omSVP [LJPW24].
- Switched the adaptive reprogramming lemma (Lemma 4) to that of [GHHM21] and amended the upper bound on distinguishing advantage.

2 Preliminaries

Vectors and matrices. We denote (column) vectors in lowercase as e.g. $\mathbf{v}$ and the number of elements in such a vector as $\text{len}(\mathbf{v})$. A vector $\mathbf{v}$ is formed of elements $v_0, \dots, v_{\text{len}(\mathbf{v})-1}$, that is, we index from 0. We denote matrices in uppercase as e.g. $\mathbf{B}$ and index from 0, so that if $\mathbf{B}$ has dimensions $r \times s$ the top right entry is $b_{0,0}$ and the bottom right entry is $b_{r-1,s-1}$. For any object X with 0 and 1 elements and positive integer n we define $\mathbf{I}_n(X)$ as the identity matrix in $X^{n \times n}$.

Distributions. We consider two families of discrete distributions.

The first is the centred binomial distribution of parameter $\eta \in \mathbb{Z}_{\geq 1}$. If $X \sim \text{Bin}(\eta)$ then

$$\text{Supp}(X) = \{-\eta, -\eta + 1, \dots, \eta\}, \quad \Pr[X = x] = \frac{1}{2^{2\eta}} \cdot \binom{2\eta}{x + \eta} \quad (1)$$

for $x \in \text{Supp}(X)$. It has $\mathbb{E}[X] = 0$ and $\mathbb{V}[X] = \eta/2$. Sampling $X \sim \text{Bin}(\eta)$ is easily achieved as

$$\left(\sum_{i=1}^{2\eta} x_i \right) - \eta, \quad (2)$$

for $x_1, \dots, x_{2\eta}$ sampled as i.i.d. bits, i.e. from the uniform distribution over $\{0, 1\}$.

The second is the discrete Gaussian distribution on $2\mathbb{Z} + c$ of parameters $\sigma \in \mathbb{R}_{>0}$ and $c \in \mathbb{R}$. Let

$$\rho_\sigma: \mathbb{R} \rightarrow \mathbb{R}, x \mapsto \exp(-x^2/2\sigma^2). \quad (3)$$

If $X \sim D_{2\mathbb{Z}+c,\sigma}$ then

$$\text{Supp}(X) = 2\mathbb{Z} + c, \quad \Pr[X = x] = \frac{1}{\sum_{z \in 2\mathbb{Z}+c} \rho_\sigma(z)} \cdot \rho_\sigma(x), \quad (4)$$

for $x \in \text{Supp}(X)$. We consider only $c \in \{0, 1\}$ in **HAWK** and for these two values of c , it holds $\mathbb{E}[X] = 0$. For σ large enough, $\mathbb{V}[X] \in \sigma^2 \cdot (1 + O(e^{-\sigma^2}))$ [MR04].

For higher dimensions $n \in \mathbb{Z}_{\geq 2}$, a sample $\mathbf{x} \in \mathbb{R}^n$ from the discrete Gaussian distribution on $2\mathbb{Z}^n + \mathbf{c}$ of parameters $\sigma \in \mathbb{R}_{>0}$ and $\mathbf{c} \in \mathbb{R}^n$ is obtained by sampling each $x_i \leftarrow D_{2\mathbb{Z}+c_i,\sigma}$ for $0 \leq i < n$ independently and combining these into a vector $\mathbf{x} \in \mathbb{R}^n$.

Rényi divergence. For two distributions P, Q with supports $\text{Supp}(P) \subseteq \text{Supp}(Q)$ we define the Rényi divergence (RD) of order $a \in (1, \infty)$ as

$$R_a(P \| Q) = \left(\sum_{x \in \text{Supp}(P)} \frac{P(x)^a}{Q(x)^{a-1}} \right)^{1/(a-1)}. \quad (5)$$

It is not symmetric in its arguments, and does not verify a triangle inequality. When the Rényi divergence is finite, which it will be for our applications, we think of it as a value $1 + \delta$ for $\delta \geq 0$; the smaller δ is the closer the two distributions. Note that for any P, Q on which the Rényi divergence is finite, $R_a(P \| Q)$ is non decreasing as a function of a , i.e. a larger order a defines a stricter notion of closeness.

We recall three properties of $R_a(P \| Q)$ and a useful result [BLL⁺15, Pre17]. For two distributions P, Q and two families $(P_i)_i, (Q_i)_i$ we have

- (data processing inequality) for any function f , $R_a(P^f \| Q^f) \leq R_a(P \| Q)$,

- (multiplicativity) $R_a(\prod_i P_i \parallel \prod_i Q_i) = \prod_i R_a(P_i \parallel Q_i)$,
- (probability preservation) for event $E \subseteq \text{Supp}(P)$, $Q(E) \geq P(E)^{a/(a-1)} / R_a(P \parallel Q)$.

Intuitively the data processing inequality says that new distributions P^f, Q^f formed by “processing” the output of P, Q with some function f can never increase the divergence. We often consider P as an inexact realisation of some ideal distribution Q and the event E as some adversary winning a search game; in this setting the probability preservation inequality allows us to bound above the probability of an adversary winning the search game using distribution P . We also consider the order a as a parameter that we may optimise over.

Finally, we note that [Pre17, (4)] gives us the following useful condition on the Rényi divergence. If a search game is λ -bit secure when instantiated with Q , the game samples Q at most q_s times, and for an order $a \geq 2\lambda + 1$

$$R_a(P \parallel Q) \leq 1 + \frac{1}{4q_s}, \quad (6)$$

then the search game is at least $(\lambda - 1)$ -bit secure when instantiated with P with the same sample bound q_s .

Number fields. We make use of number fields and their rings of integers. In particular for $n = 2^m$ with $m \in \{8, 9, 10\}$ we consider the power of two cyclotomic number field $K_n = \mathbb{Q}[X]/(X^n + 1)$ and its ring of integers $R_n = \mathbb{Z}[X]/(X^n + 1)$. For a polynomial $f \in K_n$ we let

$$f = f[0] + f[1]X + \dots + f[n-1]X^{n-1},$$

and define a bijection that maps it to the column vector of its coefficients

$$\text{vec}: K_n \rightarrow \mathbb{Q}^n, f \mapsto (f[0], \dots, f[n-1]). \quad (7)$$

Polynomials are thus also sequences of their n coefficients, in ascending degree order.

We also define

$$\text{rot}: K_n \rightarrow \mathbb{Q}^{n \times n}, f \mapsto (\text{vec}(f), \text{vec}(fX), \dots, \text{vec}(fX^{n-1})), \quad (8)$$

sometimes called the negacyclic matrix of f . We extend vec and rot to K_n^r and $K_n^{r \times r}$ as

$$\text{vec}: K_n^r \rightarrow \mathbb{Q}^{nr}, \mathbf{f} = (f_0, \dots, f_{r-1}) \mapsto (\text{vec}(f_0), \dots, \text{vec}(f_{r-1})) \quad (9)$$

and

$$\text{rot}: K_n^{r \times r} \rightarrow \mathbb{Q}^{nr \times nr}, \mathbf{B} \mapsto \begin{pmatrix} \text{rot}(b_{0,0}) & \dots & \text{rot}(b_{0,r-1}) \\ \vdots & \ddots & \vdots \\ \text{rot}(b_{r-1,0}) & \dots & \text{rot}(b_{r-1,r-1}) \end{pmatrix}, \quad (10)$$

where context informs the correct version of vec or rot to use. These number fields are CM-fields and therefore have an involutive automorphism representing complex conjugation; the Hermitian adjoint. We denote by

$$f^* = f[0] - f[n-1]X - \dots - f[1]X^{n-1} \quad (11)$$

the Hermitian adjoint of f . If $\mathbf{B} \in K_n^{r \times r}$ then $\mathbf{B}^*$ is the matrix obtained by applying the Hermitian adjoint to each entry of $\mathbf{B}^t$, the transpose matrix of $\mathbf{B}$.

Inner product and norm. We define an inner product over K_n as

$$\langle \cdot, \cdot \rangle: K_n \times K_n \rightarrow \mathbb{Q}, (f, g) \mapsto \frac{1}{n} \text{Tr}(f^* g), \quad (12)$$

where $\text{Tr}(\cdot)$ is the algebraic trace over the field extension $K_n/\mathbb{Q}$. Due to the normalisation factor $\frac{1}{n}$, if $f, g \in K_n$ then $\langle f, g \rangle$ equals the Euclidean inner product of $\text{vec}(f)$ and $\text{vec}(g)$. We define two norms,

$$\|\cdot\|: K_n \rightarrow \mathbb{Q}, f \mapsto \sqrt{\langle f, f \rangle}, \quad (13)$$

$$\|\cdot\|_\infty: K_n \rightarrow \mathbb{Q}, f \mapsto \max_{0 \leq i < n} (|f[i]|). \quad (14)$$

Note that these norms coincide with the “Euclidean” ℓ^2 and “infinity” ℓ^∞ norms of $\text{vec}(f)$ respectively. In particular, we may calculate

$$\|f\|^2 = f[0]^2 + \cdots + f[n-1]^2. \quad (15)$$

We extend the inner product in (12) to K_n^r via summation as

$$\langle \cdot, \cdot \rangle: K_n^r \times K_n^r \rightarrow \mathbb{Q}, (\mathbf{f}, \mathbf{g}) \mapsto \langle f_0, g_0 \rangle + \cdots + \langle f_{r-1}, g_{r-1} \rangle. \quad (16)$$

Similarly, we extend the norms in Eqs. (13) and (14) to K_n^r as

$$\|\cdot\|: K_n^r \rightarrow \mathbb{Q}, \mathbf{f} \mapsto \sqrt{\|f_0\|^2 + \cdots + \|f_{r-1}\|^2}, \quad (17)$$

$$\|\cdot\|_\infty: K_n^r \rightarrow \mathbb{Q}, \mathbf{f} \mapsto \max_{0 \leq i < r} (\|f_i\|_\infty). \quad (18)$$

These extended norms coincide with the ℓ^2 and ℓ^∞ norms of $\text{vec}(\mathbf{f})$ in $\mathbb{Q}^{nr}$. In particular, we may calculate

$$\|\mathbf{f}\|^2 = f_0[0]^2 + \cdots + f_0[n-1]^2 + \cdots + f_{r-1}[0]^2 + \cdots + f_{r-1}[n-1]^2. \quad (19)$$

Q-inner product and Q-norm. For $\mathbf{B} \in K_n^{r \times r}$ such that the columns of $\mathbf{B}$ are K_n -linearly independent, we consider the Hermitian form associated to the self adjoint matrix $\mathbf{Q} = \mathbf{B}^* \mathbf{B}$. We now define the **Q**-inner product, or the inner product with respect to this form, as

$$\langle \cdot, \cdot \rangle_{\mathbf{Q}}: K_n^r \times K_n^r \rightarrow \mathbb{Q}, (\mathbf{f}, \mathbf{g}) \mapsto \frac{1}{n} \text{Tr}(\mathbf{f}^* \mathbf{Q} \mathbf{g}). \quad (20)$$

Note that (20) specialises to (16) whenever $\mathbf{Q} = \mathbf{I}_r(K_n)$.

From the **Q**-inner product one may naturally define the **Q**-norm as

$$\|\cdot\|_{\mathbf{Q}}: K_n^r \rightarrow \mathbb{Q}, \mathbf{f} \mapsto \sqrt{\langle \mathbf{f}, \mathbf{f} \rangle_{\mathbf{Q}}}. \quad (21)$$

We also have the useful identities, when $\mathbf{Q} = \mathbf{B}^* \mathbf{B}$ for $\mathbf{B}$ with K_n -linearly independent columns and $\mathbf{f}, \mathbf{g} \in K_n^r$

$$\langle \mathbf{B} \mathbf{f}, \mathbf{B} \mathbf{g} \rangle = \langle \mathbf{f}, \mathbf{g} \rangle_{\mathbf{Q}}, \quad \|\mathbf{B} \mathbf{f}\| = \|\mathbf{f}\|_{\mathbf{Q}}. \quad (22)$$

These allow us to translate between the **Q**-norm and the (identity) norm if we know such a $\mathbf{B}$.

Moving from structured to unstructured. At various points we will implicitly consider “structured” objects, e.g. matrices in $R_n^{2 \times 2}$, as corresponding “unstructured” objects, e.g. matrices in $\mathbb{Z}^{2n \times 2n}$, via vec and rot . This is particularly the case in our experimental cryptanalysis, where we do not know how to exploit this structure and our tools work on matrices over the rationals or integers. This approach is also used in the implementation. We note here that for $\mathbf{B} \in K_n^{r \times s}$ and $\mathbf{C} \in K_n^{s \times t}$ we have

$$\mathbf{BC} = \text{rot}^{-1}(\text{rot}(\mathbf{B}) \text{rot}(\mathbf{C})),$$

where the inverse rot function is well defined because the product $\text{rot}(\mathbf{B}) \text{rot}(\mathbf{C})$ will consist of $r \times t$ negacyclic blocks of size $n \times n$. In the case where $t = 1$ this collapses to $\mathbf{Bc} = \text{vec}^{-1}(\text{rot}(\mathbf{B}) \text{vec}(\mathbf{c}))$. Also, for some Hermitian matrix $\mathbf{Q} \in K_n^{r \times r}$ and $\mathbf{f}, \mathbf{g} \in K_n^r$ we have

$$\langle \mathbf{f}, \mathbf{g} \rangle_{\mathbf{Q}} = \text{vec}(\mathbf{f})^t \text{rot}(\mathbf{Q}) \text{vec}(\mathbf{g}).$$

In short, to compute products and norms, we may pass from structured objects to unstructured objects.

However, in the case of performing lattice reduction on $\text{rot}(\mathbf{B})$ with $\mathbf{B} \in K_n^r$, the (unstructured) reduced basis $\mathbf{C} = \text{rot}(\mathbf{B})\mathbf{U} \in \mathbb{Q}^{nr}$ for $\mathbf{U} \in \text{GL}_{nr}(\mathbb{Z})$ is generally not of a “structured” form, i.e. not of the form $\text{rot}(\mathbf{B}')$ for some $\mathbf{B}' \in K_n^r$. We discuss this more in Section 5.

2.1 High level algorithmic description of HAWK

Here we give a high level description of the algorithms of HAWK. The high level descriptions of key generation, signature generation and signature verification for HAWK can be found in Algorithms 1, 2 and 3 respectively. These descriptions of the algorithms leave many subroutines undefined, and are not intended to be implemented; instead implementors are encouraged to read Sections 3 and 4. We assume n is a power of two and we assume the parameters $(\eta, \text{saltlen}_{\text{bits}}, \sigma_{\text{sign}}, \sigma_{\text{verify}})$, which depend on n , are known. We do not include subroutines for encoding and decoding data, but we do indicate where checks regarding encoding take place. Finally, we often compute the output of a function $\mathbf{H}$ on some data. In the full specification $\mathbf{H}$ is realised as SHAKE256 and varying amounts of output data are used depending on n and the point of use in a given algorithm. In this high level description we do not state the output length used, except when necessary, or specify how an object input to $\mathbf{H}$ may be considered as a bitstring.

Algorithm Sketch 1 High level HawkKeyGen

Ensure: $\mathbf{B} \in \text{GL}_2(R_n)$ and $\mathbf{Q} = \mathbf{B}^* \mathbf{B}$

- 1: Sample coefficients of $f, g \in R_n$ i.i.d. from $\text{Bin}(\eta)$
 - 2: **if** f-g-conditions(f, g) is false **then restart**
 - 3: $r \leftarrow \text{NTRUSolve}(f, g)$
 - 4: **if** r is $\perp$ **then restart**
 - 5: $(F, G) \leftarrow r$
 - 6: $\mathbf{B} \leftarrow \begin{pmatrix} f & F \\ g & G \end{pmatrix}$, $\mathbf{Q} \leftarrow \mathbf{B}^* \mathbf{B}$
 - 7: **if** KGen-encoding($\mathbf{Q}, \mathbf{B}$) is false **then restart**
 - 8: $\text{hpub} \leftarrow \mathbf{H}(\mathbf{Q})$
 - 9: **return** $(\text{pk}, \text{sk}) \leftarrow (\mathbf{Q}, (\mathbf{B}, \text{hpub}))$
-

Key generation. Key generation in HAWK consists of sampling a secret unimodular matrix $\mathbf{B} \in \text{GL}_2(R_n)$ from a distribution that is implicitly determined by a centred binomial distribution and the algorithm NTRUSolve .

First, two polynomials $f, g \in R_n$ have their coefficients i.i.d. sampled from $\text{Bin}(\eta)$, where η depends on the security parameter n . The conditions that make up **f-g-conditions** have several purposes, such as allowing for efficient constant time procedures within the algorithm **NTRUSolve** that follows, and to abide by the results of our practical cryptanalysis. The algorithm **NTRUSolve** outputs $F, G \in R_n$ such that $fG - gF = 1_{R_n}$, and such that F, G have somewhat small entries. We restart if **NTRUSolve** fails; this can happen if no such (F, G) exist, or because **NTRUSolve** fails to find them. The public key $\mathbf{Q}$ is then the Hermitian matrix $\mathbf{B}^* \mathbf{B}$.

The conditions that make up **KGen-encoding** check whether the keys can be properly encoded. Finally, we compute hpub , a hash of the public key, which is included in the secret key.

The recovery of sk from pk is an instance of the search module lattice isomorphism problem from a distribution determined by **HawkKeyGen**, see Section 6. We approach this cryptanalytic task experimentally via lattice reduction in Section 5. Intuitively, knowledge of $\mathbf{B}$ allows one to efficiently find short vectors in various cosets of $\mathbb{Z}^{2n}$, whereas knowledge of $\mathbf{Q}$ allows one to calculate the lengths of elements.

Algorithm Sketch 2 High level HawkSign

Require: A message m and secret key $\text{sk} = (\mathbf{B}, \text{hpub})$

Ensure: A signature sig formed of a uniform salt $\text{salt} \in \{0, 1\}^{\text{saltlen}_{\text{bits}}}$ and $s_1 \in R_n$

```

1:  $M \leftarrow \text{H}(\text{m} \parallel \text{hpub})$ 
2:  $\text{salt} \leftarrow \text{Rnd}(\text{saltlen}_{\text{bits}})$ 
3:  $\mathbf{h} \leftarrow \text{H}(M \parallel \text{salt})$ 
4:  $\mathbf{t} \leftarrow \mathbf{B} \cdot \mathbf{h} \bmod 2$ 
5:  $\mathbf{x} \leftarrow D_{2\mathbb{Z}^{2n} + \mathbf{t}, 2\sigma_{\text{sign}}}$ 
6: if  $\|\mathbf{x}\|^2 > 4 \cdot \sigma_{\text{verify}}^2 \cdot 2n$  then restart
7:  $\mathbf{w} \leftarrow \mathbf{B}^{-1} \mathbf{x}$ 
8: if  $\text{sym-break}(\mathbf{w})$  is false then  $\mathbf{w} = -\mathbf{w}$ 
9:  $\mathbf{s} \leftarrow \frac{1}{2}(\mathbf{h} - \mathbf{w})$ 
10:  $s_1 \leftarrow \text{Compress}(\mathbf{s})$ 
11: if  $\text{sig-encoding}(\text{salt}, s_1)$  is false then restart
12: return  $\text{sig} \leftarrow (\text{salt}, s_1)$ 
```

Signing. Signing in HAWK consists of using the message to determine a target coset $2R_n^2 + \mathbf{h}$ of $2R_n^2$. A random salt is used to make this choice of target coset non deterministic. A signature is the salt and (a compressed version of) a vector $\mathbf{s}$. This signature vector $\mathbf{s}$ is related via $\mathbf{h}$ to a vector $\mathbf{w}$ which is in the target coset and short under $\|\cdot\|_{\mathbf{Q}}$.

First two hashes are calculated; the first over the message m and hpub , then a uniform salt salt is sampled, which is hashed with the output of the first hash. The function $\text{Rnd}(r)$, on Line 2 of Algorithm 2, returns a uniform bitstring of length r . The function is called with input $\text{saltlen}_{\text{bits}}$, which depends on n . This design that hashes twice is chosen so that HAWK can be easily adapted to make use of the BUFF transform [CDF⁺20]. The output of the second hash has length $2n$ and is interpreted as a binary vector in R_n^2 .

We then sample a vector $\mathbf{x}$ from the discrete Gaussian distribution over $2\mathbb{Z}^{2n} + \mathbf{t}$ for $\mathbf{t} = \text{vec}(\mathbf{B}\mathbf{h})$ with the width parameter $2\sigma_{\text{sign}}$. Note that for this purpose, it is only required to know the value of $\mathbf{t}$ modulo 2. We then interpret the sampled $\mathbf{x}$ as an element of R_n^2 . Note that $\mathbf{w} = \mathbf{B}^{-1} \mathbf{x} \in 2R_n^2 + \mathbf{h}$ is in the target coset and that $\|\mathbf{w}\|_{\mathbf{Q}} = \|\mathbf{x}\|$. If $\|\mathbf{x}\|$ is too long the signature would fail in verification, so we restart.

The condition **sym-break** prevents a simple weak forgery attack. Since $\mathbf{h} - \mathbf{w} \in 2R_n^2$, we have $\mathbf{s} \in R_n^2$ and that $\mathbf{w}$ can be publicly recomputed from $\mathbf{s}$ and $\mathbf{h}$. The salt salt and a compressed version of $\mathbf{s}$ form the signature. This compression operation does not have

any secret information as input. Finally, **sig-encoding** checks whether the signature can be properly encoded.

A strong signature forgery requires one to find a short vector $\mathbf{w} \in 2R_n^2 + \mathbf{h}$ such that $\|\mathbf{w}\|_{\mathbf{Q}} \leq 2 \cdot \sigma_{\text{verify}} \sqrt{2n}$ holds for uniform $\mathbf{h} \in R_n^2$. Equivalently, one must find a vector $\mathbf{s} \in R_n^2$ that satisfies $\|\mathbf{h} - 2\mathbf{s}\|_{\mathbf{Q}} \leq 2 \cdot \sigma_{\text{verify}} \sqrt{2n}$. Both of these conditions can be stated equivalently under $\|\cdot\|$ as finding a short vector in the coset $2R_n^2 + \mathbf{Bh}$ or a close vector in $2R_n^2$ to $\mathbf{Bh}$. We approach this cryptanalytic task via estimating the complexity of lattice reduction based approaches to finding close vectors in Section 5.

Algorithm Sketch 3 High level HawkVerify

Require: A message $\mathbf{m}$, a public key $\mathbf{pk} = \mathbf{Q}$, and a signature $\text{sig} = (\text{salt}, s_1)$

Ensure: A bit determining whether sig is a valid signature on $\mathbf{m}$

```

1:  $\mathbf{h}_{\text{pub}} \leftarrow H(\mathbf{Q})$ 
2:  $M \leftarrow H(\mathbf{m} \parallel \mathbf{h}_{\text{pub}})$ 
3:  $\mathbf{h} \leftarrow H(M \parallel \text{salt})$ 
4:  $\mathbf{s} \leftarrow \text{Decompress}(s_1, \mathbf{h}, \mathbf{Q})$ 
5:  $\mathbf{w} \leftarrow \mathbf{h} - 2\mathbf{s}$ 
6: if  $\text{len}_{\text{bits}}(\text{salt}) = \text{saltlen}_{\text{bits}}$  and  $\mathbf{s} \in R_n^2$  and  $\text{sym-break}(\mathbf{w})$  and  $\|\mathbf{w}\|_{\mathbf{Q}}^2 \leq 4 \cdot \sigma_{\text{verify}}^2 \cdot 2n$ 
   then
7:   return 1
8: else
9:   return 0

```

Verification. Verification in HAWK recomputes values internal to Algorithm 2 from the signature sig , the message $\mathbf{m}$ and $\mathbf{pk}$. Ultimately it recomputes $\mathbf{h}$, then $\mathbf{s}$ via **Decompress**, and then $\mathbf{w}$. A series of conditions are then checked. Since both **Compress** and **Decompress** are public functions they do not affect security, and in Sections 5 and 6 we argue about a version of HAWK that does not perform these operations – i.e. we consider both **Compress** and **Decompress** to be the identity function on their first argument.

However, **Compress** and **Decompress** can affect correctness, whenever

$$\text{Decompress}(\text{Compress}(\mathbf{s}), \mathbf{h}, \mathbf{Q}) \neq \mathbf{s}. \quad (23)$$

By carefully selecting parameters we make sure this happens with a negligible probability.

3 Specifications

Here we give the specification for HAWK. In Section 3.1 we give more details regarding key generation, signing and verification. In Section 3.2, and in particular in Table 4, we give parameters for HAWK. In Section 3.3 we detail our encoding and decoding procedures. Finally, in Sections 3.4, 3.5 and 3.6 we detail key generation, signature generation and verification, respectively.

3.1 Overview

HAWK is a signature scheme with the following characteristics:

- Private keys, public keys and signatures use fixed size encodings. For public keys and signatures, a compression mechanism is used with variable output sizes, but padding is applied so that a fixed size specified in the scheme parameters is achieved. Key pair generation and signature generation are restarted if the public key or signature would exceed the target size.
- The signed data is processed once with SHAKE256; there is no random or key-dependent preamble, so that streamed processing of large messages with minimal RAM usage is possible, and the bulk of the data can be processed before knowing the public or private key (and even before knowing which HAWK parameter set is to be used).

Notation. We collect some additional notation that will be used throughout the formal specification.

The notation $x[i]$ is used to denote the i^{th} element of any sequence x of values. Indices always start at 0, i.e the first element of x is $x[0]$. The notation $x[j:k]$ stands for the subsequence of x for indices i such that $j \leq i < k$, i.e. the start index is included, but the end index is excluded, which corresponds to the notation used in several programming languages, such as Python, Go or Rust. Recall that the number of elements in a sequence x is denoted $\text{len}(x)$. For additional clarity, when x is a sequence of bits, we may write $\text{len}_{\text{bits}}(x)$ to denote the number of bits in x .

To go between bit sequences (of arbitrary lengths) and numbers, we define the **EncodeInt** and **DecodeInt** functions.

- For an integer $k \geq 0$ and a k -bit sequence $x_0x_1 \dots x_{k-1}$, we define

$$\text{DecodeInt}(x_0x_1 \dots x_{k-1}, k) = \sum_{i=0}^{k-1} x_i 2^i.$$

- For integers $k \geq 0$ and $0 \leq x < 2^k$, **EncodeInt**(x, k) is defined as the k -bit sequence $x_0x_1 \dots x_{k-1}$ for which $x = \text{DecodeInt}(x_0x_1 \dots x_{k-1}, k)$ holds.

For $k = 0$ we define “**EncodeInt**(0, 0)” to be the empty sequence. Moreover, for an integer x with $0 \leq x < 2^k$, we define $\text{rev}_k(x)$ as the integer associated to the bit-reversal of **EncodeInt**(x, k), i.e.,

$$\text{rev}_k \left(\sum_{j=0}^{k-1} x_j 2^j \right) = \sum_{j=0}^{k-1} x_j 2^{k-1-j}.$$

A cryptographically secure random source that outputs uniformly random bits is denoted **Rnd**. For an integer k , the notation **Rnd**(k) specifies the process of obtaining k new, freshly generated random bits.

3.1.1 Key Pairs

HAWK is defined over a degree n , which is a power of two (equal to 256, 512 or 1024). Most computations are performed on polynomials with integer coefficients, taken modulo $X^n + 1$. A HAWK private key is a randomly generated basis for the lattice $\mathbb{Z}^{2n}$, consisting of four polynomials f, g, F and G , where f and g have small coefficients and together they satisfy the NTRU equation

$$fG - gF = 1 \pmod{X^n + 1}, \quad (24)$$

i.e. the NTRU modulus is one. The lattice secret basis $\mathbf{B}$ and its inverse $\mathbf{B}^{-1}$ are

$$\mathbf{B} = \begin{pmatrix} f & F \\ g & G \end{pmatrix}, \quad \mathbf{B}^{-1} = \begin{pmatrix} G & -F \\ -g & f \end{pmatrix}. \quad (25)$$

The public key is $\mathbf{Q} = \mathbf{B}^* \mathbf{B}$, which is explicitly

$$\mathbf{Q} = \begin{pmatrix} q_{00} & q_{01} \\ q_{10} & q_{11} \end{pmatrix} = \begin{pmatrix} f^* f + g^* g & f^* F + g^* G \\ F^* f + G^* g & F^* F + G^* G \end{pmatrix}. \quad (26)$$

The matrix $\mathbf{Q}$ is self adjoint: $q_{00} = q_{00}^*$, $q_{11} = q_{11}^*$, and $q_{10} = q_{01}^*$. Moreover, a consequence of the NTRU equation is that $q_{00}q_{11} - q_{01}q_{10} = 1$ holds. We can thus always reconstruct the whole of $\mathbf{Q}$ from q_{00} and q_{01} only.

The generation of a new key pair is the following process.

- Generate **kgseed**, a random seed, from a cryptographically secure source of random bits.
- Sample the coefficients of f and g in a deterministic manner seeded with **kgseed** from the centred binomial distribution with the appropriate parameter.
- Verify a few conditions on f and g , e.g. that they both have odd parity and that the ℓ^2 norm of (f, g) is not too small. If the conditions are not fulfilled, restart with a new seed.
- Find polynomials F and G that complete the basis, i.e. such that $fG - gF = 1$. Because there may not exist any solution (F, G) , this step might fail. In that case, we restart with a new seed.
- Next, a well chosen multiple of (f, g) is subtracted from (F, G) to make (F, G) shorter. Optimised implementation methods, that combine completion and reduction, have been published previously [PP19, Por23]. This is the most expensive part of the key pair generation.
- The public key may also fail to be encodable within the target public key size defined in the scheme parameters; in that case, restart with a new seed.

The key pair generation process is detailed in Section 3.4. The actual encoding format of the private key only contains parts of the private elements, namely:

- **kgseed** (from which f and g can be regenerated using the specified deterministic process),
- $F \bmod 2$ and $G \bmod 2$ (i.e. the least significant bit of each coefficient of F and G), and
- **hpub**, a hash of the public key.

The public key itself encodes q_{00} and q_{01} , which are enough to represent the entire matrix $\mathbf{Q}$. Moreover, q_{00} being itself self adjoint, only half of its coefficients need to be encoded.

3.1.2 Signature Generation

Signature generation proceeds with the following steps.

- Decode the elements of the private key.
- Compute f, g from kgseed .
- Compute the hash $M = \text{SHAKE256}(\text{m} \parallel \text{hpub})$. Recall hpub is part of the private key.
- Generate salt , a random salt value.
- Compute the binary vector $\mathbf{h} = \text{SHAKE256}(M \parallel \text{salt})$ of size $2n$ bits (the two coordinates of $\mathbf{h}$ are the polynomials h_0 and h_1 , whose coefficients have value 0 or 1).
- Sample the vector $\mathbf{x}$ (of dimension $2n$) from a discrete Gaussian distribution, centred on zero and of width $2\sigma_{\text{sign}}$, over the lattice coset $\mathbb{Z}^{2n} + \mathbf{B}\mathbf{h}$. Note that here we are forming $\mathbf{B}$ from the recomputed (f, g) and $(F \bmod 2, G \bmod 2)$, which are sufficient to determine this coset. If the ℓ^2 norm of $\mathbf{x}$ exceeds a given threshold, restart with a new salt.
- Compute $\mathbf{w} = \mathbf{B}^{-1}\mathbf{x}$ and discard w_0 . In Algorithm 15 only w_1 is calculated.
- Check whether w_1 fulfils a symmetry breaking condition (**sym-break**). If not, replace w_1 by $-w_1$. This step prevents a simple weak forgery attack.
- Compute $s_1 = (h_1 - w_1)/2$. Note that we could have equivalently computed $\mathbf{s} = (\mathbf{h} - \mathbf{w})/2$ and then discarded s_0 . As such, this represents the application of **Compress** from the high level description in Algorithm 2.
- Encode salt and s_1 . The encoding fails when a coefficient of s_1 is too large or the encoded object becomes too large for the fixed target signature size. If this happens, restart signing with a new salt.

The signature generation process is fully specified in Section 3.5. Including only the second entry of the vector $\mathbf{s} = (s_0, s_1)$ in the signature encoding format makes signatures smaller, and also speeds up signature generation since only s_1 has to be computed. However, it then requires the signature verifier to recompute s_0 from the signed data, the public key and s_1 .

3.1.3 Signature Verification

The signature verification process is the following.

- Decode the public key $\mathbf{Q}$ and the two signature elements salt and s_1 from their respective formats.
- As in signature generation, recompute M as a hash over input data (m) and the hashed public key (hpub). Then recompute $\mathbf{h}$ as a hash over M and the salt (salt).
- Compute $w_1 = h_1 - 2 \cdot s_1$. This equals w_1 from the discussion in Section 3.1.2 above.
- Verify that **sym-break**(w_1) is fulfilled.
- Recompute s_0 using

$$s_0 = \left\lceil \frac{h_0}{2} + \frac{q_{01}}{q_{00}} \left(\frac{h_1}{2} - s_1 \right) \right\rceil. \quad (27)$$

Table 4: Parameter sets for HAWK. Note that key and signature sizes are specified in bytes.

Name	HAWK-256	HAWK-512	HAWK-1024
Targeted security	Challenge	NIST-I	NIST-V
Bit security λ	64	128	256
Private key size $\text{privlen}_{\text{bits}}/8$ (bytes)	96	184	360
Public key size $\text{publen}_{\text{bits}}/8$ (bytes)	450	1024	2440
Signature size $\text{siglen}_{\text{bits}}/8$ (bytes)	249	555	1221
Degree n	256	512	1024
Transcript size limit q_s	2^{32}	2^{64}	2^{64}
Centred binomial η for sampling (f, g)	2	4	8
Signature std. dev. σ_{sign}	1.010	1.278	1.299
Verification std. dev. σ_{verify}	1.042	1.425	1.571
Key recovery std. dev. σ_{krsec}	1.042	1.425	1.974
Salt length $\text{saltlen}_{\text{bits}}$	112	192	320
Keygen. seed length $\text{kgseedlen}_{\text{bits}}$	128	192	320
Hashed pubkey length $\text{hpublen}_{\text{bits}}$	128	256	512
Bit lengths for q_{00} ($\text{low}_{00}, \text{high}_{00}$)	5, 9	5, 9	6, 10
Bit lengths for q_{01} ($\text{low}_{01}, \text{high}_{01}$)	8, 11	9, 12	10, 14
Bit length for q_{11} (high_{11})	13	15	17
Bit length for s_0 (high_{s0})	12	13	14
Bit lengths for s_1 ($\text{low}_{s1}, \text{high}_{s1}$)	5, 9	5, 9	6, 10
Signature decompression bound β_0	1/250	1/1000	1/3000

This recomputation will be performed with integer computations using a fixed-point interpretation of 32-bit integers. This represents **Decompress** from the high level description in Algorithm 3.

- Compute $w_0 = h_0 - 2 \cdot s_0$. In the implementation this step and the one above both happen internally in **RebuildS0** in Algorithm 20.
- Let $\mathbf{w} = (w_0, w_1)$. Verify that $\|\mathbf{w}\|_{\mathbf{Q}}$ is not too long.

Signature verification is detailed in Section 3.6. Compared to the high level description of Algorithm 3 the length of the decoded salt and $\mathbf{s} \in R_n^2$ are not checked explicitly. Instead, the decoding procedure only gives a salt of the desired length from the signature and an $s_1 \in R_n$ by construction of the encoding format. By design, s_0 is rebuilt as an integer by (27), so $\mathbf{s} \in R_n^2$ is automatically satisfied.

The scheme parameters and keys are chosen such that the probability, that (27) fails to recover s_0 , is negligible.

3.2 Parameters

HAWK is defined for three sets of parameters, shown in Table 4. The name of a HAWK parameter set is derived from the degree n , which is taken to be a power of two.

The HAWK-256 parameter set is a “challenge” because its security level, estimated at “around 64 bits”, is too low for submission to NIST. However, it is a good cryptanalytic target, as breaking it is not immediate and a forgery or private key recovery would provide interesting data. At most 2^{32} signatures may be “safely” generated within the target security level of HAWK-256, as opposed to 2^{64} for HAWK-512 and HAWK-1024.

HAWK-512 and HAWK-1024 are meant to fulfil NIST-I and NIST-V security levels, respectively.

3.3 Encoding and Decoding

Encoding and decoding rules are used in HAWK for representing keys and signatures, but also internally for injecting integers as input to SHAKE256, and interpreting the SHAKE256 output as a sequence of 64-bit integers.

3.3.1 Bit Ordering

All input and output operations are described as working over sequences of bits. Such a sequence is written in left-to-right order. The “ $\parallel$ ” operator denotes concatenation. Most software implementations work over *bytes* (octets); in that case, it is assumed that bits have been grouped into bytes in a way compatible with algorithm *h2b* from section B.1 of FIPS 202 [FIP15], which itself follows the rules from [Kec11]: if eight successive bits are $x_0x_1x_2\dots x_7$, then they encode the numerical byte value $x = \sum_{i=0}^7 x_i 2^i$. We denote the mapping of a byte into 8 bits by $\text{EncodeInt}(x, 8) = x_0 \parallel x_1 \parallel x_2 \parallel \dots \parallel x_7$.

A potential source of confusion is the terminology of shift operators available in many programming languages: if some bits in a sequence should be moved *leftward*, then this corresponds to a *right shift* on the integer value corresponding to this sequence through the mapping embodied by EncodeInt and DecodeInt . Within this specification, we do not use the “left” and “right” directions; such operations on integers are expressed as multiplications or divisions by powers of two.

3.3.2 SHAKE256w and SHAKE256x4

SHAKE256, as specified in FIPS 202 [FIP15], outputs a sequence of bits. We define SHAKE256w as SHAKE256, with the output bits grouped into 64-bit words interpreted as integers. Specifically, for a binary input m , we define $\text{SHAKE256w}(m)$ such that, for any $i \geq 0$,

$$\text{SHAKE256w}(m)[i] = \sum_{j=0}^{63} \text{SHAKE256}(m)[64i + j] \cdot 2^j \quad (28)$$

$$= \text{DecodeInt}(\text{SHAKE256}(m)[64i : 64i + 64]). \quad (29)$$

Internally, SHAKE256 implementations that do not use the “bit interleaving” strategy usually keep the sponge state as an array of 25 64-bit words, and the output of SHAKE256 is really a little-endian encoding of some of these state words; the word output stream defined here is then exactly the sequence of these words.

We then define the SHAKE256x4 function, which is four SHAKE256w instances working on almost the same input, and whose respective outputs are interleaved with word granularity. Namely, for a binary input m ,

$$\text{SHAKE256x4}(m)[4i + j] = \text{SHAKE256w}(m \parallel \text{EncodeInt}(j, 8))[i]$$

for all $i \geq 0$, and $0 \leq j \leq 3$.

An implementation of HAWK running on an architecture offering SIMD opcodes may choose to run some of these internal SHAKE256 instances in parallel; the interleaving naturally corresponds to the effect of running four parallel instances with 256-bit SIMD registers, each register containing one SHAKE256 state 64-bit word for each of the four SHAKE256 instances. However, it is also possible to run the SHAKE256 instances sequentially, reusing the same memory space for the successive SHAKE256 states; all usages of SHAKE256x4 output within HAWK have been designed specifically to allow such RAM efficient usage on small embedded systems with severe RAM size constraints.

3.3.3 Low Bit Extraction

Given a polynomial u with integer coefficients, $u \bmod 2$ denotes the polynomial such that $(u \bmod 2)[i] = u[i] \bmod 2$ for $0 \leq i < n$; i.e. each coefficient of $u \bmod 2$ is the least significant bit (with value 0 or 1) of the corresponding coefficient of u .

Since polynomials are also sequences of their coefficients, a binary polynomial $u \bmod 2$ is also a sequence of n bits ($u[0] \bmod 2$ to $u[n-1] \bmod 2$) and can be used as such in encoding and decoding procedures.

3.3.4 Private Key Encoding

A HAWK private key, mathematically, consists of four polynomials f , g , F and G . As will be detailed in Section 3.4, f and g are generated from a random seed (kgseed), and only $F \bmod 2$ and $G \bmod 2$ are needed for signature generation. The encoding of the private key is thus the concatenation of kgseed , $F \bmod 2$, $G \bmod 2$, and hpub , the latter being a hash of the public key. This is expressed in `EncodePrivate` (Algorithm 4) and `DecodePrivate` (Algorithm 5). Note that the lengths of kgseed and hpub are fixed: for a given HAWK parameter set, they are defined in the scheme parameters (Table 4).

Algorithm 4 `EncodePrivate`: Private key encoding

Require: Private seed kgseed , polynomials $(F \bmod 2, G \bmod 2)$, hashed public key hpub

Ensure: Encoded private key

1: **return** $\text{kgseed} \parallel (F \bmod 2) \parallel (G \bmod 2) \parallel \text{hpub}$

Algorithm 5 `DecodePrivate`: Private key decoding

Require: Encoded private key priv

Ensure: Private seed kgseed , polynomials $(F \bmod 2, G \bmod 2)$, and hashed public key hpub

1: $\text{kgseed} \leftarrow \text{priv}[0 : \text{kgseedlen}_{\text{bits}}]$
2: $F \bmod 2 \leftarrow \text{priv}[\text{kgseedlen}_{\text{bits}} : \text{kgseedlen}_{\text{bits}} + n]$
3: $G \bmod 2 \leftarrow \text{priv}[\text{kgseedlen}_{\text{bits}} + n : \text{kgseedlen}_{\text{bits}} + 2n]$
4: $\text{hpub} \leftarrow \text{priv}[\text{kgseedlen}_{\text{bits}} + 2n : \text{kgseedlen}_{\text{bits}} + 2n + \text{hpublen}_{\text{bits}}]$
5: **return** $(\text{kgseed}, F \bmod 2, G \bmod 2, \text{hpub})$

3.3.5 Golomb–Rice Compression

Polynomials in HAWK public keys and signatures use a variable length encoding with Golomb–Rice compression of coefficients. Each integer element is split into a sign bit, a fixed length part comprising the low order bits of the value, and a variable length part that encodes the high order bits of the value. For a sequence of k values to encode, the k sign bits are first produced, followed by the k low parts of the values, then the k high parts. In all uses of this encoding in HAWK, k is a multiple of 8, ensuring that the sign bit chunk and the low part chunk start on byte boundaries.

The `CompressGR` function (Algorithm 6) takes as input a sequence of integers x (of length k , such that $k \equiv 0 \pmod 8$), as well as two sizes, `low` and `high`; the coefficients of x must be such that $-2^{\text{high}} \leq x[i] < 2^{\text{high}}$ for all $0 \leq i < k$.

In plain words, for each integer value $z = x[i]$ we perform the following:

- A sign bit is emitted, of value 0 if $z \geq 0$, or 1 if $z < 0$. In the latter case, z is replaced with its ones' complement $-z - 1$ (in a typical software implementation,

Algorithm 6 CompressGR: Golomb–Rice compression**Require:** Integer sequence $x[0:k]$, sizes **low** and **high****Ensure:** A compressed encoding y (sequence of bits), or $\perp$ on error

```

1: Set  $y$  to an empty sequence of bits.
2: Set  $v$  to an empty sequence of integers.
3: for  $i = 0$  to  $k - 1$  do
4:    $s \leftarrow 1$  if  $x[i] < 0$ , or  $0$  if  $x[i] \geq 0$ 
5:    $y \leftarrow y \parallel s$ 
6:    $v \leftarrow v \parallel x[i] - s(2x[i] + 1)$ 
7:   if  $v[i] \geq 2^{\text{high}}$  then
8:     return  $\perp$ 
9: for  $i = 0$  to  $k - 1$  do
10:   $y \leftarrow y \parallel \text{EncodeInt}(v[i] \bmod 2^{\text{low}}, \text{low})$ 
11: for  $i = 0$  to  $k - 1$  do
12:   $y \leftarrow y \parallel \text{EncodeInt}(0, \lfloor v[i]/2^{\text{low}} \rfloor) \parallel 1$ 
13: return  $y$ 

```

this is equivalent to a XOR between z and $-s$, where s is the sign bit). Note that now $z \geq 0$.

- The lowest bits of z are emitted as is; this is the fixed size part. Then z is scaled down by **low** bits (floored division by 2^{low}).
- Finally, exactly z bits of value 0 are emitted, followed by a bit of value 1 which marks the end of the encoding for that integer. This is the variable size part.

In all usages of **CompressGR** in **HAWK**, the maximum size is such that $\text{high} \leq \text{low} + 4$. As a consequence the variable size part of an encoded integer cannot be longer than 16 bits (including the terminating 1).

Algorithm 6 returns a failure indicator ($\perp$) if the input is invalid, i.e. at least one of the input integers is not in the expected range. This check may be omitted if the input is known to be in the proper range (e.g. because it has already been tested beforehand).

Decompression is performed with **DecompressGR** (Algorithm 7). Decompression can return a failure (denoted $\perp$) if the input is invalid. On success, the resulting integer sequence is returned, as well as the actual encoded length (in bits); it is up to the caller to verify that subsequent bits (e.g. padding), if any, have the expected value.

3.3.6 Public Key Encoding

A **HAWK** public key consists of four polynomials: q_{00} , q_{01} , q_{10} and q_{11} . As explained in Section 3.1.1 we only need to encode q_{00} and q_{01} in the public key. Moreover, q_{00} is self adjoint, so only its first $n/2$ coefficients must be serialized. The **EncodePublic** function (Algorithm 8) returns the encoded format for a public key.

All coefficients of q_{00} are lower than $2^{\text{high}_{00}}$, except the first coefficient, $q_{00}[0]$, which may be larger, and is thus treated specially by being first scaled down by a few bits; its low bits are then appended to the encoding of q_{00} , followed by up to 7 extra bits (of value 0) to ensure that the encoding of q_{01} starts on a byte boundary. Padding bits (also of value 0) are finally placed after q_{01} so that the total encoded size matches the expected value. The key pair generation process should ensure that the encoded public key can indeed fit in the target size. If not, the key should be discarded, and a new one generated.

The corresponding decoding algorithm is **DecodePublic** (Algorithm 9). It is noteworthy that **DecodePublic** enforces that the exact input length should match the public key length

Algorithm 7 DecompressGR: Golomb–Rice decomposition**Require:** Bit sequence y , output length $k \equiv 0 \pmod 8$, sizes low and high**Ensure:** Integers $x[0:k]$ and encoded length j , or $\perp$ (failure)

```

1: if  $\text{len}_{\text{bits}}(y) < k(\text{low} + 2)$  then
2:   return  $\perp$ 
3: for  $i = 0$  to  $k - 1$  do
4:    $x[i] \leftarrow \text{DecodeInt}(y[i \cdot \text{low} + k : (i + 1) \cdot \text{low} + k])$ 
5:  $j \leftarrow k(\text{low} + 1)$ 
6: for  $i = 0$  to  $k - 1$  do
7:    $z \leftarrow -1$ 
8:   repeat
9:      $z \leftarrow z + 1$ 
10:    if  $j \geq \text{len}_{\text{bits}}(y)$  or  $z \geq 2^{\text{high} - \text{low}}$  then
11:      return  $\perp$ 
12:     $t \leftarrow y[j]$ 
13:     $j \leftarrow j + 1$ 
14:  until  $t = 1$ 
15:   $x[i] \leftarrow x[i] + z \cdot 2^{\text{low}}$ 
16: for  $i = 0$  to  $k - 1$  do
17:    $x[i] \leftarrow x[i] - y[j](2x[i] + 1)$  ▷ Application of the sign bit.
18: return  $(x, j)$ 

```

Algorithm 8 EncodePublic: Public key encoding**Require:** Public polynomials q_{00} and q_{01} **Ensure:** Encoded public key, or $\perp$ on error

```

1: if  $q_{00}[0] < -2^{15}$  or  $q_{00}[0] \geq 2^{15}$  then
2:   return  $\perp$ 
3:  $v \leftarrow 16 - \text{high}_{00}$ 
4:  $q'_{00} \leftarrow q_{00}$ 
5:  $q'_{00}[0] \leftarrow \lfloor q_{00}[0] / 2^v \rfloor$ 
6:  $y_{00} \leftarrow \text{CompressGR}(q'_{00}[0 : n/2], \text{low}_{00}, \text{high}_{00})$  ▷ Only  $n/2$  coefficients.
7: if  $y_{00} = \perp$  then
8:   return  $\perp$ 
9:  $y_{00} \leftarrow y_{00} \parallel \text{EncodeInt}(q_{00}[0] \bmod 2^v, v)$ 
10: while  $\text{len}_{\text{bits}}(y_{00}) \not\equiv 0 \pmod 8$  do
11:    $y_{00} \leftarrow y_{00} \parallel 0$  ▷ Padding to the next byte boundary.
12:  $y_{01} \leftarrow \text{CompressGR}(q_{01}, \text{low}_{01}, \text{high}_{01})$  ▷ All  $n$  coefficients are encoded.
13: if  $y_{01} = \perp$  then
14:   return  $\perp$ 
15:  $y \leftarrow y_{00} \parallel y_{01}$ 
16: if  $\text{len}_{\text{bits}}(y) > \text{publen}_{\text{bits}}$  then
17:   return  $\perp$ 
18: while  $\text{len}_{\text{bits}}(y) < \text{publen}_{\text{bits}}$  do
19:    $y \leftarrow y \parallel 0$  ▷ Padding to  $\text{publen}_{\text{bits}}$  bits.
20: return  $y$ 

```

$\text{publen}_{\text{bits}}$, as defined in Table 4, and all padding bits (both between q_{00} and q_{01} , and after q_{01}) should be zero. A given public key has a single valid encoding, and none other shall be accepted by `DecodePublic`.

Algorithm 9 `DecodePublic`: Public key decoding

Ensure: Encoded public key y

Require: Public polynomials q_{00} and q_{01} , or $\perp$ on error.

```

1: if  $\text{len}_{\text{bits}}(y) \neq \text{publen}_{\text{bits}}$  then
2:   return  $\perp$ 
3:  $v \leftarrow 16 - \text{high}_{00}$ 
4:  $r_{00} \leftarrow \text{DecompressGR}(y, n/2, \text{low}_{00}, \text{high}_{00})$ 
5: if  $r_{00} = \perp$  then
6:   return  $\perp$ 
7:  $(q_{00}, j) \leftarrow r_{00}$ 
8: if  $\text{len}_{\text{bits}}(y) < j + v$  then
9:   return  $\perp$ 
10:  $q_{00}[0] \leftarrow 2^v q_{00}[0] + \text{DecodeInt}(y[j : j + v])$ 
11:  $j \leftarrow j + v$ 
12: while  $j \not\equiv 0 \pmod{8}$  do
13:   if  $j \geq \text{len}_{\text{bits}}(y)$  or  $y[j] \neq 0$  then
14:     return  $\perp$ 
15:    $j \leftarrow j + 1$ 
16:  $q_{00}[n/2] \leftarrow 0$ 
17: for  $i = n/2 + 1$  to  $n - 1$  do
18:    $q_{00}[i] \leftarrow -q_{00}[n - i]$ 
19:  $r_{01} \leftarrow \text{DecompressGR}(y[j : \text{len}_{\text{bits}}(y)], n, \text{low}_{01}, \text{high}_{01})$ 
20: if  $r_{01} = \perp$  then
21:   return  $\perp$ 
22:  $(q_{01}, j') \leftarrow r_{01}$ 
23:  $j \leftarrow j + j'$ 
24: while  $j < \text{len}_{\text{bits}}(y)$  do
25:   if  $y[j] \neq 0$  then
26:     return  $\perp$ 
27:    $j \leftarrow j + 1$ 
28: return  $(q_{00}, q_{01})$ 

```

3.3.7 Signature Encoding

A HAWK signature consists of a salt value (`salt`) and a polynomial s_1 . These two elements are encoded and concatenated in the function `EncodeSignature` (Algorithm 10) to yield the signature. Again, zeros are padded to produce a fixed length signature. If the signature would be bigger than the fixed length, encoding fails. However, the failure is resolved in signing by restarting the signing process, which is a randomised process. The parameters for HAWK have been chosen so that such failures are rare, see Section 3.5.

The corresponding decoding algorithm is `DecodeSignature` (Algorithm 11). Similarly to public keys, decoding enforces a signature to be of size $\text{siglen}_{\text{bits}}$ from Table 4 and all padding bits at the end should be 0.

Algorithm 10 EncodeSignature: Signature encoding

Require: Salt salt and signature polynomial s_1

Ensure: Encoded signature, or $\perp$ on error

```

1:  $y \leftarrow \text{CompressGR}(s_1, \text{low}_{s_1}, \text{high}_{s_1})$ 
2: if  $y = \perp$  or  $\text{len}_{\text{bits}}(y) > \text{siglen}_{\text{bits}} - \text{saltlen}_{\text{bits}}$  then
3:   return  $\perp$ 
4: while  $\text{len}_{\text{bits}}(y) < \text{siglen}_{\text{bits}} - \text{saltlen}_{\text{bits}}$  do
5:    $y \leftarrow y \parallel 0$ 
6: return  $\text{salt} \parallel y$ 

```

Algorithm 11 DecodeSignature: Signature decoding

Require: Encoded signature y

Ensure: Salt salt and polynomial s_1 , or $\perp$ on error.

```

1: if  $\text{len}_{\text{bits}}(y) \neq \text{siglen}_{\text{bits}}$  then
2:   return  $\perp$ 
3:  $\text{salt} \leftarrow y[0 : \text{saltlen}_{\text{bits}}]$ 
4:  $r \leftarrow \text{DecompressGR}(y[\text{saltlen}_{\text{bits}} : \text{len}_{\text{bits}}(y)], n, \text{low}_{s_1}, \text{high}_{s_1})$ 
5: if  $r = \perp$  then
6:   return  $\perp$ 
7:  $(s_1, j) \leftarrow r$ 
8:  $j \leftarrow j + \text{saltlen}_{\text{bits}}$ 
9: while  $j < \text{len}_{\text{bits}}(y)$  do
10:  if  $y[j] \neq 0$  then
11:    return  $\perp$ 
12:   $j \leftarrow j + 1$ 
13: return  $(\text{salt}, s_1)$ 

```

3.4 Key Pair Generation

The HAWK key pair generation process consists of trying random candidates for two polynomials (f, g) with a given distribution, then for each suitable candidate, solving the NTRU equation $fG - gF = 1$. If an appropriate solution is found, and the resulting public key can be encoded within the size defined in the HAWK parameters, then that key pair is kept; otherwise, the candidate (f, g) is discarded, and the process loops.

The polynomials f and g are sampled from a centred binomial distribution using pseudorandom bits obtained from a SHAKE256x4 instance initialised over a random seed (kgseed). This process is implemented by the function **Regeneratefg** (Algorithm 12). For a given seed value, the function is deterministic and the output (f, g) is fully specified, so kgseed is sufficient for the private key storage.

Algorithm 12 **Regeneratefg**: Regenerate (f, g)

Require: Key generation seed kgseed

Ensure: Polynomials (f, g)

```

1:  $b \leftarrow n/64$   $\triangleright b = 4, 8 \text{ or } 16$ , depending on  $n$ .
2:  $y \leftarrow \text{SHAKE256x4}(\text{kgseed})[0 : 2bn]$   $\triangleright b$  bits for each coefficient of  $f$  and  $g$ .
3: for  $i = 0$  to  $n - 1$  do
4:    $f[i] \leftarrow \left( \sum_{j=0}^{b-1} y[ib + j] \right) - b/2$   $\triangleright$  centred binomial with  $\eta = b/2$ .
5: for  $i = 0$  to  $n - 1$  do
6:    $g[i] \leftarrow \left( \sum_{j=0}^{b-1} y[(i+n)b + j] \right) - b/2$ 
7: return  $(f, g)$ 
```

The key pair generation process is defined in the function **HawkKeyGen** (Algorithm 13), which makes use of the following two helper functions.

- **IsInvertible** (u, p) returns **true** if the integer polynomial u is invertible modulo $X^n + 1$ and modulo the provided prime p (i.e. considering the coefficients of u as elements in the field $\mathbb{Z}/p\mathbb{Z}$), and **false** otherwise. When $p = 2$, this simplifies to a parity check: **IsInvertible** $(u, 2)$ returns **true** if and only if $\sum_{i=0}^{n-1} u[i] \equiv 1 \pmod{2}$. In **HawkKeyGen**, this function is also used for two specific 31-bit moduli amenable to efficient implementations using the NTT.
- **NTRUSolve** (f, g, q) , for polynomials (f, g) and integer q , return polynomials (F, G) such that $fG - gF = q$, and (F, G) have somewhat small coefficients. A solution does not necessarily exist, and when solutions do exist **NTRUSolve** may fail to find one, in which case the function returns $\perp$. Note that **NTRUSolve** may succeed, but (F, G) may not be short enough, in which case **HawkKeyGen** also restarts. In [PP19], the algorithms **TowerSolverR** and **TowerSolverI** are presented, which can both be used to implement **NTRUSolve**.

Implementation strategies for these helper functions are discussed in more detail in Section 4.1.

In summary, a HAWK private key describes four secret integer polynomials f, g, F and G (taken modulo $X^n + 1$). The vector (f, g) is of small norm and satisfies

$$\|(f, g)\|_\infty \leq n/128, \text{ and } \|(f, g)\|^2 > 2n\sigma_{\text{krsec}}^2. \quad (30)$$

Then, **NTRUSolve** comes up with (F, G) such that

$$\|(F, G)\|_\infty \leq 127, \text{ and } fG - gF = 1. \quad (31)$$

Algorithm 13 HawkKeyGen: HAWK key pair generation

Require: Cryptographically secure source of random bits

Ensure: New key pair (priv, pub)

```

1: kgseed  $\leftarrow$  Rnd(kgseedlenbits)  $\triangleright$  kgseedlenbits is defined in Table 4.
2:  $(f, g) \leftarrow$  Regeneratefg(kgseed)
3: if IsInvertible( $f, 2$ )  $\neq$  true or IsInvertible( $g, 2$ )  $\neq$  true then
4:   restart
5: if  $\|(f, g)\|^2 \leq 2n\sigma_{\text{krsec}}^2$  then
6:   restart
7:  $q_{00} \leftarrow ff^* + gg^*$ 
8:  $(p_1, p_2) \leftarrow (2147473409, 2147389441)$ 
9: if IsInvertible( $q_{00}, p_1$ )  $\neq$  true or IsInvertible( $q_{00}, p_2$ )  $\neq$  true then
10:  restart
11: if  $(1/q_{00})[0] \geq \beta_0$  then  $\triangleright$  Inverse over  $\mathbb{Q}[X]/(X^n + 1)$ .
12:  restart
13:  $r \leftarrow$  NTRUSolve( $f, g, 1$ )
14: if  $r = \perp$  then
15:  restart
16:  $(F, G) \leftarrow r$ 
17: if  $\|(F, G)\|_\infty > 127$  then
18:  restart
19:  $q_{01} \leftarrow Ff^* + Gg^*$ 
20:  $q_{11} \leftarrow FF^* + GG^*$ 
21: if  $|q_{11}[i]| \geq 2^{\text{high}_{11}}$  for any  $i > 0$  then
22:  restart
23: pub  $\leftarrow$  EncodePublic( $q_{00}, q_{01}$ )
24: if pub  $= \perp$  then
25:  restart
26: hpub  $\leftarrow$  SHAKE256(pub)  $\triangleright$  hpublenbits is defined in Table 4.
27: priv  $\leftarrow$  EncodePrivate(kgseed,  $F \bmod 2, G \bmod 2, \text{hpub}$ )
28: return (priv, pub)

```

A HAWK public key describes the four elements q_{00}, q_{01}, q_{10} and q_{11} that make up $\mathbf{Q}$. The key generation `HawkKeyGen` ensures that

$$-2^{15} \leq q_{00}[0] < +2^{15}, \quad (32)$$

$$-2^{\text{high}_{00}} < q_{00}[i] < +2^{\text{high}_{00}} \quad \text{for } i > 0, \quad (33)$$

$$-2^{\text{high}_{01}} < q_{01}[i] < +2^{\text{high}_{01}} \quad \text{for all } i, \quad (34)$$

$$-2^{\text{high}_{11}} < q_{11}[i] < +2^{\text{high}_{11}} \quad \text{for } i > 0. \quad (35)$$

3.5 Signature Generation

As specified in Section 3.1.2, generating a signature requires to sample from a discrete Gaussian with support $2\mathbb{Z}$ or $2\mathbb{Z} + 1$. First, we detail the discrete Gaussian sampler used in signature generation. Then, we explain all the steps done in signature generation.

3.5.1 Discrete Gaussian Sampling

Signature generation nominally entails sampling a vector $\mathbf{x}$ from a discrete Gaussian distribution over the lattice coset $2\mathbb{Z}^{2n} + \mathbf{t}$ (with the coefficients of $\mathbf{t}$ taking values 0 or 1), centred on zero and with width $2\sigma_{\text{sign}}$.

Recall the discrete Gaussian distribution given in (4). For any $b \in \{0, 1\}$ and $z \in \mathbb{R}$, let $P_b(z)$ denote the probability that $|X| \geq z$ when $X \sim D_{2\mathbb{Z}+b, 2\sigma_{\text{sign}}}$. From P_0 and P_1 , we then define two tables of integers, indexed by an integer $k \in \mathbb{Z}_{\geq 0}$, that use an up-scaling by a factor 2^{78} :

$$T_0[k] = \lfloor 2^{78} \cdot P_0(2 + 2k) \rfloor, \quad (36)$$

$$T_1[k] = \lfloor 2^{78} \cdot P_1(3 + 2k) \rfloor. \quad (37)$$

These integers are provided in hexadecimal format in Table 5.

Using tables T_0 and T_1 , we define the function `SamplerSign` (Algorithm 14) which, given an initial seeding input `seed` (a sequence of bits) and a centre vector $\mathbf{t}$ (whose coefficients are 0 or 1), samples a vector $\mathbf{x}$ from a distribution that approximates $D_{2\mathbb{Z}^{2n} + \mathbf{t}, 2\sigma_{\text{sign}}}$. We discuss the conditions we require on this approximate distribution more in Section 6.7. Here $\mathbf{x}$ is considered to be a sequence of $2n$ integers.

The following notes apply to Algorithm 14:

- SHAKE256x4 is used to produce pseudorandom 64-bit words. Since each sampled value consumes 80 bits, five words are needed for every four output values. A total of $5n/2$ words ($160n$ bits) is thus needed for the $2n$ output values.
- SHAKE256x4 runs four SHAKE256 instances in parallel. The indexing used above ensures that each output value uses bits from only one of these four instances, so that the SHAKE256 instances may be evaluated sequentially on RAM-starved architectures. The j index is really a designator of the relevant SHAKE256 instance within SHAKE256x4. The first SHAKE256 instance ($j = 0$) produces words used for output values 0, 1, 2, 3, then 16, 17, 18, 19, and so on; the second SHAKE256 instance ($j = 1$) produces words used for output values 4, 5, 6, 7, then 20, 21, 22, 23, and so on; the third SHAKE256 instance ($j = 2$) produces words used for output values 8, 9, 10, 11, then 24, 25, 26, 27, and so on; and the fourth SHAKE256 instance ($j = 3$) produces words used for output values 12, 13, 14, 15, then 28, 29, 30, 31, and so on.
- For each group of five successive words q_0 to q_4 produced by a single SHAKE256 instance, we can sample four values. Each value needs a 78-bit integer z and a sign bit. Words q_0 to q_3 contribute the low 63 bits of each z , and the sign bits; word q_4 contains the high 15 bits of each z . Four of the bits of q_4 are discarded.

Table 5: Cumulative distribution tables for sampling used in signature generation.

Name	T_0	T_1
HAWK-256	0x26B871FBD58485D45050	0x13459408A4B181C718B1
	0x07C054114F1DC2FA7AC9	0x027D614569CC54722DC9
	0x00A242F74ADDA0B5AE61	0x0020951C5CDCBAFF49A3
	0x0005252E2152AB5D758B	0x0000A3460C30AC398322
	0x00000FDE62196C1718FC	0x000001355A8330C44097
	0x000000127325DDF8CEBA	0x00000000DC8DE401FD12
	0x0000000008100822C548	0x00000000003B0FFB28F0
	0x00000000000152A6E9AE	0x00000000000005EFCDD9
	0x00000000000000014DA4A	0x000000000000000003953
	0x00000000000000000007B	0x000000000000000000000
HAWK-512	0x2C058C27920A04F8F267	0x1AFCBC689D9213449DC9
	0x0E9A1C4FF17C204AA058	0x06EBFB908C81FCE3524F
	0x02DBDE63263BE0098FFD	0x01064EBEFD8FF4F07378
	0x005156AEDFB0876A3BD8	0x0015C628BC6B23887196
	0x0005061E21D588CC61CC	0x0000FF769211F07B326F
	0x00002BA568D92EEC18E7	0x00000668F461693DFF8F
	0x000000CF0F8687D3B009	0x0000001670DB65964485
	0x0000000216A0C344EB45	0x000000002AB6E11C2552
	0x0000000002EDF0B98A84	0x00000000002C253C7E81
	0x0000000000023AF3B2E7	0x00000000000018C14ABF
	0x0000000000000000EBC6A	0x00000000000000007876E
	0x00000000000000000034CF	0x00000000000000000013D
	0x000000000000000000006	0x000000000000000000000
HAWK-1024	0x2C583AAA2EB76504E560	0x1B7F01AE2B17728DF2DE
	0x0F1D70E1C03E49BB683E	0x07506A00B82C69624C93
	0x031955CDA662EF2D1C48	0x01252685DB30348656A4
	0x005E31E874B355421BB7	0x001A430192770E205503
	0x000657C0676C029895A7	0x00015353BD4091AA96DB
	0x00003D4D67696E51F820	0x000009915A53D8667BEE
	0x0000014A1A8A93F20738	0x00000026670030160D5F
	0x00000003DAF47E8DFB21	0x00000000557CD1C5F797
	0x0000000006634617B3FF	0x00000000006965E15B13
	0x000000000005DBEFB646	0x00000000000047E9AB38
	0x00000000000002F93038	0x000000000000001B2445
	0x0000000000000000D5A7	0x000000000000000005AA
	0x000000000000000000021	0x000000000000000000000

Algorithm 14 SamplerSign: Gaussian sampling in HAWK signature generation

Require: Seeding input *seed*, centre vector *t***Ensure:** *x*

```

1:  $y \leftarrow \text{SHAKE256x4}(s)[0 : 5n/2]$   $\triangleright 5n/2$  64-bit words =  $160n$  bits
2: for  $j = 0$  to  $3$  do
3:   for  $i = 0$  to  $n/8 - 1$  do
4:     for  $k = 0$  to  $3$  do
5:        $r \leftarrow 16i + 4j + k$ 
6:        $a \leftarrow y[j + 4(5i + k)]$ 
7:        $b \leftarrow \lfloor y[j + 4(5i + 4)] / 2^{16k} \rfloor \bmod 2^{15}$ 
8:        $c \leftarrow (a \bmod 2^{63}) + 2^{63}b$ 
9:        $(v_0, v_1) \leftarrow (0, 0)$ 
10:       $z \leftarrow 0$ 
11:      while  $T_0[z] \neq 0$  or  $T_1[z] \neq 0$  do
12:        if  $c < T_0[z]$  then
13:           $v_0 \leftarrow v_0 + 1$ 
14:        if  $c < T_1[z]$  then
15:           $v_1 \leftarrow v_1 + 1$ 
16:         $z \leftarrow z + 1$ 
17:      if  $t[r] = 0$  then
18:         $v \leftarrow 2v_0$ 
19:      else
20:         $v \leftarrow 2v_1 + 1$ 
21:      if  $a \geq 2^{63}$  then
22:         $v \leftarrow -v$ 
23:       $x[r] \leftarrow v$ 
24: return d

```

- The actual sampling is done by counting how many elements of T_0 and T_1 are strictly greater than z ; thus, lower z values yield larger outputs. Ultimately, only one of the two results v_0 and v_1 is retained, depending on the corresponding bit in $\mathbf{t}$; Algorithm 14 still uses *both* tables because that corresponds to what a constant-time implementation of HAWK should do: the $\mathbf{t}$ vector is secret, and thus the memory access pattern should not depend on whether $\mathbf{t}[r]$ is 0 or 1. Similarly, all table elements should be used each time (no early abort), and the application of the sign bit (conditional replacement of v with $-v$) should be done in a constant-time way.
- The number of iterations of the inner “while” loop depends on the scheme parameters. The T_0 and T_1 tables formally contain an infinite number of entries, but they only contain zeros beyond a certain index that depends on the scheme parameters. As seen in Table 5, with the currently defined parameter sets, only up to at most 13 iterations are needed.

3.5.2 Generating signatures

Using `SamplerSign`, we can now define the `HawkSign` function (Algorithm 15), which generates a HAWK signature on an arbitrary message. `HawkSign` also uses the `sym-break` function, defined as follows, for a given polynomial w :

- If there exists an index i such that $w[i] > 0$, and $w[j] = 0$ for all $j < i$, then `sym-break( $w$ ) = true`.
- Otherwise, `sym-break( $w$ ) = false`.

In other words, `sym-break( $w$ )` is `true` only if w is not zero, and its first non-zero coefficient is positive.

Algorithm 15 HawkSign: HAWK signature generation

Require: Message m , private key priv , secure random source

Ensure: Signature sig

```

1:  $(\text{kgseed}, F \bmod 2, G \bmod 2, \text{hpub}) \leftarrow \text{DecodePrivate}(\text{priv})$ 
2:  $(f, g) \leftarrow \text{Regeneratefg}(\text{kgseed})$ 
3:  $M \leftarrow \text{SHAKE256}(m \parallel \text{hpub})[0 : 512]$   $\triangleright M$  has size 64 bytes.
4:  $a \leftarrow 0$ 
5: loop
6:    $\text{salt} \leftarrow \text{SHAKE256}(M \parallel \text{kgseed} \parallel \text{EncodeInt}(a, 32) \parallel \text{Rnd}(\text{saltlen}_{\text{bits}}))[0 : \text{saltlen}_{\text{bits}}]$ 
7:    $(h_0, h_1) \leftarrow \text{SHAKE256}(M \parallel \text{salt})[0 : 2n]$ 
8:    $(t_0, t_1) \leftarrow ((h_0 f + h_1 F) \bmod 2, (h_0 g + h_1 G) \bmod 2)$ 
9:    $\text{seed} \leftarrow M \parallel \text{kgseed} \parallel \text{EncodeInt}(a + 1, 32) \parallel \text{Rnd}(320)$ 
10:   $(x_0, x_1) \leftarrow \text{SamplerSign}(\text{seed}, (t_0, t_1))$ 
11:   $a \leftarrow a + 2$ 
12:  if  $\|(x_0, x_1)\|^2 > 8n\sigma_{\text{verify}}^2$  then
13:    continue loop
14:   $w_1 \leftarrow f x_1 - g x_0$ 
15:  if sym-break( $w_1$ ) = false then
16:     $w_1 \leftarrow -w_1$ 
17:   $s_1 \leftarrow (h_1 - w_1)/2$ 
18:   $\text{sig} \leftarrow \text{EncodeSignature}(\text{salt}, s_1)$ 
19:  if  $\text{sig} \neq \perp$  then
20:    return sig
```

The following notes apply to `HawkSign`:

- The message (m) is used once, at the start of a **SHAKE256** instance, and is followed by the public key hash. This allows streamed processing, i.e. data can be signed as it flows without buffering the entire message. Moreover, this initial hashing process is independent of the signing key or any randomly chosen value.
- The signature salt ($salt$) and the sampling seed ($seed$) are both obtained from a **SHAKE256** hash, computed over the concatenation of the input message (m), the private key seed ($kgseed$), an attempt counter (a or $a + 1$), and some random bits. This is a *derandomisation* mechanism that provides extra protection in case the random source happens not to be cryptographically strong in a practical situation; in the extreme case where **Rnd** is fully non-random and outputs only zeros, then this step degrades to deterministic signatures, but it is still safe. It is nonetheless recommended to inject true random bits, or at least some varying data (even if not unpredictable), since that improves resistance to some active physical attacks (fault attacks). Also note that a cryptographically strong random source, indistinguishable from true randomness, is still a strict requirement for key pair generation (**HawkKeyGen**).
- Conversely to the derandomisation process explained above, it is possible to generate safe and interoperable signatures by generating the salt and the random bits used in **SamplerSign** directly from a cryptographically strong random source, thus bypassing all use of **SHAKE256** on private data. Such alternative implementations can be useful on constrained devices for which **SHAKE256** is expensive but a cheaper safe RNG is available, e.g. CTR-DRBG built over a hardware AES-256 implementation.
- Since h_0 and h_1 are binary polynomials, only $F \bmod 2$ and $G \bmod 2$ are needed, not the full F and G .
- The computation of **sym-break** does not need to be performed in constant-time since w_1 is a public value, and for any sampled (x_0, x_1) , the opposite $(-x_0, -x_1)$ (that would lead to the opposite result for **sym-break**) could have been obtained with the same probability.
- We note that if w_1 is the zero polynomial, then both **sym-break**(w_1) and **sym-break**($-w_1$) are **false**. In this case, a valid signature may fail to pass verification, see Algorithm 20. However, for $w_1 = 0$ to be generated in Algorithm 15 it must be that $h_1 = 0$, since $w_1 \in 2R_n^2 + h_1$. We therefore assume this event happens with probability approximately 2^{-n} and disregard it.
- The whole process is organised as a loop because it can fail for three possible reasons:
 - The sampled (x_0, x_1) has a larger than acceptable ℓ^2 norm; this may happen with probability less than 2^{-17} for **HAWK-512** and less than 2^{-121} for **HAWK-1024** [DPPvW22a, Sec. 5.1].
 - One of the coefficients of s_1 is lower than $-2^{\text{high}_{s_1}}$ or greater than $2^{\text{high}_{s_1}} - 1$, making it incompatible with **CompressGR**. This is extremely rare.
 - The signature encoding (**EncodeSignature**) yields an output larger than the size defined in the scheme parameters. This is uncommon, because the target sizes have been set using the average encoded signature size (before padding) plus at least 4.5 times the standard deviation; however, it does happen in practice, on an occasional basis.

Most signature attempts still succeed the first time. Retries do not have a significant impact on average performance.

In particular, over ten million **HAWK-512** signatures there were two that failed because (x_0, x_1) was too long, and fifty-one that failed because the signature encoding was too

large. Similarly, for HAWK-1024 there were no failures due to length and twenty-five due to the signature encoding being too large. The second failure condition was never observed. Hence the approximate restart probabilities for HAWK-512 and HAWK-1024 are 5.3×10^{-6} and 2.5×10^{-6} respectively, and these are dominated by encoding failures.

- The polynomial s_1 is only half of the (formal) “uncompressed signature”, which is a vector $\mathbf{s} = (s_0, s_1)$. The value of s_0 is reconstructed by the signature verification algorithm. That reconstruction can fail, with a very low probability; private keys are generated with an explicit test against the bound β_0 in Algorithm 13 precisely so that this failure probability is as low as possible. For HAWK-512 and HAWK-1024, the failure probability is indeed negligible, i.e. it is estimated as less than 2^{-105} and 2^{-315} respectively [DPPvW22a]. For HAWK-256, the failure probability may be up to about 2^{-40} , which is considered tolerable for a “challenge” scheme.

3.6 Signature Verification

The formal (“uncompressed”) signature is a vector $\mathbf{s} = (s_0, s_1)$. However, the encoded signature contains only s_1 and the signature generation process did not even compute s_0 . Thus, the first step of the verification process is to rebuild s_0 using (27):

$$s_0 = \left\lceil \frac{h_0}{2} + \frac{q_{01}}{q_{00}} \left(\frac{h_1}{2} - s_1 \right) \right\rceil,$$

where the rounding is performed coefficientwise.

We define here an implementation of the above equation that uses only integer computations (interpreted as scaled-up versions of approximations of the actual real coefficients). In many usage contexts of signatures, other implementation strategies may be used, e.g. leveraging floating-point hardware.³ However, there are some contexts, in particular related to consensus protocols, where it is important that all implementations fully agree on whether a given signature is valid for a given message and public key pair; in that case, applications should use only implementations that can be proven to return the same output status as the one specified here.

For the specification of the FFT, InvFFT and RebuildS0 functions (Algorithms 16 to 18), we use the following conventions:

- Polynomial coefficients, in both normal and FFT representations, are stored as 32-bit signed integers, using two’s complement for negative values, and truncation for oversize values. The truncation of an integer z to a signed 32-bit value is mathematically equivalent to $((z + 2^{31}) \bmod 2^{32}) - 2^{31}$. For valid keys and signatures, no such truncation happens (values do not overflow), but invalid, maliciously crafted key pairs may induce such a condition.
- The same rules apply to temporary variables with a lowercase name (such as “ $x_{1, \text{re}}$ ”).
- Temporary variables with an uppercase name, such as “ T_{re} ”, use 64-bit signed integers.

In the fixed-point notation, a real value x is represented by an integer, which is a rounded version of x multiplied by a power of two. The scaling factor varies depending on the context (so the “fixed-point” terminology is slightly abusive), but not depending on the actual values, thus avoiding all the complexity of floating-point numbers. In practical implementations, all operations are on integers with a few extra shift operations that use shift counts independent of the data.

³Since signature verification uses only public data, the tricky question of whether a given floating-point hardware or software implementation is constant-time does not arise.

Let $\delta = e^{2i\pi/2048}$. We then define the following table of (precomputed) complex numbers, whose real and imaginary parts are both integers, for index $k = 0$ to 1023:

$$\Delta[k] = \left\lfloor 2^{31} \Re(\delta^{\text{rev}_{10}(k)}) \right\rfloor + i \left\lfloor 2^{31} \Im(\delta^{\text{rev}_{10}(k)}) \right\rfloor$$

The first two entries ($\Delta[0]$ and $\Delta[1]$) are not actually used. All other entries have real and imaginary integers that are strictly lower than 2^{31} in absolute value, so these can fit in a signed 32-bit representation.

For a polynomial a with real coefficients, its FFT representation $\hat{a}$ is the set of values $a(\zeta)$ for all complex roots ζ of $X^n + 1$. Since a is real, $a(\bar{\zeta})$ is the conjugate of $a(\zeta)$ when $\bar{\zeta}$ is the complex conjugate of ζ , and $\bar{\zeta}$ is another root of $X^n + 1$. Thus, it is sufficient to keep $n/2$ complex values in $\hat{a}$. The FFT function (Algorithm 16) computes $\hat{a}$ from a ; in the $\hat{a}$ representation, the real and imaginary parts of $a(\zeta)$ are stored at indices v and $v + n/2$, for some integer $v < n/2$.

Algorithm 16 FFT: Conversion to FFT representation (fixed point)

Require: Polynomial a (fixed-point)

Ensure: FFT representation $\hat{a}$ (fixed-point)

```

1:  $\hat{a} \leftarrow a$ 
2:  $t \leftarrow n/2$ 
3:  $m \leftarrow 2$ 
4: while  $m < n$  do
5:    $v_0 \leftarrow 0$ 
6:   for  $u = 0$  to  $m/2 - 1$  do
7:      $\varepsilon_{\text{re}} \leftarrow \Re(\Delta[u + m])$ 
8:      $\varepsilon_{\text{im}} \leftarrow \Im(\Delta[u + m])$ 
9:     for  $v = v_0$  to  $v_0 + t/2 - 1$  do
10:       $x_{1,\text{re}} \leftarrow \hat{a}[v]$ 
11:       $x_{1,\text{im}} \leftarrow \hat{a}[v + n/2]$ 
12:       $x_{2,\text{re}} \leftarrow \hat{a}[v + t/2]$ 
13:       $x_{2,\text{im}} \leftarrow \hat{a}[v + t/2 + n/2]$ 
14:       $T_{\text{re}} \leftarrow x_{2,\text{re}}\varepsilon_{\text{re}} - x_{2,\text{im}}\varepsilon_{\text{im}}$ 
15:       $T_{\text{im}} \leftarrow x_{2,\text{re}}\varepsilon_{\text{im}} + x_{2,\text{im}}\varepsilon_{\text{re}}$ 
16:       $\hat{a}[v] \leftarrow \lfloor (2^{31}x_{1,\text{re}} + T_{\text{re}})/2^{32} \rfloor$ 
17:       $\hat{a}[v + n/2] \leftarrow \lfloor (2^{31}x_{1,\text{im}} + T_{\text{im}})/2^{32} \rfloor$ 
18:       $\hat{a}[v + t/2] \leftarrow \lfloor (2^{31}x_{1,\text{re}} - T_{\text{re}})/2^{32} \rfloor$ 
19:       $\hat{a}[v + t/2 + n/2] \leftarrow \lfloor (2^{31}x_{1,\text{im}} - T_{\text{im}})/2^{32} \rfloor$ 
20:    $v_0 \leftarrow v_0 + t$ 
21:    $t \leftarrow t/2$ 
22:    $m \leftarrow 2m$ 
23: return  $\hat{a}$ 

```

In Algorithm 16, the operations on integers have been expressed in a detailed way, i.e. real and imaginary parts of the complex values are treated as separate variables. The divisions by 2^{32} are rounded toward minus infinity, which is what an “arithmetic right shift” operator computes in most programming languages.⁴

The inverse FFT is given by the function `InvFFT` (Algorithm 17). This algorithm uses the inverses of the roots of $X^n + 1$, as stored in the Δ table. Since all these roots lie on the unit circle, their inverse is obtained by complex conjugation, which is easily obtained by negating the imaginary part.

⁴This is guaranteed in Java, C#, Go, Rust... C and C++ are, formally, an exception, since right-

Algorithm 17 InvFFT: Conversion back from FFT representation (fixed point)

Require: FFT representation $\hat{a}$ (fixed-point)

Ensure: Polynomial a (fixed-point)

```

1:  $a \leftarrow \hat{a}$ 
2:  $t \leftarrow 2$ 
3:  $m \leftarrow n/2$ 
4: while  $m > 1$  do
5:    $v_0 \leftarrow 0$ 
6:   for  $u = 0$  to  $m/2 - 1$  do
7:      $\eta_{\text{re}} \leftarrow \Re(\Delta[u + m])$ 
8:      $\eta_{\text{im}} \leftarrow -\Im(\Delta[u + m])$ 
9:     for  $v = v_0$  to  $v_0 + t/2 - 1$  do
10:       $x_{1,\text{re}} \leftarrow a[v]$ 
11:       $x_{1,\text{im}} \leftarrow a[v + n/2]$ 
12:       $x_{2,\text{re}} \leftarrow a[v + t/2]$ 
13:       $x_{2,\text{im}} \leftarrow a[v + t/2 + n/2]$ 
14:       $t_{1,\text{re}} \leftarrow x_{1,\text{re}} + x_{2,\text{re}}$ 
15:       $t_{1,\text{im}} \leftarrow x_{1,\text{im}} + x_{2,\text{im}}$ 
16:       $t_{2,\text{re}} \leftarrow x_{1,\text{re}} - x_{2,\text{re}}$ 
17:       $t_{2,\text{im}} \leftarrow x_{1,\text{im}} - x_{2,\text{im}}$ 
18:       $a[v] \leftarrow \lfloor t_{1,\text{re}}/2 \rfloor$ 
19:       $a[v + n/2] \leftarrow \lfloor t_{1,\text{im}}/2 \rfloor$ 
20:       $a[v + t/2] \leftarrow \lfloor (t_{2,\text{re}}\eta_{\text{re}} - t_{2,\text{im}}\eta_{\text{im}})/2^{32} \rfloor$ 
21:       $a[v + t/2 + n/2] \leftarrow \lfloor (t_{2,\text{re}}\eta_{\text{im}} + t_{2,\text{im}}\eta_{\text{re}})/2^{32} \rfloor$ 
22:    $v_0 \leftarrow v_0 + t$ 
23:    $t \leftarrow 2t$ 
24:    $m \leftarrow m/2$ 
25: return  $a$ 

```

Using FFT and InvFFT, we can define **RebuildS0** (Algorithm 18) which, given the public key polynomials q_{00} and q_{01} , and the polynomials $w_1 = h_1 - 2s_1$ and h_0 , recomputes the corresponding s_0 polynomial, and returns $w_0 = h_0 - 2s_0$. Algorithm 18 uses the function **sgn**, which returns the “sign bit” of an integer: $\text{sgn}(z) = 1$ if $z < 0$, or 0 if $z \geq 0$.

Algorithm 18 RebuildS0: Rebuild s_0 from public key and signature

Require: Public polynomials q_{00} and q_{01} , w_1 , h_0

Ensure: w_0 , or $\perp$ on error

```

1:  $c_{w1} \leftarrow 2^{29-(1+\text{high}_{s1})}$ 
2:  $c_{q00} \leftarrow 2^{29-\text{high}_{00}}$ 
3:  $c_{q01} \leftarrow 2^{29-\text{high}_{01}}$ 
4:  $c_{s0} \leftarrow (2 \cdot c_{w1} \cdot c_{q01}) / (n \cdot c_{q00})$ 
5:  $\hat{w}_1 \leftarrow \text{FFT}(c_{w1} \cdot w_1)$ 
6:  $z_{00} \leftarrow q_{00}$ 
7: if  $z_{00}[0] < 0$  then
8:   return  $\perp$ 
9:  $z_{00}[0] \leftarrow 0$ 
10:  $\hat{q}_{00} \leftarrow \text{FFT}(c_{q00} \cdot z_{00})$ 
11:  $\hat{q}_{01} \leftarrow \text{FFT}(c_{q01} \cdot q_{01})$ 
12:  $\alpha \leftarrow (2 \cdot c_{q00} \cdot q_{00}[0]) / n$ 
13: for  $u = 0$  to  $n/2 - 1$  do
14:    $X_{\text{re}} \leftarrow \hat{q}_{01}[u] \hat{w}_1[u] - \hat{q}_{01}[u + n/2] \hat{w}_1[u + n/2]$ 
15:    $X_{\text{im}} \leftarrow \hat{q}_{01}[u] \hat{w}_1[u + n/2] + \hat{q}_{01}[u + n/2] \hat{w}_1[u]$ 
16:    $(X_{\text{re}}, z_{\text{re}}) \leftarrow (|X_{\text{re}}|, \text{sgn}(X_{\text{re}}))$ 
17:    $(X_{\text{im}}, z_{\text{im}}) \leftarrow (|X_{\text{im}}|, \text{sgn}(X_{\text{im}}))$ 
18:    $v \leftarrow \alpha + \hat{q}_{00}[u]$ 
19:   if  $v \leq 0$  or  $v \geq 2^{30}$  or  $X_{\text{re}} \geq 2^{32}v$  or  $X_{\text{im}} \geq 2^{32}v$  then
20:     return  $\perp$ 
21:    $y_{\text{re}} \leftarrow \lfloor X_{\text{re}} / v \rfloor$ 
22:    $y_{\text{im}} \leftarrow \lfloor X_{\text{im}} / v \rfloor$ 
23:    $\hat{q}_{01}[u] \leftarrow y_{\text{re}} - 2 \cdot z_{\text{re}} \cdot y_{\text{re}}$ 
24:    $\hat{q}_{01}[u + n/2] \leftarrow y_{\text{im}} - 2 \cdot z_{\text{im}} \cdot y_{\text{im}}$ 
25:  $t \leftarrow \text{InvFFT}(\hat{q}_{01})$ 
26: for  $u = 0$  to  $n - 1$  do
27:    $v \leftarrow c_{s0} \cdot h_0[u] + t[u]$ 
28:    $z \leftarrow \lfloor (v + c_{s0}) / (2 \cdot c_{s0}) \rfloor$   $\triangleright z$  is the rebuilt coefficient  $s_0[u]$ .
29:   if  $z < -2^{\text{high}_{s0}}$  or  $z \geq 2^{\text{high}_{s0}}$  then
30:     return  $\perp$ 
31:    $w_0[u] \leftarrow h_0[u] - 2 \cdot z$ 
32: return  $w_0$ 

```

In Algorithm 18, the local variables X_{re} and X_{im} should use a 64-bit integer type, but all other intermediate values will fit in signed 32-bit variables. Moreover, all divisions in this algorithm are by powers of two and can therefore be implemented with bit shifts, except for Lines 21 and 22, which require “true” divisions. The tests on the operands ensure that:

- the dividend (X_{re} or X_{im}) is non-negative and less than 2^{62} ,
- the divisor (v) is positive and less than 2^{30} ,

shifting a signed negative value yields an implementation-defined result; however, in practice, all software platforms use two’s complement for negative values, and the right-shift operator extends the sign bit.

- the quotient is strictly lower than 2^{32} .

The division is therefore well-defined, and can use a hardware division opcode that takes a 64-bit unsigned dividend and a 32-bit unsigned divisor (such as the one-operand variant of `div` on 32-bit x86) without triggering a CPU exception through a division by zero or an overflow.

The verification algorithm entails computing $\|\mathbf{w}\|_{\mathbf{Q}}^2$. We can write the following equation:

$$n\|\mathbf{w}\|_{\mathbf{Q}}^2 = \text{Tr}(q_{00}w_0w_0^* + q_{10}w_0w_1^* + q_{01}w_0^*w_1 + q_{11}w_1w_1^*)$$

Using the known relations between the elements of $\mathbf{Q}$, this can be rewritten as:

$$d = w_1/q_{00} \tag{38}$$

$$e = w_0 + q_{01}d \tag{39}$$

$$n\|\mathbf{w}\|_{\mathbf{Q}}^2 = \text{Tr}(q_{00}ee^* + dw_1^*) \tag{40}$$

The final value of $\|\mathbf{w}\|_{\mathbf{Q}}^2$ is a non-negative integer; we can thus perform its computation modulo sufficiently many small distinct primes, and rebuild the integer result with the CRT. It can be shown that thanks to the bounds on q_{00} , q_{01} and q_{11} (enforced during key generation with `HawkKeyGen`) and on s_0 and s_1 (enforced by `RebuildS0` and `DecodeSignature`, respectively), the result cannot be larger than $15 \cdot 2^{58}$. It is therefore sufficient to perform the computation modulo the two primes $p_1 = 2147473409$ and $p_2 = 2147389441$, since $p_1p_2 > 15 \cdot 2^{58}$. In fact, since the target norm is upper bounded in valid signatures by a value which is much lower than both p_1 and p_2 , we can avoid the CRT computation: for a valid signature, the computation of $n\|\mathbf{w}\|_{\mathbf{Q}}^2$ must yield the same integer result (when normalised in the 0 to $p-1$ range) for both moduli. For $p = p_1$ or $p = p_2$, the following properties hold:

- p is prime and $\mathbb{Z}/p\mathbb{Z}$ (the integers modulo p) is a finite field.
- $p-1$ is a multiple of 2048, allowing use of the number theoretic transform (NTT) to speed up multiplications and divisions of polynomials modulo $X^n + 1$ and modulo p .
- p is slightly lower than 2^{31} : multiplication modulo p can be implemented with a Montgomery multiplication, implementable with 32-bit integers in an efficient and portable way.
- q_{00} was verified during key pair generation, to be invertible modulo $X^n + 1$ and modulo p . Therefore, the division by q_{00} in NTT representation is well-defined and always works (no division by zero).

When p is prime and $p-1$ is a multiple of $2n$, the polynomial X^n+1 splits completely over $\mathbb{Z}/p\mathbb{Z}$, and the NTT representation of a polynomial u is the sequence of values $u(\mu) \bmod p$ for all n roots μ of $X^n + 1$ modulo p . In NTT representation, additions, subtractions, multiplications and divisions of polynomials are simply performed by applying the operation coefficientwise. Conversion to and from NTT representation can be done in $O(n \log n)$ operations (see Section 4.1). Two other useful properties of the NTT are noteworthy:

- The NTT representation of u^* contains the same values as the NTT representation of u , in a different order. If the NTT uses the classic “bit-reversal” order, then the coefficients of the NTT representation of u^* simply use the reverse order of the coefficients of the NTT representation of u .
- For a polynomial u , we have that $\text{Tr}(u) \bmod p$ is congruent to the sum of the coefficients of the NTT representation of u , modulo p .

We suppose that we have a function called NTT , such that $\text{NTT}(u, p)$ is the NTT representation of u modulo p ; we also use a function NTTadj such that $\text{NTTadj}(\text{NTT}(u, p)) = \text{NTT}(u^*, p)$. Using these functions, **PolyQnorm** (Algorithm 19), given q_{00}, q_{01}, w_0, w_1 , and a modulus p , returns $n\|\mathbf{w}\|_{\mathbf{Q}}^2 \bmod p$. It does not matter, for **PolyQnorm**, which ordering convention NTT uses, as long as NTTadj uses the same convention.

Algorithm 19 PolyQnorm: polynomial Q-norm evaluation (modular)

Require: Polynomials q_{00}, q_{01}, w_0, w_1 ; modulus p

Ensure: $n\|\mathbf{w}\|_{\mathbf{Q}}^2 \bmod p$

```

1:  $\bar{q}_{00} \leftarrow \text{NTT}(q_{00}, p)$ 
2:  $\bar{q}_{01} \leftarrow \text{NTT}(q_{01}, p)$ 
3:  $\bar{w}_0 \leftarrow \text{NTT}(w_0, p)$ 
4:  $\bar{w}_1 \leftarrow \text{NTT}(w_1, p)$ 
5:  $\bar{d} \leftarrow \bar{w}_1 / \bar{q}_{00}$ 
6:  $\bar{e} \leftarrow \bar{w}_0 + \bar{d} \cdot \bar{q}_{01}$ 
7:  $\bar{c} \leftarrow \bar{q}_{00} \cdot \bar{e} \cdot \text{NTTadj}(\bar{e}) + \bar{d} \cdot \text{NTTadj}(\bar{w}_1)$ 
8:  $r \leftarrow 0$ 
9: for  $i = 0$  to  $n - 1$  do
10:    $r \leftarrow r + \bar{c}[i]$ 
11: return  $r \bmod p$ 

```

In Algorithm 19, all operations performed on NTT representations are computed coefficientwise and modulo p . The final result (r) is also computed modulo p normalised into the 0 to $p - 1$ range.

We can now express the **HawkVerify** function (Algorithm 20), which performs the signature verification and returns **true** for a valid (message, public key, signature) triplet, or **false** otherwise.

Algorithm 20 HawkVerify: HAWK signature verification

Require: Message m , public key pub , signature sig

Ensure: true if valid, false otherwise

```

1:  $r \leftarrow \text{DecodeSignature}(\text{sig})$ 
2: if  $r = \perp$  then
3:   return false
4:  $(\text{salt}, s_1) \leftarrow r$ 
5:  $r \leftarrow \text{DecodePublic}(\text{pub})$ 
6: if  $r = \perp$  then
7:   return false
8:  $(q_{00}, q_{01}) \leftarrow r$ 
9:  $\text{hpub} \leftarrow \text{SHAKE256}(\text{pub})$ 
10:  $M \leftarrow \text{SHAKE256}(m \parallel \text{hpub})$ 
11:  $(h_0, h_1) \leftarrow \text{SHAKE256}(M \parallel \text{salt})[0 : 2n]$ 
12:  $w_1 \leftarrow h_1 - 2 \cdot s_1$ 
13: if  $\text{sym-break}(w_1) = \text{false}$  then
14:   return false
15:  $w_0 \leftarrow \text{RebuildS0}(q_{00}, q_{01}, w_1, h_0)$ 
16: if  $w_0 = \perp$  then
17:   return false
18:  $r_1 \leftarrow \text{PolyQnorm}(q_{00}, q_{01}, w_0, w_1, p_1)$ 
19:  $r_2 \leftarrow \text{PolyQnorm}(q_{00}, q_{01}, w_0, w_1, p_2)$ 
20: if  $r_1 \neq r_2$  or  $r_1 \not\equiv 0 \pmod n$  then
21:   return false
22:  $r_1 \leftarrow r_1/n$ 
23: if  $r_1 > 8n \cdot \sigma_{\text{verify}}^2$  then
24:   return false
25: return true

```

4 Implementations

In this section, we give recommendations for implementors to end up with a fast implementation that uses little RAM. In comparison to Section 3, these recommendations do not need to be followed, in contrast to e.g. the fixed-point FFT specified in Section 3.6.

In Section 4.1, we highlight some techniques that were used in the provided implementations. These allow for fast implementations. Timings, and the method to obtain them, are provided in Section 4.2. There, we also present two trade-offs between time and memory that are possible in signing and verification.

4.1 Software Implementation Techniques

4.1.1 NTT

The NTT transform is a useful tool for performing fast computations over polynomials defined modulo $X^n + 1$. The HAWK signature verification was specified in Section 3.6 using that transform, but without prescribing a specific method of computing the NTT or its inverse, nor a specific order of NTT coefficients. Algorithms 21 and 22 show the classic “bit-reversal” methods for computing the NTT and its inverse, respectively. These algorithms expect the following:

- All computations are done in the finite field $\mathbb{Z}/p\mathbb{Z}$ of integers modulo p for a prime $p \geq 3$ such that $p \equiv 1 \pmod{2n}$.
- Let γ be a primitive $2n$ -th root of unity modulo p . The tables Γ and Γ^{-1} contain some precomputed powers of γ in a specific order:

$$\begin{aligned}\Gamma[i] &\equiv \gamma^{\text{rev}_{\log n}(i)} \pmod{p}, \\ \Gamma^{-1}[i] &\equiv \gamma^{-\text{rev}_{\log n}(i)} \pmod{p},\end{aligned}$$

where the bit-reversal function (cf. Section 3.1) is over $\log n = \log_2(n)$ bits.

Algorithm 21 NTT: Conversion to NTT representation

Require: Polynomial u (modulo $X^n + 1$ and modulo p)

Ensure: NTT representation of u (in place)

```

1:  $t \leftarrow n$ 
2:  $m \leftarrow 1$ 
3: while  $m < n$  do
4:    $t \leftarrow t/2$ 
5:   for  $i = 0$  to  $m - 1$  do
6:      $s \leftarrow \Gamma[i + m]$ 
7:     for  $j = 0$  to  $t - 1$  do
8:        $u_0 \leftarrow u[2ti + j]$ 
9:        $u_1 \leftarrow u[2ti + t + j]$ 
10:       $u[2ti + j] \leftarrow u_0 + su_1$ 
11:       $u[2ti + t + j] \leftarrow u_0 - su_1$ 
12:    $m \leftarrow 2m$ 
13: return  $u$ 

```

There are several possible variants for the implementation of NTT and iNTT; for instance, the halvings (division by 2) in iNTT may be delayed and performed with a single pass after completion of the outer loop.

Algorithm 22 iNTT: Inverse NTT transform

Require: NTT representation of polynomial u
Ensure: Plain representation of u

```

1:  $t \leftarrow 1$ 
2:  $m \leftarrow m$ 
3: while  $m < n$  do
4:    $m \leftarrow m/2$ 
5:   for  $i = 0$  to  $m - 1$  do
6:      $s \leftarrow \Gamma^{-1}[i + m]$ 
7:     for  $j = 0$  to  $t - 1$  do
8:        $u_0 \leftarrow u[2ti + j]$ 
9:        $u_1 \leftarrow u[2ti + t + j]$ 
10:       $u[2ti + j] \leftarrow (u_0 + u_1)/2$ 
11:       $u[2ti + t + j] \leftarrow s(u_0 - u_1)/2$ 
12:    $t \leftarrow 2t$ 
13: return  $u$ 

```

A noteworthy point is that the Γ and Γ^{-1} tables for degree n are strict prefixes of the corresponding tables for degree $2n$; in other words, a precomputed table for degree $n = 1024$ can also be used for computing NTTs with the same prime modulus for lower degrees (512, 256, ...). Moreover, Γ^{-1} can be recomputed from Γ using the fact that $\gamma^n = -1$, thus $\gamma^{-i} = -\gamma^{n-i}$.

4.1.2 Key Pair Generation

The HAWK key pair generation (Algorithm 13) consists of several steps, some of which are amenable to various implementation techniques. In the HAWK reference and optimised implementations, we use the techniques and implementation from [Por23]. The following points are noteworthy:

- The polynomial $q_{00} = ff^* + gg^*$ is computed modulo the prime integer $p_1 = 2147473409$. Since the coefficients of f and g are at most 8 in absolute value, it is guaranteed that the coefficients of q_{00} cannot exceed $128n$, which is lower than $p_1/2$. Similar considerations apply for the computation of q_{01} and q_{11} .
- Invertibility of q_{00} modulo p_1 is tested by verifying that none of the coefficients of $\text{NTT}(q_{00})$ are zero. The same test is applied modulo p_2 .
- The verification that $(1/q_{00})[0] \geq \beta_0$ is performed by computing the $1/q_{00}$ polynomial (over the rationals, not modulo a prime p) with a fixed-point FFT, which is also used within NTRUSolve. Note that this check is meant to ensure that the probability of algorithm RebuildS0 failing to recompute the s_0 part of a signature during verification is negligible. As such, there are no great precision requirements on this computation, and an approximate fixed-point FFT is good enough.
- NTRUSolve uses the approach of [PP19]. The field norm is applied repeatedly to halve the degrees of the involved polynomial until solving the NTRU equation becomes a simple extended GCD computation over big integers. The full degree solution is then obtained by unwinding the halvings, and applying Babai's round-off algorithm at each step. Computations with the NTT modulo small 31-bit primes p_i , are used extensively. The Babai rounding uses a FFT with fixed-point 64-bit numbers.

Algorithm HawkKeyGen enforces the check of invertibility modulo the specific primes p_1 and p_2 so that signature verification may use the same moduli; proper interoperability

of HAWK implementations requires that key pair generators and signature verifiers use the same primes. However, nothing forces the key pair generation to use 31-bit moduli for its internal computations. For instance, hardware platforms with large 64-bit multipliers might get performance improvements with 63-bit moduli.

4.1.3 Signature Generation

After obtaining the initial hash h_0, h_1 from the message, public key and a salt, the polynomials (t_0, t_1) are computed by multiplying with $\mathbf{B}$ on the left. These computations can be performed modulo 2, since we are ultimately interested only in the least significant bit of the resulting polynomial coefficients. This is why only $F \pmod{2}$ and $G \pmod{2}$ are stored in the private key, not the full F and G . It is possible to perform the computations of t_0 and t_1 modulo a large enough prime that supports the NTT. However, it appears to be more efficient, especially relatively to RAM usage, to avoid the NTT and directly use binary polynomials, i.e. polynomials with coefficients in $\mathbb{Z}/2\mathbb{Z}$. A product of binary polynomials of degree less than n can be performed with three products of binary polynomials of degree less than $n/2$, and a few polynomial additions (which are just bitwise XORs, since we work here in $(\mathbb{Z}/2\mathbb{Z})[X]$), using the classic Karatsuba–Ofman method [KO62]. For instance, if u and v are binary polynomials of degree less than n , then we can break each in its low and high halves:

$$\begin{aligned} u_0 &= u[0:n/2], \\ u_1 &= u[n/2:n], \\ v_0 &= v[0:n/2], \\ v_1 &= v[n/2:n], \end{aligned}$$

and we can then compute the product uv as:

$$\begin{aligned} uv &= (u_0 + X^{n/2}u_1)(v_0 + X^{n/2}v_1) \\ &\equiv (u_0v_0) + X^n(u_1v_1) + X^{n/2}((u_0 + u_1)(v_0 + v_1) + u_0v_0 + u_1v_1) \pmod{2}. \end{aligned}$$

Applying this method recursively, the polynomial product boils down to products of small polynomials. Some software platforms provide a “carryless multiplication” opcode (e.g. `pclmul` on x86) which can perform these products; otherwise, integer multiplications can be used (separating the data bits with enough zeros to prevent carries from propagating). Once the product of two binary polynomials has been computed (with a result over $2n - 1$ bits), reduction modulo $X^n + 1$ is a simpler matter of bitwise XORing the high half into the low half.

The `SamplerSign` function uses secret data and thus should be implemented in a way that does not leak information about that data through side channels. Algorithm 14 shows a design that can at least be constant-time, i.e. avoiding leaks leveraging timing measurements:

- The input random 78-bit value c is compared to the elements of the T_0 and T_1 tables; for each table, the count of table elements greater than c is computed. Although only one of the two counts is ultimately retained, both are systematically computed to avoid leaking the corresponding bit of $\mathbf{t}$.
- Similarly, all table elements are always used, even though the values in each table are monotonically decreasing. An early exit strategy **must not** be used if the signature generation time is potentially observable by outsiders.
- Each comparison between c and a value $T_0[z]$ is done by taking the sign bit of $c - T_0[z]$. Values consist of 78 bits, which exceeds the 64-bit maximum size that

can be expected from portable C systems; the split of c into a 63-bit low part and a 15-bit high part is meant to support manual carry propagation in a portable way.

As was pointed out in Section 3.5, the SHAKE256 $\times$ 4 output words are used in such an order that every value is being sampled from the output of a single SHAKE256 instances among the four instances that SHAKE256 $\times$ 4 formally runs in parallel. This allows RAM-constrained implementations to run the four SHAKE256 instances sequentially, reusing the same space for the internal SHAKE256 state (of size about 200 bytes), while platforms with SIMD opcodes can run two or four SHAKE256 instances truly in parallel, and apply the T_0 and T_1 tables also in parallel. On x86 CPUs with AVX2 opcodes, the four SHAKE256 instances in SHAKE256 $\times$ 4 can indeed run in parallel.

Once the (d_0, d_1) vector has been sampled, the HawkSign algorithm needs to compute $w = fd_1 - gd_0$, potentially negate w , then produce $s_1 = (h_1 - w)/2$ (the division by 2 is exact, so the coefficients of s_1 are integers). For these computations, the NTT can be used with a small prime modulus; the elements of s_1 are normally distributed around zero with a standard deviation less than 400, thus a modulus $p \geq 400 \times 32$ covers all values up to 16 times the standard deviation, and the probability of a value of s_1 being incorrectly computed when working modulo such a p is less than 2^{-189} , hence negligible. For software implementations, moduli $p = 12289$ and $p = 18433$ are especially appropriate, since they are compatible with the use of the NTT for all specified HAWK parameter sets, and they fit on 15 bits, which is convenient for both small microcontrollers, and for powerful CPUs with SIMD opcodes. The reference and optimised implementations of HAWK use $p = 18433$; integers modulo p use Montgomery representation with $R = 2^{32}$ i.e. an integer value x is represented by $2^{32}x \bmod p$, normalised to the 1 to p range (this specific representation allows one to avoid the conditional subtraction of the modulus that usually follows the Montgomery reduction of a product). Other moduli are possible; if multiplication of small integers is expensive on a given platform, then $p = 65537$ is a good choice, since it allows a multiplicationless reduction.

4.1.4 Signature Verification

Signature verification starts with reconstruction of the s_0 polynomial (Algorithm RebuildS0), followed by a computation of $\|\mathbf{h} - 2\mathbf{s}\|_{\mathbf{Q}}^2$. The first step involves computing an approximation of a polynomial over the rationals, and rounding its coefficients to the nearest integers. Such a rounding may exercise threshold conditions: if the value to round is very close to, for instance, $1/2$, then two different implementations using distinct approximation methods may round such a value differently (one rounding the value to 0, the other rounding the value to 1) and come up with distinct signature verification outcomes (the signature would be deemed valid by the implementation that rounded “correctly”, and invalid by the other implementation). The key pair generation enforces a maximum bound on $(1/q_{00})[0]$ precisely so that properly generated signatures cause signature verifiers to rebuild s_0 reliably and correctly, because s_0 has coefficients far enough from the threshold values. Nevertheless, it is conceptually feasible for a malicious signer to craft a key pair and a signature value that would be arbitrarily close to such thresholds. This can be an issue in some usage contexts, in particular distributed consensus protocols that require all parties to agree on the validity of a given signature for a given message and public key.

In order to avoid such issues, implementations of RebuildS0 **should** follow the steps of RebuildS0 (Algorithm 18) exactly. A fixed point approximation of values is used, with 32-bit integers and floored rounding of divisions. This corresponds to the simplest implementation on typical software platforms (using two’s complement for negative integers, and arithmetic shifts for division by powers of two).

The second part of HawkVerify, i.e. the computation of $\|\mathbf{h} - 2\mathbf{s}\|_{\mathbf{Q}}^2$, is specified in Algorithms 19 and 20 with polynomials modulo two specific 31-bit primes p_1 and p_2 . Other

moduli p_i are usable, provided that q_{00} is invertible modulo $X^n + 1$ and p_i ; key pair generation ensures that this is true for the specified p_1 and p_2 , but any implementation using another modulus must account for the possibility that one of the modular divisions in PolyQnorm fails because the divisor turns out to be zero. On 64-bit platforms, one may get an improved performance by using a single modulus larger than $15 \cdot 2^{58}$. An alternative implementation strategy for this step is to use floating-point operations: since the squared norm is an integer, an approximate computation is good enough.

4.2 Benchmarks

We measured the speed of the HAWK implementations on two platforms: a recent x86 CPU (using the optimised AVX2 implementation), and an embedded ARM Cortex M4 CPU (using the reference implementation):

- x86 system: Intel Core i5-8259U (“Coffee Lake”) running at 2.3 GHz, with Linux (Ubuntu 22.04) in 64-bit mode (`x86_64`). Frequency scaling (“TurboBoost”) is disabled. A single CPU core is used, on an otherwise idle machine. Compiler is Clang-14.0.0, with optimisation flags “-O2”. AVX2 support is enabled on all functions that use AVX2 intrinsics through function attributes.
- ARM Cortex M4 system: STM32F4 “discovery” board (STM32F407VG-DISC1), running at 24 MHz; memory access times, for both RAM and ROM (Flash), have no extra wait states (local caches are not used at such frequencies). Compiler is GCC-10.3.1, with optimisation flags “-O2 -mthumb -mcpu=cortex-m4”. The Keccak-f permutation which is at the core of SHAKE256 is implemented in assembly. Apart from this, the plain C reference code is used.

The measured performance figures are shown on Table 6.

Speed Measurements. All timings are averages expressed in clock cycles. The code is isochronous in the sense that execution time variations cannot be correlated with secret information; however, execution time still varies, in particular in key pair generation (for restarts when a candidate (f, g) pair is unsuitable), but also for all public key and signature encoding and decoding routines.

RAM Usage Estimation. The shown RAM usage is an estimate for embedded systems such as the ARM Cortex M4. Each of the key pair generation, signature generation and signature verification functions expects the caller to provide a temporary buffer in which most of the computations happen, and output values are also returned in that buffer. The stack space is used only for a small number of index values and pointers, as well as up to two concurrently running SHAKE256 contexts (208 bytes each), and an additional buffer for h_1 (in signature generation, up to 128 bytes) or (h_0, h_1) (in signature verification, up to 256 bytes). The implementation does not use recursion. Since the code is written in C, the C compiler may allocate space for its own purposes in mostly uncontrollable ways, and tends to be somewhat generous in that respect. Moreover, for improved API clarity and flexibility, some extra call layers have been added, which increases the amount of such compiler allocated stack space. To account for stack usage in a hypothetical streamlined implementation, we add 1024 bytes to the size of the temporary buffer. On x86, stack usage is greater (mostly because of the use of four parallel SHAKE256 instances in SHAKE256x4, and also because all registers are twice larger) but that matters less in practice, since systems using 64-bit CPUs have large amounts of available RAM, and the whole computation still fits entirely within the L1 cache.

Table 6: HAWK performance on x86 (“Coffee Lake”) and ARM Cortex M4. Speed is expressed in clock cycles, and RAM usage in bytes.

	HAWK-256	HAWK-512	HAWK-1024
Targeted security	Challenge	NIST-1	NIST-5
Speed on x86 “Coffee Lake” with AVX2 (clock cycles)			
Key pair generation	1877122	8432840	43660958
Signature generation	54361	85372	180816
Signature generation (fast)	25979	43686	85381
Signature verification	73260	148224	302861
Signature verification (fast)	61125	124286	255312
Speed on ARM Cortex M4 (clock cycles)			
Key pair generation	18717711	52316870	225658496
Signature generation	1585967	2801495	6179673
Signature generation (fast)	581590	1161174	2494513
Signature verification	702905	1418539	3006983
Signature verification (fast)	607064	1230466	2609859
RAM usage (bytes)			
Key pair generation	7680	14336	27648
Signature generation	2560	4096	7168
Signature generation (fast)	3152	5272	9512
Signature verification	3584	6144	11264
Signature verification (fast)	4928	8768	16512

Fast Signature Generation. Speed and RAM usage for “normal” signature generation correspond to the HAWK scheme exactly as specified in this document. The “fast” signature generation is modified in two aspects:

- In the normal signature generation, first the (encoded) private key is decoded, by the function `DecodePrivate`, into `kgseed`, $F \bmod 2$, $G \bmod 2$ and `hpub`, and then f, g are generated from `kgseed` in the function `Regeneratefg`. The fast signature generation, however, takes $f, g, F \bmod 2, G \bmod 2$ and `hpub` as input, removing the CPU cost of `DecodePrivate` and `Regeneratefg` from the signing process. This does increase the RAM requirement for storing the decoded private key.
- The random source used for `SamplerSign` is no longer `SHAKE256x4`, but a fast RNG (based on AES-128 on x86, or on ChaCha8 on the Cortex M4).

As was noted in Section 3.5, using an alternative RNG for signature generation does not break interoperability; in fact, outsiders cannot detect whether `SHAKE256x4` or another RNG was used for producing any given signature. The measurements provided here are meant only as an illustration of the fact that `SHAKE256` related costs are an important part of the runtime cost of signature generation (up to 2/3 of the overall cost on the ARM Cortex M4); thus, substantial performance improvements can be expected if the platform provides a hardware accelerated `SHAKE256` implementation, or if the RNG is switched to another secure but faster primitive, e.g. AES-256 or TurboSHAKE [BDH⁺23].

Fast Signature Verification. The cost of signature verification depends on the amount of available RAM. When only a minimal amount of temporary space is provided, the public key and signature values are decoded several times from their respective Golomb–Rice representations. When more space is available, such decoding is done only once, and the decoded polynomials s_1 , q_{00} and q_{01} are stored in that extra space, thereby avoiding the cost of repeated decompression. It is easy for an implementation to opportunistically choose the compact or fast modes, depending on the size of the caller provided temporary buffer. Table 6 illustrates this trade-off: the fast signature verification is about 15% faster (depending on degree and architecture), but uses up to 50% more RAM.

5 Cryptanalysis

For the cryptanalytic ideas behind the modelling of HAWK we refer back to [DvW22, DPPvW22a]. In this chapter we explain, reproduce and extend the practical cryptanalysis from [DPPvW22a] and verify some additional changes, such as the switch to the centred binomial distribution for key generation. The cryptanalysis of HAWK is separated into two sections. In Section 5.1 we consider the problem of directly recovering the secret key from the public key; this is an instance of breaking the module search lattice isomorphism problem and we approach it using lattice reduction. We examine a threshold phenomenon on the lengths of vectors found during lattice reduction that determines a lower bound on the lengths of f and g required for security. In Section 5.2 we consider strong forgery attacks via mapping them to lattice reduction tasks. We end with tables reporting the estimated BKZ blocksize required for the above two cryptanalytic tasks, also taking into account the dimensions for free techniques of [Duc18].

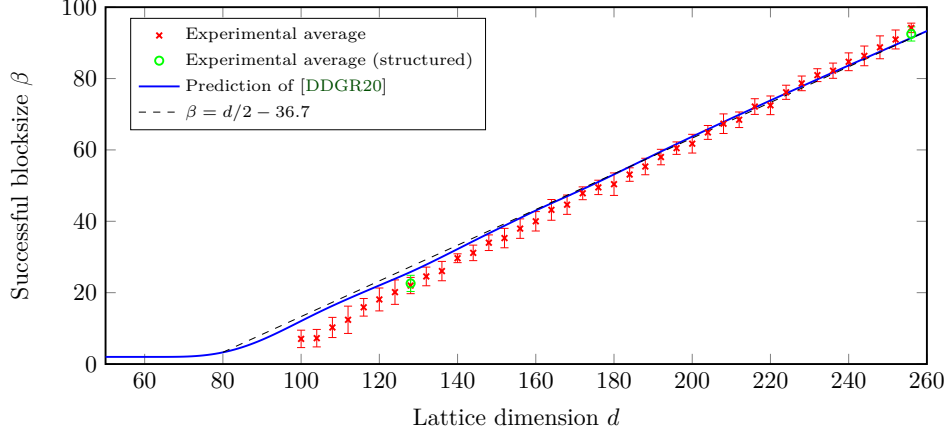
In each section we provide a model for the attack strategy, and for Section 5.1 we provide experimental evidence. For functions relating to our model, see https://github.com/hawk-sign/aux/blob/main/code/find_params.sage. The experiments are conducted over two types of instances. When n is a power of two, we are able to generate HAWK public keys as detailed in Section 3.4, minus the conditions that determine whether encoding will be successful or not, as these are not defined except for HAWK- $\{256, 512, 1024\}$. When n is not a power of two, we generate instances following the method of [DvW22, Alg. 1], i.e. the NTRU structure is lacking. Since the original specification of HAWK in [DPPvW22a] we have altered the key generation procedure to use a centred binomial distribution rather than a discrete Gaussian, as our practical cryptanalysis suggested that only the lengths of f and g were important. Therefore the distribution internal to [DvW22, Alg. 1] in our experiments will be the centred binomial distribution by default, with an explicitly indicated fallback to the discrete Gaussian distribution when we need more control over the width of the distribution. Finally, our experiments treat the rank two module basis given by $\mathbf{B}$ in HAWK as the rank $2n$ integer basis given by $\text{rot}(\mathbf{B})$; equivalently, we treat the module gram matrix $\mathbf{Q}$ as $\text{rot}(\mathbf{Q})$.

5.1 Secret key recovery

We first note that any unimodular matrix $\mathbf{V} \in \text{GL}_{2n}(\mathbb{Z})$ such that $\mathbf{V}^t \mathbf{V} = \mathbf{Q}$ allows one to sign as if one has the secret key $\mathbf{B}$; run an unstructured version of the signature algorithm with $\mathbf{V}$ in place of $\mathbf{B}$. This is equivalent to finding a $\mathbf{U} = \mathbf{V}^{-1} \in \text{GL}_{2n}(\mathbb{Z})$, such that $\mathbf{U}^t \mathbf{Q} \mathbf{U} = \mathbf{I}_{2n}$, i.e. reducing any basis corresponding to $\mathbf{Q}$ to an orthonormal basis. In particular, we consider finding a single vector of length one as a success.

Recovering unusual short vectors. We first study the BKZ blocksize required to recover a vector of length one, as a function of the dimension of the instance, in experiments where the centred binomial distribution has a ‘large’ parameter η , i.e. when the initial basis is badly reduced. This is an instance of the shortest vectors problem, where we have (up to sign) $d = 2n$ unusually short vectors of length 1. When performing lattice reduction on $\mathbf{Q}$, at a certain point the length of the shortest vector in the basis drops suddenly from what lattice reduction heuristics would suggest, from say $\sigma_{\text{krsec}}(d) \cdot \sqrt{d}$, to length one. This threshold phenomenon is due to a projection of a unit vector being found in a terminal block during BKZ reduction. Asymptotically, the required blocksize that triggers this behaviour is given by $\beta = d/2 + o(d)$, when $\sigma_{\text{krsec}} = \Theta(\sqrt{d})$ [ADPS16]. For a more precise concrete estimate of β and σ_{krsec} , we use the estimator from [DDGR20]: it relies on the BKZ simulator of Chen and Nguyen [CN11], a probabilistic model for the projected length of unusual short vectors, and accounts for the number of such short vectors. In Fig. 1 we

show experimental evidence that the blocksize estimates are closely followed in practice, both for the unstructured and for the structured cases.



We ran progressive BKZ (one tour per blocksize) over $\mathbb{Z}^d$ using an input form generated with $\eta = 20$ and report the average successful β that recovered a length one vector over 20 instances. We used the BKZ simulator and probabilistic model of [DDGR20], accounting for the d target solutions.

Figure 1: Blocksize required to recover a shortest vector via lattice reduction as a function of dimension d .

A threshold phenomenon on basis lengths. To keep our public key $\mathbf{Q}$ small we want to choose the centred binomial parameter η as small as possible. However, if η is too small then $\mathbf{Q}$ is already well reduced, which may make it easier to recover the full orthonormal basis. We consider, for some fixed dimensions, the key recovery experiment by running BKZ on public keys $\mathbf{Q}$ of different reduction quality, to observe how the hardness of this lattice reduction task behaves as a function of η . To make these experiments more granular we considered public keys where the sampling of f and g is performed using a discrete Gaussian distribution for smaller increments of standard deviation $\widetilde{\sigma}_{\text{kgen}}$, as in [DPPvW22a, Fig. 4]. We clarify that here we report the *standard deviations* $\widetilde{\sigma}_{\text{kgen}}$, which significantly differ from the small *widths* σ_{kgen} of the discrete Gaussian distributions. Concretely, this is because $\sigma_{\text{kgen}} \geq \eta_\varepsilon(\mathbb{Z}^d)$ only for large ε in these experiments, see Definition 1. This approach also allows us to observe whether in these experiments there is a difference in practical hardness between the centred binomial distribution and the discrete Gaussian distribution of equal standard deviation. The results are shown in Fig. 2 and show a certain threshold behaviour, where at some point, fixing the dimension, the required blocksize stops increasing and stays the same for growing $\widetilde{\sigma}_{\text{kgen}}$. Based on this cryptanalysis we want to pick our public key around this threshold value of $\widetilde{\sigma}_{\text{kgen}}$ to minimise its size while maintaining the maximal hardness of the problem. We argue that this maximum hardness, even though the actual threshold might be lower, is attained when $\widetilde{\sigma}_{\text{kgen}} > \sigma_{\text{krsec}}$. During the usual progressive lattice reduction attack vectors of this length are by definition found *before* recovering a unit vector. So even if such short vectors are given away one would still require the last BKZ tour(s), or large SVP call in the last block, to actually recover a unit vector. We therefore also plot our simulation of σ_{krsec} on Fig. 2, noting that for the smaller dimensions these simulations deviate from our experimentally observed values of σ_{krsec} . From about successful blocksize 40 onwards, or $d = 170$, the estimates for σ_{krsec} grows slowly with dimension, matching the experimentally observed growth.

To verify that the maximum hardness is achieved at σ_{krsec} we compare the successful

blocksize, for several fixed dimensions, when $\widetilde{\sigma_{\text{ngen}}}$ is large and when $\widetilde{\sigma_{\text{ngen}}} = \sigma_{\text{krsec}}(d)$ is on the estimated threshold. In particular, for these experiments, we fix the length of the secret basis vectors we sample in the key generation from below by $\sigma_{\text{krsec}}\sqrt{d}$, and resample them otherwise. For the structured case this is comparable to the condition $\|(f, g)\| \geq \sigma_{\text{krsec}}\sqrt{d}$ that is used in HAWK. Fig. 3 shows that the successful blocksize are comparable in both cases, with an almost identical average, and only slightly bigger negative deviations in the threshold case.

We thus conclude that if $\|(f, g)\| \geq \sigma_{\text{krsec}}\sqrt{d}$ then to recover the secret key an adversary must run BKZ with the average blocksize β suggested in Section 5.1, specifically Fig. 1.

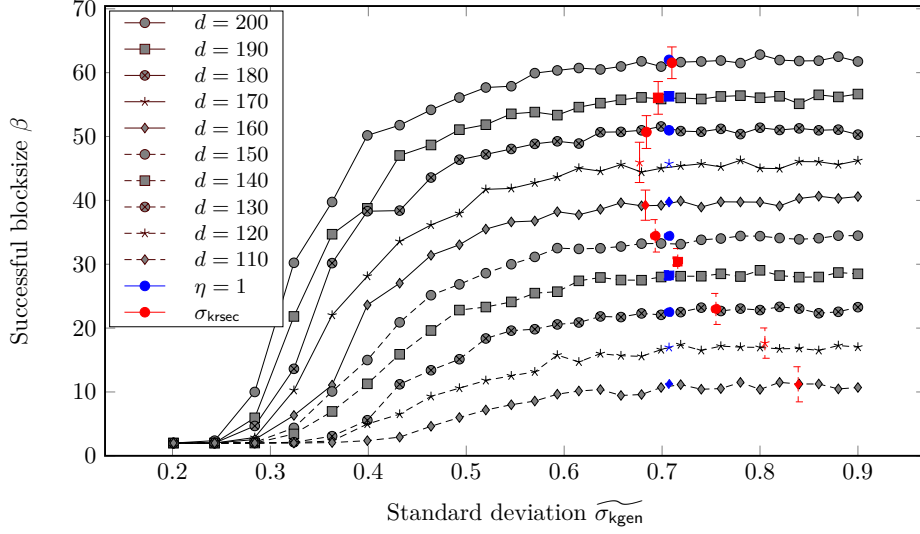
Prior occurrences of the same phenomenon. This threshold phenomenon is not specific to our cryptanalysis of this lattice isomorphism problem, but is merely a reminiscence of a well know phenomenon in the pure geometric formalism of LIP. It is implicitly present in the cryptanalysis of SIS and LWE, i.e. lattice reduction problems in q -ary lattices. For example, when attacking LWE, one usually considers “forgetting” some LWE samples to trade dimension for volume and thereby optimise the attack parameters. A geometric interpretation given in [DDGR20] is that the attacker is implicitly projecting the problem against a q -vector, and this is advantageous whenever q is smaller than the first basis vector length achieved by BKZ with a certain blocksize on a random lattice of the same volume and dimension. That is: the public description of the lattice freely gives mildly short vectors that are helpful. A similar exploit was considered in the cryptanalysis of DILITHIUM [LDK⁺22], though it was ineffective in their context. It was recently successfully applied [DEP23] against a SIS type problem with low modulus q , breaking a proposed variation of FALCON [ETWY22].

Our parametrisation strategy is precisely meant to avoid this pitfall. The short vectors given to the adversary are no shorter than what he would find during a sufficiently strong lattice reduction to attack the scheme.

A note on tours in progressive BKZ. Throughout our experiments we use one tour per blocksize in our progressive BKZ. It is possible to construct more optimal approaches, see [AWHT16]. However, the approach of running some number $\tau \geq 1$ tours for each blocksize, while offering improvements for small dimensions d , quickly becomes a worse strategy. For example, by simulation [DDGR20], at the HAWK-256 challenge parameters with $\tau \in \{1, 2, 4, 8\}$, the estimated required blocksize decreases by one for each doubling of τ . This is an unfavourable trade; twice as many tours required for a BKZ run that costs more than half as much. More generally, one may vary the tours in `key_recovery_beta_ssec`.

5.2 Strong forgery

A strong forgery is when an adversary is able to produce a valid signature on a message for which it does not have any signatures. In particular, given a point $\mathbf{h} \in \mathbb{Z}^{2n}$ and $\mathbf{Q}$ one must find an $\mathbf{s} \in \mathbb{Z}^{2n}$ such that $\|\mathbf{h} - 2\mathbf{s}\|_{\mathbf{Q}}^2 \leq 8n\sigma_{\text{verify}}^2$. This is equivalent to being given a point $\mathbf{t} = \mathbf{B} \cdot \mathbf{h}$ and $\mathbf{B}$ and having to find some $\mathbf{y} \in \mathbb{Z}^{2n}$ such that $\|\mathbf{t} - 2\mathbf{y}\| \leq 2\sqrt{2n}\sigma_{\text{verify}}$. This is an approximate CVP instance over a rotation of $2\mathbb{Z}^{2n}$. In particular if we measure the approximation factor γ with respect to the normalised volume we have $\|\mathbf{t} - 2\mathbf{y}\| \leq \gamma \cdot \text{vol}(2\mathbb{Z}^{2n})^{1/(2n)} = 2\gamma$ for $\gamma = \sqrt{2n}\sigma_{\text{verify}}$. We can therefore appeal to the nearest colattice algorithm [EK20]. In particular [EK20, Thm. 4.3] implies a heuristic algorithm that allows one to solve the above approximate CVP with approximation factor $\gamma = \text{gh}(\beta)^{(n-1)/(\beta-1)}$ where $\text{gh}(\beta)$ is the expected first minimum of a random lattice with unit volume. This is achieved by the application of the DBKZ algorithm [MW16] with blocksize β and an exact CVP oracle for rank β lattices. In particular, one picks β minimal such that $\text{gh}(\beta)^{(n-1)/(\beta-1)} \leq \sqrt{2n}\sigma_{\text{verify}}$.



We ran progressive BKZ (one tour per blocksize) over $\mathbb{Z}^d$ using an input form generated with various σ_{ngen} and report the average successful β that recovered a length one vector over 40 instances. To allow us to vary σ_{ngen} gradually we sampled from a discrete Gaussian distribution, except for the centered binomial case ($\eta = 1$) at $\sigma_{\text{ngen}} = \sqrt{1/2}$. Note that the range of σ_{ngen} includes values “below smoothing”, for which the actual standard deviation $\widetilde{\sigma}_{\text{ngen}}$ can be significantly lower than the Gaussian width parameter σ_{ngen} .

Figure 2: Blocksize required to recover a shortest vector via lattice reduction as a function of the standard deviation $\widetilde{\sigma}_{\text{ngen}}$.

Table 7: Estimated BKZ blocksizes for cryptanalytic tasks using the BKZ simulator of [DDGR20].

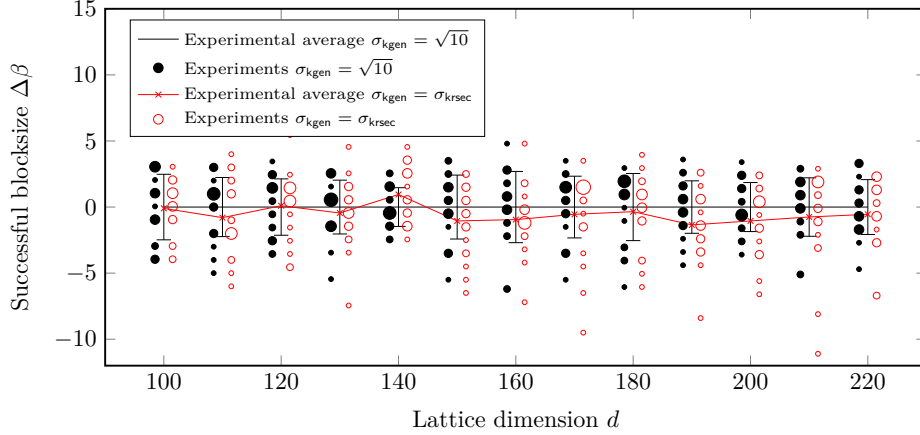
	β_{key}	$\beta_{\text{key}}(\text{d4f})$	β_{forge}	$\beta_{\text{forge}}(\text{d4f})$
HAWK-256	211	187	211	187
HAWK-512	452	412	452	412
HAWK-1024	940	873	1009	938

Crucial to this argument is that DBKZ with blocksize β heuristically finds vectors of length $\text{gh}(\beta)^{(n-1)/(\beta-1)} \cdot \text{vol}(2\mathbb{Z}^{2n})^{1/(2n)}$. When we use simulation for this approach we estimate the blocksize required in progressive BKZ with one tour per blocksize that finds vectors of this length. We report this β below and disregard the cost of the exact CVP oracle in rank β lattices.

5.3 Estimated blocksizes

In Table 7 we report the “raw” estimated blocksizes that simulation via [DDGR20] suggests are necessary for the cryptanalytic tasks of Sections 5.1 and 5.2, as well as the respective blocksizes once the dimensions for free technique has been taken into account [Duc18]. See functions `key_recovery_beta_ssec` and `approx_SVP_beta`.

We assume a single tour at blocksize β costs at least n calls to an SVP oracle of rank β for HAWK- n . From https://github.com/jschanck/eprint-2019-1161/blob/main/data/cost-estimate-list_decoding-classical.csv we have an estimated gate count for realising the SVP oracle of rank β via sieving [AGPS20], specifically the fastest known classical sieve [BDGL16]. Taking the lowest blocksizes in the rows associated to HAWK-512



We ran progressive BKZ (one tour per blocksize) over $\mathbb{Z}^d$ using an input form generated with $\sigma_{\text{kgen}} = \sqrt{10}$ or with $\sigma_{\text{kgen}} = \sigma_{\text{krsec}}$ and report the (average) successful β that recovered a length one vector over 20 instances. In order to sample at σ_{krsec} we sampled from a discrete Gaussian distribution instead of the binomial one. We normalised the blocksize with respect to the average successful blocksize when $\sigma_{\text{kgen}} = \sqrt{10}$.

Figure 3: Blocksize required to recover a shortest vector via lattice reduction as a function of dimension d for $\sigma_{\text{kgen}} = \sqrt{10}$ and $\sigma_{\text{kgen}} = \sigma_{\text{krsec}}$.

Table 8: Estimated BKZ blocksizes for cryptanalytic tasks using the GSA-intersect method of [AGVW17] for β_{key} and the nearest colattice approach [EK20] for β_{forge} .

	β_{key}	β_{key} (d4f)	β_{forge}	β_{forge} (d4f)
HAWK-256	220	195	205	181
HAWK-512	456	416	438	399
HAWK-1024	933	866	974	905

and HAWK-1024, and lowering them to the nearest multiple of eight to appeal to the concrete values from [AGPS20], we have estimated gate counts of 2^{141} and 2^{278} . Applying a factor of n to this cost gives us 2^{150} and 2^{288} . This still ignores other factors, such as the progressivity factor $C^2 \approx 2^5$ discussed in [SAB⁺22, Sec. 5.2.1], and the hidden overhead of the Becker–Ducas–Gama–Laarhoven sieves of a similar order of magnitude studied in [Duc22].

The above analysis only considers a pure BKZ attack, while a typical optimisation consists of applying a final SVP call in a slightly larger blocksize on the terminating block (a distinction sometime referred to as the uSVP versus BDD attack), to amortise the cost of $2n - \beta$ many SVP calls inside BKZ. This approach however consumes even more memory (which is already above 2^{100} bits for HAWK-512), for a limited gain on time (at most nC , and significantly less in practice).

While we expect simulation to give more accurate blocksizes, for reference we also provide in Table 8 the estimated blocksizes not using simulation, similar to the analysis of FALCON [PFH⁺22, Sec. 2.5.1]. However, in our version of this analysis we do not simplify some exponents, see <https://github.com/hawk-sign/aux/blob/main/code/falcon.sage> for a comparison of FALCON- $\{512, 1024\}$ and HAWK- $\{512, 1024\}$ in various analyses. We also do not apply the overconservative “double” dimensions for free in key recovery, as explained in [DPP⁺W22b, App. D]. If we apply the same conservative analysis as for Table 7 above to HAWK-512 and HAWK-1024 we calculate gate costs of 2^{145} and 2^{286} .

We might also consider the methodologies of [PFH⁺22, Sec. 2.5.1] and [LDK⁺22, Tab. 4] in deciding what is an acceptable blocksize, after taking dimensions for free into account, to achieve a particular security level. For example, **FALCON** considers $\beta = 374$ and $\beta = 869$ to be sufficient to achieve NIST-I and NIST-V, whereas **DILITHIUM** considers $\beta = 394$ and $\beta = 818$ sufficient for NIST-II and NIST-V. We have arrived at our blocksizes via different analyses; here we are solely considering the conversion from blocksize to security level.

We see that the minimum blocksizes, after dimensions for free have been applied, required to perform key recovery or forge across Tables 7 and 8 are 399 and 866 for **HAWK-512** and **HAWK-1024** respectively. While the lattice dimensions are slightly larger in **DILITHIUM**, by no more than a factor of two, all of our estimated blocksizes are at least as large for equal (or smaller) security levels. In the comparison to **FALCON** we see that **HAWK-512** has higher blocksizes for all cryptanalytic tasks, and β_{key} (d4f) for **HAWK-1024** is three smaller than for **FALCON-1024**. Given the margin expressed for the **FALCON-1024** parameters in [PFH⁺22, Sec. 2.5.1] this small decrease is acceptable.

6 Formal Security

In this section we provide formal justification for the security of HAWK. We start in Section 6.1 with local preliminaries. Then in Section 6.2 we formally introduce omSVP. In Sections 6.3 and 6.4 we provide reductions in the (quantum) random oracle model from HAWK to omSVP. We then introduce the search module lattice isomorphism problem, which is the problem upon which direct key recovery relies. Finally, we discuss these problems with respect to the parameters of HAWK and our design decisions.

6.1 Preliminaries

For a statement P we let $\llbracket P \rrbracket$ equal 1 if P is true, and 0 otherwise. We now define the smoothing parameter [MR04] of the integer lattice with respect to width parameter σ ; recall (3).

Definition 1. For $\varepsilon > 0$ the smoothing parameter $\eta_\varepsilon(\mathbb{Z}^n)$ is the minimal σ such that $\rho_{1/2\pi\sigma}(\mathbb{Z}^n \setminus \{0\}) \leq \varepsilon$.

Note η_ε is often defined with respect to a width parameter $s = \sqrt{2\pi}\sigma$ and therefore its value in Definition 1 is a factor of $\sqrt{2\pi}$ smaller than when defined with respect to s , and that $\mathbb{Z}^n$ is a self dual lattice. Also, $\eta_\varepsilon(\mathbb{Z}^n)$ is continuous and strictly increasing as ε decreases, with $\lim_{\varepsilon \rightarrow 0}(\eta_\varepsilon(\mathbb{Z}^n)) = \infty$. The smoothing parameter $\eta_\varepsilon(\mathbb{Z}^{2n})$ implicitly determines the Rényi divergence of the two distributions we consider in Lemma 1 and also determines the upper bounds on the probabilities in Lemma 2. In each case a smaller ε is better for our reduction, but requires a larger σ by the discussion above.

We fix the section $s: R_n/2R_n \rightarrow R_n$ of the natural surjection $R_n \rightarrow R_n/2R_n$ that has $s(a + 2R_n) = a_0 + a_1X + \dots + a_{n-1}X^{n-1}$ with $(a_0, \dots, a_{n-1}) \in \{0, 1\}^n$. We thus consider reduction mod 2 in R_n as a map $R_n \rightarrow R_n/2R_n \rightarrow R_n$ via s . In order to abstract away the technical details of sym-break we consider the function $\langle \cdot \rangle: R_n^2 \rightarrow R_n^2$ defined by

$$\mathbf{w} \mapsto \langle \mathbf{w} \rangle = \begin{cases} -\mathbf{w} & \text{if } \text{sym-break}(\mathbf{w}) = 1, \\ \mathbf{w} & \text{otherwise,} \end{cases}$$

where sym-break is as specified in Section 3.5.2. It is convenient to think of $\langle \mathbf{w} \rangle$ as a representation of the equivalence class $\{\mathbf{w}, -\mathbf{w}\}$, with the representation being unique if $\mathbf{w} \notin R_n \times \{0\}$. Indeed, what will be relevant is that

$$\langle \mathbf{w} \rangle = \langle -\mathbf{w} \rangle \in \{\mathbf{w}, -\mathbf{w}\} \quad \forall \mathbf{w} \notin R_n \times \{0\}, \quad (41)$$

while $\langle \mathbf{w} \rangle = \mathbf{w}$ for $\mathbf{w} \in R_n \times \{0\}$.

Definition 2. Given $\sigma > 0$ and $\mathbf{Q} = \mathbf{B}^*\mathbf{B}$ for some $\mathbf{B} \in \text{GL}_2(R_n)$, let

$$\rho_{\mathbf{Q},\sigma}: R_n^2 \rightarrow \mathbb{R}, \mathbf{x} \mapsto \exp(-\|\mathbf{x}\|_{\mathbf{Q}}^2/2\sigma^2).$$

Note here that for $\mathbf{Q} = \mathbf{B}^*\mathbf{B}$, we have $\rho_{\mathbf{Q},\sigma}(\mathbf{x}) = \rho_\sigma(\mathbf{B} \cdot \mathbf{x})$ by (22).

Definition 3. Let $\mathbf{B} \in \text{GL}_2(R_n)$ and $\mathbf{Q} = \mathbf{B}^*\mathbf{B}$. For every $\sigma > 0$ and $\mathbf{h} \in (R_n/2R_n)^2 \subset R_n^2$ we consider the following $\mathbf{Q}$ dependent distributions.

- $D_{\mathbf{Q},\sigma}$, the discrete Gaussian distribution over R_n^2 under $\|\cdot\|_{\mathbf{Q}}$ given by

$$D_{\mathbf{Q},\sigma}: R_n^2 \rightarrow \mathbb{R}, \mathbf{x} \mapsto \rho_{\mathbf{Q},\sigma}(\mathbf{x}) / \sum_{\mathbf{y} \in R_n^2} \rho_{\mathbf{Q},\sigma}(\mathbf{y}).$$

- $\tilde{D}_{\mathbf{Q},\sigma}[\mathbf{h}]$, the discrete Gaussian distribution over $\mathbf{h} + 2R_n^2 \subset R_n^2$ under $\|\cdot\|_{\mathbf{Q}}$ given by

$$\tilde{D}_{\mathbf{Q},\sigma}[\mathbf{h}] : \mathbf{h} + 2R_n^2 \rightarrow \mathbb{R}, \mathbf{x} \mapsto \rho_{\mathbf{Q},\sigma}(\mathbf{x}) / \sum_{\mathbf{y} \in \mathbf{h} + 2R_n^2} \rho_{\mathbf{Q},\sigma}(\mathbf{y}).$$

- $\tilde{D}_{\mathbf{Q},\sigma}$, the distribution implied by sampling a uniform $\mathbf{h} \leftarrow (R_n/2R_n)^2$ and returning a sample from $\tilde{D}_{\mathbf{Q},\sigma}[\mathbf{h}]$.

Given $\mathbf{B}$ we may sample the above distributions by sampling according to $D_{\mathbf{I}_2(K_n),\sigma}$, in the coset defined by $\mathbf{t} = \mathbf{B}\mathbf{h}$ if required, and multiplying by $\mathbf{B}^{-1}$ as in HAWK signing, e.g. Algorithm 2. Note that for every $\mathbf{h} \in (R_n/2R_n)^2$

$$\forall \mathbf{w} \in \text{Supp}(\tilde{D}_{\mathbf{Q},\sigma}[\mathbf{h}]) = \mathbf{h} + 2R_n^2, \mathbf{w} \bmod 2 = \mathbf{h}. \quad (42)$$

We call the maximum probability of any element in the support of a distribution the guessing probability of the distribution. As a direct consequence of [DPPvW22a, Lemma 3], the guessing probability of $\mathbf{w} \bmod 2$ for $\mathbf{w} \leftarrow D_{\mathbf{Q},2\sigma}$ with $\sigma \geq \eta_\varepsilon(\mathbb{Z}^{2n})$ has the following upper bound

$$\text{guess}(\mathbf{w} \bmod 2) = \max_{\mathbf{h}^\circ \in (R_n/2R_n)^2} \Pr[\mathbf{w} \bmod 2 = \mathbf{h}^\circ] \leq 2^{-2n} \cdot \frac{1 + \varepsilon}{1 - \varepsilon}. \quad (43)$$

We will also make use of the Rényi divergence between $\tilde{D}_{\mathbf{Q},2\sigma}$ and $D_{\mathbf{Q},2\sigma}$, which we compute in the following lemma.

Lemma 1. *Let $\sigma > 0$, $\mathbf{Q} = \mathbf{B}^* \mathbf{B}$ for some $\mathbf{B} \in \text{GL}_2(R_n)$ and $a \in (1, \infty)$ be an order. Furthermore, denote by*

$$\alpha = \frac{\rho_{2\sigma}(\mathbb{Z})}{2\rho_\sigma(\mathbb{Z})}, \text{ and } \beta = \frac{\rho_{2\sigma}(\mathbb{Z})}{2\rho_\sigma(\mathbb{Z} + \frac{1}{2})},$$

then

$$R_a(\tilde{D}_{\mathbf{Q},2\sigma} \| D_{\mathbf{Q},2\sigma}) = \left(\frac{\alpha^{a-1} + \beta^{a-1}}{2} \right)^{\frac{2n}{a-1}}.$$

Proof. Since each $\mathbf{w} \in R_n^2$ lies in $2R_n^2 + \mathbf{h}$ for exactly one $\mathbf{h} \in (R_n/2R_n)^2$ we have

$$\tilde{D}_{\mathbf{Q},2\sigma}(\mathbf{w}) = 2^{-2n} \rho_{\mathbf{Q},2\sigma}(\mathbf{w}) / \rho_{\mathbf{Q},2\sigma}(2R_n^2 + \mathbf{h}).$$

Similarly we have

$$D_{\mathbf{Q},2\sigma}(\mathbf{w}) = \rho_{\mathbf{Q},2\sigma}(\mathbf{w}) / \rho_{\mathbf{Q},2\sigma}(R_n^2).$$

Hence, by definition of the Rényi divergence,

$$\begin{aligned} R_a(\tilde{D}_{\mathbf{Q},2\sigma} \| D_{\mathbf{Q},2\sigma})^{a-1} &= \sum_{\mathbf{h} \in (R_n/2R_n)^2} \sum_{\mathbf{w} \in 2R_n^2 + \mathbf{h}} \tilde{D}_{\mathbf{Q},2\sigma}(\mathbf{w})^a / D_{\mathbf{Q},2\sigma}(\mathbf{w})^{a-1} \\ &= \sum_{\mathbf{h} \in (R_n/2R_n)^2} 2^{-2n} \cdot \left(\frac{\rho_{\mathbf{Q},2\sigma}(R_n^2)}{2^{2n} \rho_{\mathbf{Q},2\sigma}(2R_n^2 + \mathbf{h})} \right)^{a-1} \sum_{\mathbf{w} \in 2R_n^2 + \mathbf{h}} \frac{\rho_{\mathbf{Q},2\sigma}(\mathbf{w})}{\rho_{\mathbf{Q},2\sigma}(2R_n^2 + \mathbf{h})}. \end{aligned}$$

Note that $\sum_{\mathbf{w}} \rho_{\mathbf{Q},2\sigma}(\mathbf{w}) / \rho_{\mathbf{Q},2\sigma}(2R_n^2 + \mathbf{h}) = 1$, simplifying the equation to

$$\begin{aligned} R_a(\tilde{D}_{\mathbf{Q},2\sigma} \| D_{\mathbf{Q},2\sigma})^{a-1} &= 2^{-2n} \sum_{\mathbf{h} \in (R_n/2R_n)^2} \left(\frac{\rho_{\mathbf{Q},2\sigma}(R_n^2)}{2^{2n} \rho_{\mathbf{Q},2\sigma}(2R_n^2 + \mathbf{h})} \right)^{a-1} \\ &= 2^{-2n} \sum_{\mathbf{h} \in (R_n/2R_n)^2} \left(\frac{\rho_{2\sigma}(R_n^2)}{2^{2n} \rho_{2\sigma}(2R_n^2 + \mathbf{B} \cdot \mathbf{h})} \right)^{a-1}, \end{aligned}$$

where the second equality relies on the factor that $\mathbf{B} \in \text{GL}_2(R_n)$ and therefore that $\mathbf{B}R_n^2 = R_n^2$. We also note that $\left\{2R_n^2 + \mathbf{B}\mathbf{h} \bmod 2 : \mathbf{h} \in (R_n/2R_n)^2\right\} = \left\{2R_n^2 + \mathbf{t} : \mathbf{t} \in (R_n/2R_n)^2\right\}$. Therefore, we may sum over $\mathbf{t} \in (R_n/2R_n)^2$. We also here pass to the unstructured setting, noting that $\rho_{2\sigma}(R_n^2) = \rho_{2\sigma}(2\mathbb{Z}^{2n})$ and that, since $\mathbb{Z}^{2n}$ has an orthogonal basis $\mathbf{I}_{2n}(\mathbb{Z})$, $\rho_{2\sigma}(2\mathbb{Z}^{2n}) = \rho_{2\sigma}(2\mathbb{Z})^{2n}$. Similarly, we have $\rho_{2\sigma}(2R_n^2 + \mathbf{t}) = \rho_{2\sigma}(2\mathbb{Z}^{2n} + \mathbf{t}) = \prod_i \rho_{2\sigma}(2\mathbb{Z} + t_i)$, with $\mathbf{t} \in \{0, 1\}^{2n}$ after the first equality. Thus,

$$\begin{aligned} R_a(\tilde{D}_{\mathbf{Q}, 2\sigma} \| D_{\mathbf{Q}, 2\sigma})^{a-1} &= 2^{-2n} \sum_{\mathbf{t} \in \{0, 1\}^{2n}} \left(\frac{\rho_{2\sigma}(\mathbb{Z}^{2n})}{2^{2n} \rho_{2\sigma}(2\mathbb{Z}^{2n} + \mathbf{t})} \right)^{a-1} \\ &= 2^{-2n} \sum_{\mathbf{t} \in \{0, 1\}^{2n}} \prod_{i=0}^{2n-1} \left(\frac{\rho_{2\sigma}(\mathbb{Z})}{2\rho_{2\sigma}(2\mathbb{Z} + t_i)} \right)^{a-1}. \end{aligned}$$

By swapping these (finite) sums and products, and considering the contribution of a single t_i , half of which are zero and half of which are one over $\mathbf{t} \in \{0, 1\}^{2n}$ for any fixed index i , we have

$$\begin{aligned} R_a(\tilde{D}_{\mathbf{Q}, 2\sigma} \| D_{\mathbf{Q}, 2\sigma})^{a-1} &= \prod_{i=0}^{2n-1} \frac{1}{2} \cdot \left[\left(\frac{\rho_{2\sigma}(\mathbb{Z})}{2\rho_{2\sigma}(2\mathbb{Z})} \right)^{a-1} + \left(\frac{\rho_{2\sigma}(\mathbb{Z})}{2\rho_{2\sigma}(2\mathbb{Z} + 1)} \right)^{a-1} \right] \\ &= \left[\frac{1}{2} \cdot \left(\left(\frac{\rho_{2\sigma}(\mathbb{Z})}{2\rho_{2\sigma}(2\mathbb{Z})} \right)^{a-1} + \left(\frac{\rho_{2\sigma}(\mathbb{Z})}{2\rho_{2\sigma}(2\mathbb{Z} + 1)} \right)^{a-1} \right) \right]^{2n} \end{aligned}$$

As $\rho_{2\sigma}(2\mathbb{Z} + b) = \rho_{\sigma}(\mathbb{Z} + b/2)$, one can substitute α and β in the above. $\square$

As σ grows the Rényi divergence decreases towards one. When σ is not too big, α and β in the above lemma can be efficiently numerically computed to any desired precision via a tailcut.

6.1.1 Roots of Unity

The $2n$ roots of unity of R_n are denoted by μ_K and given by $\{1, X, X^2, \dots, X^{2n-1}\}$. The following lemma is extracted from the proof of [DPPvW22b, Lem. 10] and a replica (up to a variable name change) of [FH23, Lem. 1]; see the latter for a detailed proof.

Lemma 2. *Let n be a power of 2, $\varepsilon > 0$, $\sigma \geq \eta_\varepsilon(\mathbb{Z}^{2n})$, and $\mathbf{h}^\circ \in (R_n/2R_n)^2$. Consider $\mathbf{w} \leftarrow D_{\mathbf{Q}, 2\sigma}$ and set $\mathbf{h} = \mathbf{w} \bmod 2$. Then*

$$\Pr \left[\exists \alpha \in \mu_K \setminus \{\pm 1\} : \frac{1}{2}(\mathbf{h} + \alpha \mathbf{w}) \in R_n^2 \right] \leq 2^{-n} \cdot \frac{1 + \varepsilon}{1 - \varepsilon}, \text{ and} \quad (44)$$

$$\Pr \left[\exists \alpha \in \mu_K : \frac{1}{2}(\mathbf{h}^\circ + \alpha \mathbf{w}) \in R_n^2 \right] \leq n \cdot 2^{-2n} \cdot \frac{1 + \varepsilon}{1 - \varepsilon}. \quad (45)$$

6.1.2 Symplectic automorphism

In [LJPW24] it is observed that, given $\mathbf{Q}$, there is an efficiently computable symplectic automorphism. Denote the Hermitian adjoint of a matrix (or vector) $\mathbf{B} \in K_n^{r \times r'}$ by $\overline{\mathbf{B}} = (\mathbf{B}^*)^\mathbf{t}$, which is obtained by applying the Hermitian adjoint of (11) to each entry of $\mathbf{B}$ (and not to $\mathbf{B}^\mathbf{t}$). Reusing notation from [LJPW24] let

$$\mathbf{J}_2 = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix},$$

and observe $\mathbf{J}_2^2 = -\mathbf{I}_2(K)$. Via (25) we relate the dual $\mathbf{D} = (\mathbf{B}^{-1})^*$ and primal bases

$$\mathbf{D} = (\mathbf{B}^{-1})^* = (-\mathbf{J}_2 \mathbf{B}^\dagger \mathbf{J}_2)^* = -\mathbf{J}_2 \cdot \overline{\mathbf{B}} \cdot \mathbf{J}_2.$$

Suppose vector $\mathbf{z} = \mathbf{B} \cdot \mathbf{x}$ has small norm $\|\mathbf{x}\|_{\mathbf{Q}} = \|\mathbf{B}\mathbf{x}\|$. For $\mathbf{z}' = \mathbf{J}_2 \cdot \overline{\mathbf{z}}$ it follows from (17) and inspection that $\|\mathbf{z}\| = \|\mathbf{z}'\|$. Rewriting yields

$$\mathbf{z}' = \mathbf{J}_2 \cdot \overline{\mathbf{B} \cdot \mathbf{x}} = \mathbf{J}_2 \cdot \overline{\mathbf{B}} \cdot \overline{\mathbf{x}} = \mathbf{D} \cdot \mathbf{J}_2 \cdot \overline{\mathbf{x}}.$$

Expressing $\mathbf{z}'$ now in terms of the primal basis, and using only the public Hermitian matrix $\mathbf{Q}$, another lattice point is obtained of the same norm

$$\omega(\mathbf{x}) = \mathbf{B}^{-1} \mathbf{D} \mathbf{J}_2 \cdot \overline{\mathbf{x}} = \mathbf{Q}^{-1} \mathbf{J}_2 \cdot \overline{\mathbf{x}}.$$

This defines an additive, bijective map $\omega: R_n^2 \rightarrow R_n^2$, which is *not* R_n -linear as $\omega(\alpha \mathbf{x}) = \overline{\alpha} \omega(\mathbf{x})$ for $\alpha \in R_n$. It is however an *isometry* with respect to $\mathbf{Q}$, i.e. for all $\mathbf{x} \in R_n^2$,

$$\|\mathbf{x}\|_{\mathbf{Q}} = \|\omega(\mathbf{x})\|_{\mathbf{Q}}.$$

Let us define the group of isometries $R_n^2 \rightarrow R_n^2$ that are easily computable from $\mathbf{Q}$ by

$$\mathcal{G}_K = \langle X \cdot \mathbf{I}_2(K), \omega \rangle = \bigcup_{i=0}^{2n-1} \{X^i \mathbf{I}_2(K), X^i \omega\},$$

which is isomorphic to the dihedral group D_{2n} of order $4n$. The following lemma is a modification of Lemma 2, taking the symplectic automorphism ω into account.

Lemma 3. *Let n be a power of 2, $\varepsilon > 0$, $\sigma \geq \eta_\varepsilon(\mathbb{Z}^{2n})$, and $\mathbf{h}^\circ \in (R_n/2R_n)^2$. Consider $\mathbf{w} \leftarrow D_{\mathbf{Q}, 2\sigma}$ and set $\mathbf{h} = \mathbf{w} \bmod 2$. Then,*

$$\Pr [\exists \alpha \in \mathcal{G}_K \setminus \{\pm 1\} : \tfrac{1}{2}(\mathbf{h} + \alpha(\mathbf{w})) \in R_n^2] \leq (n+1) \cdot 2^{-n} \cdot \frac{1+\varepsilon}{1-\varepsilon}, \text{ and} \quad (46)$$

$$\Pr [\exists \alpha \in \mathcal{G}_K : \tfrac{1}{2}(\mathbf{h}^\circ + \alpha(\mathbf{w})) \in R_n^2] \leq 2n \cdot 2^{-2n} \cdot \frac{1+\varepsilon}{1-\varepsilon}. \quad (47)$$

Proof. Equation (46). We split $\mathcal{G}_K = \mu_K \cup \{\alpha \omega \mid \alpha \in \mu_K\}$ and use a union bound. The probability there exists $\alpha \in \mu_K \setminus \{\pm 1\}$ is bounded by Lemma 2. For the other set, similar to the proof of [FH23, Lem. 1], we have

$$\begin{aligned} & \Pr [\exists \alpha \in \mu_K : \tfrac{1}{2}(\mathbf{h} + \alpha \omega(\mathbf{w})) \in R_n^2] \\ & \leq 2^{-2n} \frac{1+\varepsilon}{1-\varepsilon} \cdot \sum_{i=0}^{n-1} \# \left\{ \mathbf{w} \in (R_n/2R_n)^2 \mid \mathbf{w} \equiv X^i \omega(\mathbf{w}) \bmod 2R_n^2 \right\}. \end{aligned}$$

For all $i \in \{0, 1, \dots, n-1\}$ and $\mathbf{w} = \begin{pmatrix} w_0 \\ w_1 \end{pmatrix}$, we have $\mathbf{w} \equiv X^i \omega(\mathbf{w}) \bmod 2R_n^2$ if and only if both $w_0 \equiv X^i w_1^* \bmod 2R_n$ and $w_1 \equiv X^i w_0^* \bmod 2R_n$ hold. A choice for $w_0 \in R_n/2R_n$ uniquely determines $w_1 \in R_n/2R_n$, and the second condition is equivalent to the first: $w_1 \equiv X^i w_0^*$ if and only if $w_1^* \equiv X^{-i} w_0$. Thus, for each $i \in \{0, 1, \dots, n-1\}$

$$\# \left\{ \mathbf{w} \in (R_n/2R_n)^2 \mid \mathbf{w} \equiv X^i \omega(\mathbf{w}) \bmod 2R_n^2 \right\} = 2^n.$$

Equation (47). Similar to [FH23, Lem. 1] note for any $\alpha \in \mathcal{G}_K$ that $\tfrac{1}{2}(\mathbf{h}^\circ + \alpha(\mathbf{w})) \in R_n^2$ if and only if $\mathbf{w} \equiv \alpha^{-1}(\mathbf{h}^\circ) \bmod 2R_n^2$ holds. Since multiplication by -1 acts trivially modulo 2, there are at most $2n$ cosets of $2R_n^2$ to consider for $\mathbf{w}$ such that $\tfrac{1}{2}(\mathbf{h}^\circ \mathbf{w} + \alpha(\mathbf{w})) \in R_n^2$ holds. Together with (43), this proves (47). $\square$

6.1.3 Adaptive Reprogramming Lemma

The following reprogramming lemma is immediate from [GHHM21, Thm. 1]. Intuitively, it states that if the location x of a reprogramming of the random oracle is hard to guess prior to it occurring then such a reprogramming is hard to notice.

Lemma 4 (Adaptive Reprogramming Lemma from [GHHM21, Thm. 1]). *Let $H : \mathcal{X} \rightarrow \mathcal{Y}$ be a random oracle, $\varepsilon > 0$ and Ω be a family of distributions on $\mathcal{X}$ where every $\mathcal{D} \in \Omega$ has guessing probability $\text{guess}(\mathcal{D}) = \max_{x^\circ} \Pr_{x \leftarrow \mathcal{D}} [x = x^\circ] \leq \gamma$. Define the reprogramming oracle Repro_b for $b \in \{0, 1\}$ that, on input (a suitable representation of) $\mathcal{D} \in \Omega$, functions as*

$\text{Repro}_0(\mathcal{D})$	$\text{Repro}_1(\mathcal{D})$
1: $x \leftarrow \mathcal{D}$	1: $x \leftarrow \mathcal{D}$
2: $y = H(x)$	2: $H(x) = y \leftarrow \mathcal{Y}$
3: return (x, y)	3: return (x, y)

Suppose $\mathcal{A}^{\text{Repro}_b, H}$ for $b \in \{0, 1\}$ makes at most q_r queries to the reprogramming oracle Repro_b , and at most q_h quantum queries to H before the last reprogramming query. Then,

$$\left| \Pr [1 \leftarrow \mathcal{A}^{\text{Repro}_0, H}] - \Pr [1 \leftarrow \mathcal{A}^{\text{Repro}_1, H}] \right| \leq q_r \sqrt{(q_h + q_r) \cdot \gamma} + \frac{q_r(q_h + q_r) \cdot \gamma}{2}.$$

6.1.4 SUF-CMA security

For a signature scheme $\Pi = (\text{Sign}, \text{Vf}, \text{KGen})$ we define the game (Q)ROM-SUF-CMA $_{\Pi, \mathcal{A}}$ in Fig. 4. For a signature scheme to exhibit strong unforgeability under chosen message attacks no (quantum) probabilistic polynomial time adversary against (Q)ROM-SUF-CMA $_{\Pi, \mathcal{A}}$ should win with non negligible probability. The prefix (Q)ROM indicates that the adversary gets (quantum) access to the random oracle H .

(Q)ROM-SUF-CMA $_{\Pi, \mathcal{A}}(1^\lambda)$
1: $\mathcal{L}_{\text{Sign}} \leftarrow \emptyset$
2: $(\text{pk}, \text{sk}) \leftarrow \text{KGen}^H(1^\lambda)$
3: $(m^*, \text{sig}^*) \leftarrow \mathcal{A}^{\text{OSign}, H}(1^\lambda, \text{pk})$
4: return $\llbracket \text{Vf}_{\text{pk}}^H(m^*, \text{sig}^*) = 1 \wedge (m^*, \text{sig}^*) \notin \mathcal{L}_{\text{Sign}} \rrbracket$

OSign(m)
1: $\text{sig} \leftarrow \text{Sign}_{\text{sk}}^H(m)$
2: $\mathcal{L}_{\text{Sign}} \leftarrow \mathcal{L}_{\text{Sign}} \cup \{(m, \text{sig})\}$
3: return sig

Figure 4: The (Q)ROM-SUF-CMA game.

Definition 4 ((Q)ROM-SUF-CMA security). Let $\Pi = (\text{KGen}, \text{Sign}, \text{Vf})$ be a signature scheme. Let t, ε, q_s, q_h be functions of λ . We say that Π is $(t, \varepsilon, q_s, q_h)$ -(Q)ROM-SUF-CMA secure, or strongly unforgeable under chosen message attacks in the (quantum) random oracle model, if for any adversary $\mathcal{A}$ running in time at most t , making at most q_s queries to its signing oracle, and making at most q_h queries to its random oracle, the probability $\text{Adv}_{\Pi, \mathcal{A}}^{\text{SUF-CMA}}(\lambda) := \Pr[(\text{Q)ROM-SUF-CMA}_{\Pi, \mathcal{A}}(1^\lambda) = 1] \leq \varepsilon(\lambda)$ for all λ .

$\text{SAMPLE}_{\text{ac-omSVP}, \mathcal{A}}(1^\lambda)$	samp
1 : $\mathcal{L}_{\text{samples}} \leftarrow \{\mathbf{0}\}$	1 : $\mathbf{w} \leftarrow D_{\mathbf{Q}, \sigma}$
2 : $(\mathbf{Q}, \mu_K, L, \sigma) \leftarrow \text{Init}(1^\lambda)$	2 : $\mathcal{L}_{\text{samples}} \leftarrow \mathcal{L}_{\text{samples}} \cup \{\alpha(\mathbf{w})\}_{\alpha \in \mathcal{G}_K}$
3 : $\mathbf{w}^* \leftarrow \mathcal{A}^{\text{samp}}(1^\lambda, \mathbf{Q}, \mu_K, L, \sigma)$	3 : return $\mathbf{w}$
4 : return $\llbracket \mathbf{w}^* \in R^r \wedge \ \mathbf{w}^*\ _{\mathbf{Q}} \leq L \wedge \mathbf{w}^* \notin \mathcal{L}_{\text{samples}} \rrbracket$	

Figure 5: The SAMPLE game.

In the above definition we have (mostly) dropped the dependence of $\mathcal{A}$ and t, ε, q_s, q_h on λ . The superscript in $\text{Adv}_{\Pi, \mathcal{A}}^{\text{SUF-CMA}}(\lambda)$ does not read (Q)ROM-SUF-CMA, and instead only SUF-CMA, for typographical aesthetics. Also, outside of definitions, we drop the dependence on the security parameter; $\text{Adv}_{\Pi, \mathcal{A}}^{\text{SUF-CMA}}$.

6.2 The one more shortest vector problem

Informally an average case omSVP instance samples a $\mathbf{Q}$ from a distribution over some $\mathcal{H}_r^{>0}(K)$ and gives Gaussian samples according to this $\mathbf{Q}$. Here, for a CM-field K and $r \in \mathbb{Z}_{\geq 1}$, we define $\mathcal{H}_r^{>0}(K) \subset K^{r \times r}$ as the $\mathbf{Q} \in K^{r \times r}$ such that $\mathbf{Q} = \mathbf{Q}^*$ and $\text{Tr}(\mathbf{v}^* \mathbf{Q} \mathbf{v}) > 0$ for all $\mathbf{v} \in K^r \setminus \{\mathbf{0}\}$, i.e. matrices that represent Hermitian positive definite forms. If one can find some non zero $\mathbf{x}$ in the ring of integers R of K that is sufficiently short with respect to $\mathbf{Q}$, and that is in some sense non trivially new, then one solves the problem. We show that for certain parameters, if one can forge a signature against HAWK then in the programmable quantum random oracle model one can solve an instance of this problem.

Definition 5 (Average case omSVP). An average case omSVP instance is the pair $\text{ac-omSVP} = (\text{Init}, \text{samp})$. Init returns a form $\mathbf{Q}$ sampled from some distribution over some $\mathcal{H}_r^{>0}(K)$, the roots of unity μ_K for K , a length bound L , and a Gaussian parameter σ , where r, K, L, σ are functions of λ . Each call to samp returns a sample from $D_{\mathbf{Q}, \sigma}$.

The adversary in Fig. 5 wins whenever it can use the output of Init and the samples it receives from samp to return some non trivial new element of R^r that is short enough.

Definition 6 (SAMPLE security). Let $\text{ac-omSVP} = (\text{Init}, \text{samp})$ be an average case omSVP instance. Let t, ε, q_o be functions of λ . We say that ac-omSVP is (t, ε, q_o) -SAMPLE secure, if for any adversary $\mathcal{A}$ running in time at most t , and making at most q_o queries to samp , the probability $\text{Adv}_{\text{ac-omSVP}, \mathcal{A}}^{\text{SAMPLE}}(\lambda) := \Pr[\text{SAMPLE}_{\text{ac-omSVP}, \mathcal{A}}(1^\lambda) = 1] \leq \varepsilon(\lambda)$ for all λ .

6.3 Quantum security of HAWK

Let $\eta, \sigma_{\text{sign}}, \sigma_{\text{verify}}$, and $\text{saltlen}_{\text{bits}}$ be the parameters as specified in Section 3.2. Let ε be minimal such that $\sigma_{\text{sign}} \geq \eta_\varepsilon(\mathbb{Z}^{2n})$. We write HawkKeyGen for the key generation procedure of Algorithm 1. It produces a key pair $(\mathbf{Q}, (\mathbf{B}, \text{hpub}))$ with $\mathbf{B} \in \text{GL}_2(R_n)$ and $\mathbf{Q} = \mathbf{B}^* \mathbf{B}$. We first analyse a simplified variant of HAWK with signing Sign_{sk} and verification Vf_{pk} as in Fig. 6. Hence, define

$$\Pi_{\text{HAWK}} = (\text{HawkKeyGen}, \text{Sign}, \text{Vf}). \quad (48)$$

We discuss these simplifications below, and in Section 6.3.3 consider HAWK without these simplifications, as in Section 2.1.

$\text{Sign}_{(\mathbf{B}, \text{hpub})}(M)$	$\text{Vf}_{\mathbf{Q}}(M, \text{sig})$
1 : $\text{salt} \leftarrow \text{Rnd}(\text{saltlen}_{\text{bits}})$	1 : $(\text{salt}, \mathbf{s}) := \text{sig}$
2 : $\mathbf{h} = \text{H}(M \parallel \text{salt})$	2 : $\mathbf{h} := \text{H}(M \parallel \text{salt})$
3 : $\mathbf{w} \leftarrow \tilde{D}_{\mathbf{Q}, 2\sigma_{\text{sign}}}[\mathbf{h}]$	3 : $\mathbf{w} := 2\mathbf{s} - \mathbf{h}$
4 : $\mathbf{s} := \frac{1}{2}(\mathbf{h} + \langle \mathbf{w} \rangle) \in R_n^2$	4 : check $\text{sig} \in \{0, 1\}^{\text{saltlen}_{\text{bits}}} \times R_n^2$
5 : return $\text{sig} := (\text{salt}, \mathbf{s})$	5 : check $\mathbf{w} = \langle \mathbf{w} \rangle$ and $\mathbf{w} \notin R_n \times \{0\}$
	6 : check $\ \mathbf{w}\ _{\mathbf{Q}} \leq 2\sigma_{\text{ver}} \cdot \sqrt{2n}$
	7 : return 1 if all checks pass

Figure 6: Simplified HAWK.

The description in Fig. 6 matches the specification of HAWK (see Section 2.1 and in particular Algorithms 2 and 3) up to some small changes.

Several operations look different, but are equivalent. In particular, the sampling of $\mathbf{w}$ in both descriptions coincides, and $\mathbf{s} := \frac{1}{2}(\mathbf{h} + \langle \mathbf{w} \rangle)$ captures the sign flip of $\mathbf{w}$ when $\text{sym-break}(\mathbf{w})$ is false. Finally, whenever the check $\text{sym-break}(\mathbf{h} - 2\mathbf{s})$ in Algorithm 2 is unsatisfied, it is because either $\mathbf{h} - 2\mathbf{s} = -\mathbf{w} \in R_n \times \{0\}$ and therefore $\mathbf{w} \in R_n \times \{0\}$ also, or the first nonzero coefficient of the second component of $-\mathbf{w}$ is negative, in which case $\langle \mathbf{w} \rangle = -\mathbf{w} \neq \mathbf{w}$. Hence the check $\mathbf{w} = \langle \mathbf{w} \rangle$ and $\mathbf{w} \notin R_n \times \{0\}$ in Vf is equivalent to checking $\text{sym-break}(\mathbf{h} - 2\mathbf{s})$ as in Algorithm 2.

We now enumerate the simplifications, where we describe what happens in Section 2.1 but not in Fig. 6. First, in Algorithm 2 signing is restarted whenever $\mathbf{w}$ is too long or when various encoding requirements are not satisfied. Second, a hash $M = \text{H}(\mathbf{m} \parallel \text{hpub})$ is computed initially in Algorithm 2. Finally, signatures are compressed in Algorithm 2, and decompressed in Algorithm 3.

Consider a (Q)ROM-SUF-CMA attacker $\mathcal{A}^{\text{OSign}, \text{H}}(\text{pk})$ against Π_{HAWK} , which on input the public key makes at most q_h (quantum) queries to the random oracle H and at most q_s queries to the signing oracle OSign, and eventually outputs a message forgery pair (M^*, sig^*) with $\text{sig}^* = (\text{salt}^*, \mathbf{s}^*) \in \{0, 1\}^{\text{saltlen}_{\text{bits}}} \times R_n^2$. Without loss of generality, we assume $\mathcal{A}$ makes exactly (q_s, q_h) queries to (OSign, H) respectively.⁵ The goal is to turn $\mathcal{A}$ into an algorithm $\mathcal{B}$ that solves an instance of ac-omSVP, in which $\text{Init}(1^\lambda)$ produces $\sigma = 2\sigma_{\text{sign}}$, $L = 2\sigma_{\text{ver}} \cdot \sqrt{2n}$, μ_K as in Section 6.1.1, and $\mathbf{Q}$ sampled via $(\mathbf{Q}, \text{sk}) \leftarrow \text{HawkKeyGen}$. From here we take it as understood that ac-omSVP = (Init, samp) is as described above.

Theorem 1 (Quantum security of (simplified) Π_{HAWK}). *Let $\mathcal{A}$ be an adversary against the (Q)ROM-SUF-CMA game making at most q_s queries to OSign and at most q_h quantum queries to H. Then there exists an adversary $\mathcal{B}$ against the SAMPLE game making $q_o = q_s$ queries to samp, with running time $\text{TIME}(\mathcal{B}) = \text{TIME}(\mathcal{A}) + \text{Overhead}(q_s, q_h)$. This overhead consists of simulating q_h hash queries to H and q_s queries to Sim (specified in Fig. 7). Furthermore, for each $a \in (1, \infty)$ we have*

$$\text{Adv}_{\Pi_{\text{HAWK}}, \mathcal{A}}^{\text{SUF-CMA}} \leq \left[R_a(\tilde{D}_{\mathbf{Q}, 2\sigma_{\text{sign}}} \parallel D_{\mathbf{Q}, 2\sigma_{\text{sign}}})^{q_s} \cdot \left(\text{Adv}_{\text{ac-omSVP}, \mathcal{B}}^{\text{SAMPLE}} + O(q_h^2 \cdot n \cdot q_s / 2^{2n}) \right) \right]^{1-1/a} \\ + q_s \sqrt{q_h + q_s} \cdot 2^{-\text{saltlen}_{\text{bits}}/2} + q_s(q_h + q_s) \cdot 2^{-\text{saltlen}_{\text{bits}}-1}.$$

The constant hidden by the O is independent of the choice of a .

⁵Otherwise, we let $\mathcal{A}$ make dummy queries to H and OSign respectively, with the dummy queries to OSign being on messages different from M^* , so that they do not affect the freshness of a forgery.

6.3.1 Simulating the signing queries

We first show we can replace the signing oracle OSign that $\mathcal{A}$ has access to with a simulator Sim that does not know the secret key sk , but instead may sample the $\mathbf{Q}$ dependent distribution $D_{\mathbf{Q}, 2\sigma_{\text{sign}}}$, and that can reprogram the random oracle $\mathbf{H}$; see Fig. 7 (right) below. This, in essence, means that Sim can be replaced by adversary $\mathcal{B}$ against ac-omSVP that has access to the samp oracle and can reprogram $\mathbf{H}$. As an intermediate step we consider Trans_{sk} as specified in Fig. 7 (left). Letting SUF-CMA-Trans and SUF-CMA-Sim denote the games where the OSign oracle has been replaced by Trans and Sim respectively, we describe the difference in advantage of $\mathcal{A}$ and in particular show

$$\text{Adv}_{\Pi_{\text{HAWK}}, \mathcal{A}}^{\text{SUF-CMA}} \approx \text{Adv}_{\Pi_{\text{HAWK}}, \mathcal{A}}^{\text{SUF-CMA-Trans}} \approx \text{Adv}_{\Pi_{\text{HAWK}}, \mathcal{A}}^{\text{SUF-CMA-Sim}}.$$

Note that via OSign the adversary $\mathcal{A}$ depends on sk , and similarly for $\mathcal{A}$ against Trans , which we have highlighted with a subscript. For convenience we may later omit this subscript.

$\text{Trans}_{(\mathbf{B}, \text{hpub})}(M)$	$\text{Sim}^{D_{\mathbf{Q}, 2\sigma_{\text{sign}}}}(M)$
1 : $\text{salt} \leftarrow \text{Rnd}(\text{saltlen}_{\text{bits}})$	1 : $\text{salt} \leftarrow \text{Rnd}(\text{saltlen}_{\text{bits}})$
2 : $\mathbf{h} \leftarrow (R_n/2R_n)^2$	2 : $\mathbf{w} \leftarrow D_{\mathbf{Q}, 2\sigma_{\text{sign}}}$
3 : $\mathbf{H}(M \parallel \text{salt}) := \mathbf{h}$	3 : $\mathbf{h} := \mathbf{w} \bmod 2$
4 : $\mathbf{w} \leftarrow \tilde{D}_{\mathbf{Q}, 2\sigma_{\text{sign}}}[\mathbf{h}]$	4 : $\mathbf{H}(M \parallel \text{salt}) := \mathbf{h}$
5 : $\mathbf{s} := \frac{1}{2}(\mathbf{h} + \langle \mathbf{w} \rangle) \in R_n^2$	5 : $\mathbf{s} := \frac{1}{2}(\mathbf{h} + \langle \mathbf{w} \rangle) \in R_n^2$
6 : return $\text{sig} := (\text{salt}, \mathbf{s})$	6 : return $\text{sig} := (\text{salt}, \mathbf{s})$

Figure 7: Oracles Trans_{sk} and $\text{Sim}^{D_{\mathbf{Q}, 2\sigma_{\text{sign}}}}$.

The only difference between OSign and Trans_{sk} is that when the former calls Sign_{sk} it computes $\mathbf{h} := \mathbf{H}(M \parallel \text{salt})$, while the latter instead reprogrammes $\mathbf{H}(M \parallel \text{salt}) := \mathbf{h} \leftarrow (R_n/2R_n)^2$. We note that, since Trans_{sk} has $\mathbf{B}$ it is able to sample $\mathbf{w}$ from $\tilde{D}_{\mathbf{Q}, 2\sigma_{\text{sign}}}[\mathbf{h}]$.

Considering Lemma 4, the guessing probability of an input $M \parallel \text{salt}$ is at most $2^{-\text{saltlen}_{\text{bits}}}$ for any distribution over M and therefore, letting $\gamma = 2^{-\text{saltlen}_{\text{bits}}}$ and $q_r = q_s$, we have

$$\left| \text{Adv}_{\Pi_{\text{HAWK}}, \mathcal{A}}^{\text{SUF-CMA}} - \text{Adv}_{\Pi_{\text{HAWK}}, \mathcal{A}}^{\text{SUF-CMA-Trans}} \right| \leq q_s \sqrt{q_h + q_s} \cdot 2^{-\text{saltlen}_{\text{bits}}/2} + q_s(q_h + q_s) \cdot 2^{-\text{saltlen}_{\text{bits}}-1}, \quad (49)$$

where it is understood that the verification $\text{Vf}^{\mathbf{H}}$ is performed using the possibly reprogrammed $\mathbf{H}$.

We now hop from Trans to Sim . First note that in Trans after line 4, one may redefine $\mathbf{h} := \mathbf{w} \bmod 2$ and $\mathbf{H}(M \parallel \text{salt}) := \mathbf{h}$ without affecting the execution, since $\mathbf{w} \bmod 2 = \mathbf{h}$ holds when $\mathbf{w} \leftarrow \tilde{D}_{\mathbf{Q}, 2\sigma_{\text{sign}}}[\mathbf{h}]$ by (42). After this, the only difference between Trans and Sim is that in the former $\mathbf{w}$ is sampled from $\tilde{D}_{\mathbf{Q}, 2\sigma_{\text{sign}}}$ and in the latter by $D_{\mathbf{Q}, 2\sigma_{\text{sign}}}$. Therefore, as there are at most q_s independent samples drawn from one of the two distributions, the probability preservation of the Rényi divergence implies for every $a \in (1, \infty)$ that

$$\text{Adv}_{\Pi_{\text{HAWK}}, \mathcal{A}}^{\text{SUF-CMA-Sim}} \geq \frac{\left(\text{Adv}_{\Pi_{\text{HAWK}}, \mathcal{A}}^{\text{SUF-CMA-Trans}} \right)^{a/(a-1)}}{R_a(\tilde{D}_{\mathbf{Q}, 2\sigma_{\text{sign}}} \parallel D_{\mathbf{Q}, 2\sigma_{\text{sign}}})^{q_s}}, \quad (50)$$

holds, where $R_a(\tilde{D}_{\mathbf{Q}, 2\sigma_{\text{sign}}} \| D_{\mathbf{Q}, 2\sigma_{\text{sign}}})$ can be computed by Lemma 1. We thus conclude that the *validity* of a forgery is preserved when replacing the signing oracle OSign by Sim up to the additive factor of (49) and the multiplicative factor of (50).

Furthermore, the *freshness* of a signature forgery is also preserved, in that we can assume without loss of generality that $\mathcal{A}$ never outputs a forgery (M^*, sig^*) that matches the response of a signing query.

6.3.2 Extracting a fresh short vector

Consider the algorithm $\mathcal{E}^H$ that on input a message forgery pair (M^*, sig^*) computes

$$\mathbf{h}^* := H(M^* \| \text{salt}^*), \text{ and } \mathbf{w}^* := 2\mathbf{s}^* - \mathbf{h}^* \quad (51)$$

and outputs $\mathbf{w}^*$. Slightly abusing notation, we define the algorithm $\mathcal{B}^{\text{samp}} := \mathcal{E}^H \circ \mathcal{A}^{\text{Sim}, H}$. We take it as understood that when $\mathcal{A}$ makes a query to Sim then $\mathcal{B}$ simulates this query; in particular it obtains $\mathbf{w} \leftarrow D_{\mathbf{Q}, 2\sigma_{\text{sign}}}$ as required by Sim by calling its samp oracle. Furthermore, $\mathcal{B}$ locally simulates the random oracle H . It follows that if $\mathcal{A}^{\text{Sim}, H}$ succeeds in producing a valid forgery then the $\mathbf{w}^*$ output by $\mathcal{B}$ is a short non-zero vector, i.e. $0 < \|\mathbf{w}^*\|_{\mathbf{Q}} \leq 2\sigma_{\text{verify}} \cdot \sqrt{2n} = L$. That $\mathbf{w}^*$ is non zero follows from the check on line 5 of Vf in Fig. 6. It remains to show that $\mathbf{w}^*$ is a fresh ac-omSVP solution as well, i.e. $\mathbf{w}^* \notin \mathcal{L}_{\text{samples}}$; recall $\mathcal{L}_{\text{samples}}$ is defined in Fig. 5.

To lower bound the probability that $\mathbf{w}^*$ is such a fresh ac-omSVP solution, assume that (M^*, sig^*) is a valid and fresh signature forgery such that $\mathbf{w}^* = \alpha(\mathbf{w}_j)$ for some $(j, \alpha) \in [q_s] \times \mathcal{G}_K$, i.e. that $\mathbf{w}^*$ is not fresh. Note that we have $\mathbf{w}^* \neq \mathbf{0}$. Let $(M_i, \text{salt}_i, \mathbf{h}_i, \mathbf{s}_i, \mathbf{w}_i)$ be the values taken by $(M, \text{salt}, \mathbf{h}, \mathbf{s}, \mathbf{w})$ in the i^{th} query of $\mathcal{A}$ to Sim , and let $\text{sig}^* = (\text{salt}^*, \mathbf{s}^*)$ be the signature output by $\mathcal{A}$. We distinguish between the following two cases.

First, $M^* \| \text{salt}^* = M_i \| \text{salt}_i$ for some $i \in [q_s]$, where we consider i to be maximal such that the equality holds.⁶ Then $\mathbf{h}^* = \mathbf{h}_i$, and so

$$\mathbf{s}^* = \frac{1}{2}(\mathbf{h}^* + \mathbf{w}^*) = \frac{1}{2}(\mathbf{h}_i + \alpha(\mathbf{w}_j)) \in R_n^2.$$

If $i \neq j$ then for any fixed choice of $\mathbf{h}_i$, the probability over the choice of $\mathbf{w}_j$ of there being an $\alpha \in \mathcal{G}_K$ as above, is at most $2n \cdot 2^{-2n} \cdot \frac{1+\varepsilon}{1-\varepsilon}$ by (47). There are q_s choices for i and $q_s - 1$ choices for $j \neq i$. On the other hand, if $i = j$ then we get that

$$R_n^2 \ni \mathbf{s}^* = \frac{1}{2}(\mathbf{h}^* + \mathbf{w}^*) = \frac{1}{2}(\mathbf{h}_i + \alpha(\mathbf{w}_i)).$$

Furthermore, when $i = j$ we have $\alpha \neq \pm 1$. Indeed, if we assume $\alpha \in \{-1, 1\}$ then $\langle \mathbf{w}_i \rangle = \langle \mathbf{w}^* \rangle = \mathbf{w}^*$, where the second equality holds by the validity of sig^* and the first follows from $\alpha \mathbf{w}_i = \mathbf{w}^* \notin R_n \times \{0\}$ and (41). Therefore $\mathbf{s}^* = \frac{1}{2}(\mathbf{h}_i + \langle \mathbf{w}_i \rangle) = \mathbf{s}_i$ which contradicts the freshness of sig^* . The probability over the choice of $\mathbf{w}_i$ of there being an $\alpha \in \mathcal{G}_K \setminus \{\pm 1\}$ as above is at most $(n+1)2^{-n} \cdot \frac{1+\varepsilon}{1-\varepsilon}$ by (46). There are q_s choices for $i = j$.

Second, $M^* \| \text{salt}^* \neq M_i \| \text{salt}_i$ for every $i \in [q_s]$. In this case we have that

$$R_n^2 \ni \mathbf{s}^* = \frac{1}{2}(\mathbf{h}^* + \mathbf{w}^*) = \frac{1}{2}(\mathbf{h}^* + \alpha(\mathbf{w}_j)),$$

and so $H(M^* \| \text{salt}^*) = \mathbf{h}^* = \alpha(\mathbf{w}_j) \bmod 2$; furthermore, H has not been reprogrammed throughout the execution at the location $M^* \| \text{salt}^*$. Hence

$$H_{\text{init}}(M^* \| \text{salt}^*) = H(M^* \| \text{salt}^*) \in \{\alpha(\mathbf{w}_j) \bmod 2 \mid (j, \alpha) \in [q_s] \times \mathcal{G}_K\} =: S, \quad (52)$$

⁶If i is not the largest, it can be $(M^*, \text{salt}^*) = (M_i, \text{salt}_i)$ yet $\mathbf{h}^* \neq \mathbf{h}_i$ because $\mathbf{h}^*$ is computed via the possibly reprogrammed H .

where H_{init} is the initial H without any reprogramming. Thus, parsing $\mathcal{A}^{\text{Sim}, H}$ as $\mathcal{C}^{H_{\text{init}}}$, which runs the calls to Sim (for arbitrary but fixed samples $\mathbf{w}_1, \dots, \mathbf{w}_{q_s}$ of $D_{\mathbf{Q}, 2\sigma_{\text{sign}}}$) and the reprogramming of H internally, we obtain an algorithm that finds a preimage under H_{init} of an element in S , making q_h queries to H_{init} . Note that $\#S \leq nq_s$, so such an algorithm can succeed with probability at most $O(q_h^2 \cdot n \cdot q_s / 2^{2n})$ via the standard preimage finding bound. Hence by combining the two cases to get a non fresh forgery,

$$\text{Adv}_{\text{ac-omSVP}, \mathcal{B}}^{\text{SAMPLE}} \geq \text{Adv}_{\Pi_{\text{HAWK}}, \mathcal{A}}^{\text{SUF-CMA-Sim}} - O(q_h^2 \cdot n \cdot q_s / 2^{2n}) - \frac{2nq_s}{2^n} \left(1 + \frac{q_s}{2^n}\right) \cdot \frac{1 + \varepsilon}{1 - \varepsilon}. \quad (53)$$

Theorem 1 is now readily proven by combining Eqs. (49), (50) and (53).

6.3.3 Lifting to Unsimplified HAWK

We now argue in a black box manner that the security of Π_{HAWK} is essentially the same as that of the following scheme:

$$\Pi_{\text{HAWK}}^{\text{full}} := (\text{HawkKeyGen}, \text{HawkSign}, \text{HawkVerify}),$$

where the signing and verification are now as specified in Section 2.1. In particular a (Q)ROM-SUF-CMA adversary against this game has a signing oracle $\text{OSign}^{\text{full}}$ that restarts on the same conditions as HawkSign .

The signature scheme Π_{HAWK} lacks the following steps compared to $\Pi_{\text{HAWK}}^{\text{full}}$. First, a single evocation of HawkSign restarts with probability at most pr when certain conditions fail. We define

$$\text{pr} = \max_{\text{sk}} [\Pr[\text{HawkSign}_{\text{sk}} \text{ restarts when } \mathbf{h} = \mathbf{h}_{\text{max}}]] \quad (54)$$

where we quantify over all possible sk output by HawkKeyGen and $\mathbf{h}_{\text{max}}$ is the realisation of $\mathbf{h}$ that maximises the probability of a restart for a given sk . This is conservative in the sense that we are assuming an adversary can determine $\mathbf{h}_{\text{max}}$ without knowing sk and can always choose a message M such that $\mathbf{h}_{\text{max}}$ is internally sampled, despite the salt. We note that the public key and choice of H plays no role in signing once $\mathbf{h}$ is chosen.

Given access to OSign one may adaptively query it κ times for some positive integer κ , and choose uniformly from the signatures that would not cause HawkSign to restart. Note that whether a signature output by OSign should cause a restart is a publicly and efficiently checkable condition. One can then simulate a call to $\text{OSign}^{\text{full}}$ except if all κ queries to OSign would cause a restart in HawkSign . All κ queries would cause a restart with probability bounded above by pr^κ , and therefore by a union bound one may simulate q_s queries to $\text{OSign}^{\text{full}}$ using $\kappa \cdot q_s$ queries to OSign , except with probability $q_s \cdot \text{pr}^\kappa$.

Second, in both HawkSign and HawkVerify an initial hash $M := H(\mathbf{m} \parallel \text{hpub})$ is computed. As long as no restart occurs, a successful forgery against $\Pi_{\text{HAWK}}^{\text{full}}$ implies either a collision for H , which, via the standard collision resistance bound [AS04, Zha15], is found with probability $O((q_h + \kappa \cdot q_s)^3 / \#\mathcal{M})$ where $\mathcal{M}$ is the codomain for the initial hash $H(\mathbf{m} \parallel \text{hpub})$, or a successful forgery against Π_{HAWK} .

Finally, in HawkSign the vector $\mathbf{s}$ is compressed for HawkVerify to decompress later. Since the compression and decompression are public, a forgery attack against such a compressed signature scheme can always be transformed to one against the uncompressed counterpart.

Accounting for the above differences, for every attacker $\mathcal{A}$ against $\Pi_{\text{HAWK}}^{\text{full}}$ making (q_s, q_h) respective queries to $(\text{OSign}^{\text{full}}, H)$, there is an attacker $\mathcal{A}'$ against Π_{HAWK} as in (48), making $(\kappa \cdot q_s, q_h)$ respective queries to (OSign, H) , such that

$$\text{Adv}_{\Pi_{\text{HAWK}}^{\text{full}}, \mathcal{A}}^{\text{SUF-CMA}} \leq \text{Adv}_{\Pi_{\text{HAWK}}, \mathcal{A}'}^{\text{SUF-CMA}} + q_s \cdot \text{pr}^\kappa + O((q_h + \kappa \cdot q_s)^3 / \#\mathcal{M}), \quad (55)$$

This concludes the security of $\Pi_{\text{HAWK}}^{\text{full}}$.

Theorem 2 (Quantum security of $\Pi_{\text{HAWK}}^{\text{full}}$). *Let $\kappa \in \mathbb{Z}_{\geq 1}$ and $\mathcal{A}$ be an adversary against the (Q)ROM-SUF-CMA game making at most q_s queries to OSign and at most q_h quantum queries to H respectively. Then there exists an adversary $\mathcal{B}$ against the SAMPLE game making $q_o = \kappa \cdot q_s$ queries to samp , with running time $\text{TIME}(\mathcal{B}) = \text{TIME}(\mathcal{A}) + \text{Overhead}(q_o, q_h)$. This overhead consists of simulating q_h hash queries to H and q_o queries to Sim (specified in Fig. 7). Furthermore, for each $a \in (1, \infty)$ we have*

$$\begin{aligned} \text{Adv}_{\Pi_{\text{HAWK}}^{\text{full}}, \mathcal{A}}^{\text{SUF-CMA}} &\leq \left[R_a(\tilde{D}_{\mathbf{Q}, 2\sigma_{\text{sign}}} \| D_{\mathbf{Q}, 2\sigma_{\text{sign}}})^{q_s} \cdot \left(\text{Adv}_{\text{ac-omSVP}, \mathcal{B}}^{\text{SAMPLE}} + O(q_h^2 \cdot n \cdot q_o / 2^{2n}) \right) \right]^{1-1/a} \\ &\quad + q_s \sqrt{q_h + q_s} \cdot 2^{-\text{saltlen}_{\text{bits}}/2} + q_s(q_h + q_s) \cdot 2^{-\text{saltlen}_{\text{bits}}-1} \\ &\quad + q_s \cdot \text{pr}^\kappa + O((q_h + q_o)^3 / \#\mathcal{M}), \end{aligned}$$

where $\mathcal{M}$ is the codomain of $\text{H}(\text{m} \| \text{hpub})$ and pr is an upper bound on the probability that HawkSign restarts. The constants hidden by the O are independent of a and κ .

6.4 Classical Security

As our proof is modular, one may substitute parts of the proof of Theorem 2 to obtain better bounds when considering $\mathcal{A}$ that only makes classical queries to H . In (49), where the closeness between Sign and Trans is examined, we may substitute

$$\left| \text{Adv}_{\Pi_{\text{HAWK}}, \mathcal{A}}^{\text{SUF-CMA}} - \text{Adv}_{\Pi_{\text{HAWK}}, \mathcal{A}}^{\text{SUF-CMA-Trans}} \right| \leq 2q_s(q_h + q_s) \cdot 2^{-\text{saltlen}_{\text{bits}}}. \quad (56)$$

Moreover, to control the event (52) of finding a preimage of at most $n \cdot q_s$ elements, classically we have

$$\Pr[\text{H}_{\text{init}}(M^*, \text{salt}^*) \in S] \leq (q_h + 1) \cdot n \cdot q_s / 2^{2n}.$$

Finally, the probability of finding a collision in (55) can be replaced with $O((q_h + \kappa \cdot q_s)^2 / \#\mathcal{M})$, thus

$$\text{Adv}_{\Pi_{\text{HAWK}}^{\text{full}}, \mathcal{A}}^{\text{SUF-CMA}} \leq \text{Adv}_{\Pi_{\text{HAWK}}, \mathcal{A}'}^{\text{SUF-CMA}} + q_s \cdot \text{pr}^\kappa + O((q_h + \kappa \cdot q_s)^2 / \#\mathcal{M}).$$

We obtain the classical security of HAWK as follows.

Theorem 3 (Classical security of $\Pi_{\text{HAWK}}^{\text{full}}$). *Let $\kappa \in \mathbb{Z}_{\geq 1}$ and $\mathcal{A}$ be an adversary against the (Q)ROM-SUF-CMA game making at most q_s queries to OSign and at most q_h classical queries to H respectively. Then there exists an adversary $\mathcal{B}$ against the SAMPLE game making $q_o = \kappa \cdot q_s$ queries to samp , with running time $\text{TIME}(\mathcal{B}) = \text{TIME}(\mathcal{A}) + \text{Overhead}(q_o, q_h)$. This overhead consists of simulating q_h hash queries to H and q_o queries to Sim (specified in Fig. 7). Furthermore, for each $a \in (1, \infty)$ we have*

$$\begin{aligned} \text{Adv}_{\Pi_{\text{HAWK}}^{\text{full}}, \mathcal{A}}^{\text{SUF-CMA}} &\leq \left[R_a(\tilde{D}_{\mathbf{Q}, 2\sigma_{\text{sign}}} \| D_{\mathbf{Q}, 2\sigma_{\text{sign}}})^{q_s} \cdot \left(\text{Adv}_{\text{ac-omSVP}, \mathcal{B}}^{\text{SAMPLE}} + (q_h + 1) \cdot n \cdot q_o / 2^{2n} \right) \right]^{1-1/a} \\ &\quad + 2q_o(q_h + q_o) \cdot 2^{-\text{saltlen}_{\text{bits}}} + q_s \cdot \text{pr}^\kappa + O((q_h + q_o)^2 / \#\mathcal{M}), \end{aligned}$$

where $\mathcal{M}$ is the codomain for the initial hash $\text{H}(\text{m} \| \text{hpub})$ and pr is an upper bound on the probability that HawkSign restarts. The constants hidden by the O are independent of a and κ .

6.5 To table based sampling HAWK

Theorem 2 is not quite the end of our reduction. There is one final leap; from $\Pi_{\text{HAWK}}^{\text{full}}$ to the HAWK specified in Section 3, namely to a HAWK that samples from a table based approximation of $\tilde{D}_{\mathbf{Q}, 2\sigma_{\text{sign}}}$. This table based approximation is achieved by sampling one entry at a time, as described in Section 3.5.1 and in particular in Algorithm 14.

We first consider the Rényi divergence between $D_{2\mathbb{Z}, 2\sigma_{\text{sign}}}$ and $D_{2\mathbb{Z}+1, 2\sigma_{\text{sign}}}$ and their table based approximations. Then, in Theorem 4 we collect the results of this section and those above to finally relate HAWK as specified to omSVP.

Table based sampling. Let D'_b be the distribution of a single coefficient $\mathbf{d}[r]$ for $0 \leq r < 2n$ in Algorithm 14, given that $\mathbf{t}[r] = b$ for $b \in \{0, 1\}$. Internally, the distribution D'_b is determined by the values in T_b as part of Table 5, and has support some subset of $2\mathbb{Z} + b$.

First, we consider two distributions, P_1 and P_2 , where P_1 is the distribution of all randomness that is involved in the signing of at most q_s signatures with the table based sampler, which are requested by an adversary against HAWK, and P_2 is the distribution when using exact discrete Gaussian samplers. Let E be the event of the adversary providing a forgery in either case. If $P_1(E) \geq 2^{-\lambda}$ then, by the probability preservation property of the Rényi divergence [Pre17, Sec. 3.3],⁷ for all $a \in (1, \infty)$ we have

$$P_2(E) \geq P_1(E)^{a/(a-1)} / R_a(P_1 \parallel P_2) \geq P_1(E) \cdot \frac{1}{R_a(P_1 \parallel P_2) \cdot 2^{\lambda/(a-1)}}. \quad (57)$$

Note that if $P_1(E) < 2^{-\lambda}$ then we are done; our adversary against table based sampling HAWK outputs a forgery with small enough probability.

It is now left to determine $R_a(P_1 \parallel P_2)$ for a well chosen a . For sampling one coefficient from either D'_b or $D_{2\mathbb{Z}+b, 2\sigma_{\text{sign}}}$ the Rényi divergence can be computed numerically. Then, the multiplicativity and data processing inequality give us

$$R_a(P_1 \parallel P_2) \leq \left(\max_{b \in \{0, 1\}} R_a(D'_b \parallel D_{2\mathbb{Z}+b, 2\sigma_{\text{sign}}}) \right)^{2nq_s}, \quad (58)$$

as there are at most q_s signature queries, each requiring $2n$ discrete Gaussian samples.

In the auxiliary data⁸ the Rényi divergences are calculated for HAWK- $\{256, 512, 1024\}$. These show that for HAWK-512 and HAWK-1024 the Rényi divergence for both $b \in \{0, 1\}$ is smaller than $1 + 2^{-78}$ at an order $a = 513$, which implies at most one bit of security being lost in this reduction by (6). In fact, the script shows that the security loss is even smaller than one bit by considering higher orders a . In particular, for HAWK-512, at order $a = 37286$ we obtain $P_2(E) \geq P_1(E)/1.07$, and for HAWK-1024 at order $a = 4181$ we obtain $P_2(E) \geq P_1(E)/1.17$.

Total security loss when reducing omSVP to specified HAWK. We now relate the security of HAWK as specified in Section 3 to omSVP. In particular when the sampling discussed in Section 2.1 is implemented via Section 3.5.1 we define

$$\Pi_{\text{HAWK}}^{\text{tables}} = (\text{HawkKeyGen}, \text{HawkSign}, \text{HawkVerify}),$$

Specifically, $\Pi_{\text{HAWK}}^{\text{tables}}$ is identical to $\Pi_{\text{HAWK}}^{\text{full}}$ except HawkSign is now Algorithm 15, which calls a sampler that uses precomputed tables, whereas $\Pi_{\text{HAWK}}^{\text{full}}$ sampled from $\tilde{D}_{\mathbf{Q}, 2\sigma_{\text{sign}}}[\mathbf{h}]$.

⁷See <https://tprest.github.io/pdf/pub/renyi.pdf> for the most recent version.

⁸See https://github.com/hawk-sign/aux/blob/main/code/generate_C_tables.py for the computation.

Theorem 4. Let $\lambda \in \{128, 256\}$, $q_s \leq 2^{64}$ and $q_h \leq 2^\lambda$. Let $\mathcal{A}$ be a (quantum) adversary against the (Q)ROM-SUF-CMA game against $\Pi_{\text{HAWK}}^{\text{tables}}$ that runs in time at most $t_{\mathcal{A}} \geq 1$, makes (q_s, q_h) queries to (OSign, H) respectively, and has success probability

$$\varepsilon_{\mathcal{A}} = \text{Adv}_{\Pi_{\text{HAWK}}^{\text{tables}}, \mathcal{A}}^{\text{SUF-CMA}}(\lambda),$$

such that $t_{\mathcal{A}}/\varepsilon_{\mathcal{A}} < 2^\lambda$. In particular $\lambda = 128$ corresponds to HAWK-512 and $\lambda = 256$ corresponds to HAWK-1024. Moreover, assume $\text{saltlen}_{\text{bits}}$ and $\#\mathcal{M}$ are taken arbitrarily large (in particular, at least as large as listed in Table 4).

Then, there exists a (quantum) adversary $\mathcal{B}$ against the SAMPLE game with the appropriate $\text{Init}(1^\lambda)$ running in time at most $t_{\mathcal{B}} = t_{\mathcal{A}} + \text{Overhead}(\kappa \cdot q_s, q_h)$. This overhead consists of making $\kappa \cdot q_s$ queries to **samp** and simulating q_h queries to H. The success probability of $\mathcal{B}$ satisfies

$$\text{Adv}_{\text{ac-omSVP}, \mathcal{B}}^{\text{SAMPLE}}(\lambda) > \frac{1}{2} \varepsilon_{\mathcal{A}}.$$

Proof. Note $\varepsilon_{\mathcal{A}} > 2^{-\lambda}$ as $t_{\mathcal{A}} \geq 1$. We make the following five hops

1. Table based sampling ($\Pi_{\text{HAWK}}^{\text{tables}}$) against (Q)ROM-SUF-CMA,
2. Exact sampling ($\Pi_{\text{HAWK}}^{\text{full}}$) against (Q)ROM-SUF-CMA,
3. Simplified signing (Π_{HAWK}) against (Q)ROM-SUF-CMA,
4. Reprogramming H with uniform targets $\mathbf{h}$ ($\mathcal{A}^{\text{Trans}, \text{H}}$),
5. Reprogramming H with targets $\mathbf{h} = \mathbf{w} \bmod 2$ with $\mathbf{w} \leftarrow D_{\mathbf{Q}, 2\sigma_{\text{sign}}}(\mathcal{A}^{\text{Sim}, \text{H}})$,
6. Using oracle calls to **samp**, against SAMPLE ($\mathcal{B}^{\text{samp}}$).

For $i \in \{2, 3, 4, 5\}$, we use ε_i to mean the advantage of $\mathcal{A}$ transformed to the i th game.

Transforming $\mathcal{A}$ from 1 to 2, using Eqs. (57) and (58) and the computation from the auxiliary data, we obtain

$$\varepsilon_2 \geq \varepsilon_{\mathcal{A}}/1.17. \quad (59)$$

We now estimate $\text{pr} \leq \frac{1}{2}$, and discuss this more in Section 6.7. Let $\kappa = 67 + \lambda$ and assume $\#\mathcal{M}$ is large enough such that $O\left((q_h + \kappa \cdot q_s)^3 / \#\mathcal{M}\right) \leq 2^{-\lambda}/16$ in (55). This allows us to bound the additive loss in (55) above by $2^{-\lambda}/8 + 2^{-\lambda}/16 = \frac{3}{16}2^{-\lambda}$, giving

$$\varepsilon_3 \geq \varepsilon_2 - \frac{3}{16}2^{-\lambda} \geq 0.60 \varepsilon_{\mathcal{A}}. \quad (60)$$

Now to hop from 3 to 4, we use (49). We assume $\text{saltlen}_{\text{bits}}$ is sufficiently large. In particular, taking $\text{saltlen}_{\text{bits}}$ large enough such that the term in (49) is at most $2^{-\lambda}/16$ (requiring it to be larger than in Table 4), gives

$$\varepsilon_4 \geq \varepsilon_3 - 2^{-\lambda}/16 \geq \varepsilon_2 - \frac{1}{4}2^{-\lambda}. \quad (61)$$

By Lemma 1 we compute the concrete security loss from 4 to 5. The function `security_loss_cosets` of the `generate_C_tables.py` script⁹ computes the minimal security loss in (50). Here (61) and our starting assumption on $\varepsilon_{\mathcal{A}}$ imply $\varepsilon_4 \geq 2^{-(\lambda+10)}$ holds, which allows us the following multiplicative loss from the Rényi divergence

$$\varepsilon_5 \geq \varepsilon_4 \cdot \left(\varepsilon_4^{1/(a-1)} / R_a(\tilde{D}_{\mathbf{Q}, 2\sigma_{\text{sign}}} \| D_{\mathbf{Q}, 2\sigma_{\text{sign}}})^{q_s} \right) \geq \varepsilon_4 / \left(2^{\frac{\lambda+10}{a-1}} R_a(\tilde{D}_{\mathbf{Q}, 2\sigma_{\text{sign}}} \| D_{\mathbf{Q}, 2\sigma_{\text{sign}}})^{q_s} \right).$$

⁹See <https://github.com/hawk-sign/aux>.

In particular, we calculated for HAWK-512 that $\varepsilon_5 \geq \varepsilon_4/1.04$ holds by taking the Rényi divergence at order $a = 5051$, while for HAWK-1024 we get $\varepsilon_5 \geq \varepsilon_4/1.03$ by taking the Rényi divergence at order $a = 14428$. Taking the worst, from (61) we obtain

$$\varepsilon_5 \geq 0.60 \varepsilon_{\mathcal{A}}/1.04 \geq 0.57 \varepsilon_{\mathcal{A}}. \quad (62)$$

For the final hop we use (53). We use $\varepsilon < \frac{1}{2}$, $q_s \leq 2^{64} \leq 2^{n/4}$ and $q_h \leq 2^\lambda \leq 2^{n/4}$, yielding the negligible additive loss of at most $(1 + 12nq_s)/2^n$. This gives us

$$\varepsilon_{\mathcal{B}} \geq \frac{1}{2} \varepsilon_{\mathcal{A}}. \quad (63)$$

Hence the success probability of $\mathcal{B}$ decreases by at most a factor of two. $\square$

Let $\lambda \in \{128, 256\}$. Theorem 4 tells us that if there is an adversary $\mathcal{A}$ against HAWK-(4 λ) with $t_{\mathcal{A}}/\varepsilon_{\mathcal{A}} < 2^\lambda$ that makes (q_s, q_h) queries to (OSign, H), then there exists a $(t_{\mathcal{B}}, \varepsilon_{\mathcal{B}})$ adversary against the relevant ac-omSVP instance that makes $\kappa \cdot q_s$ queries to **samp** and has $t_{\mathcal{B}}/\varepsilon_{\mathcal{B}} \leq 2t_{\mathcal{B}}/\varepsilon_{\mathcal{A}}$.

It remains to consider $t_{\mathcal{B}}$. We see that $t_{\mathcal{B}}/t_{\mathcal{A}} = 1 + \text{Overhead}(\kappa \cdot q_s, q_h)/t_{\mathcal{A}}$. In the worst case, where $\mathcal{A}$ makes no queries to H and performs no computation, while maximising q_s , this quotient is $1 + \kappa = 1 + \lambda + 67 \leq 324 < 2^9$.

Therefore $t_{\mathcal{B}}/\varepsilon_{\mathcal{B}} \leq 2t_{\mathcal{B}}/\varepsilon_{\mathcal{A}} < 2^{10}t_{\mathcal{A}}/\varepsilon_{\mathcal{A}} < 2^{\lambda+10}$. The following corollary takes the contrapositive of the above statement.

Corollary 1. *Let $\lambda \in \{128, 256\}$, $n = 4\lambda$, $\kappa = \lambda + 67$, $q_s \leq 2^{64}$ and $q_h \leq 2^\lambda$. Consider the ac-omSVP instance where $\text{Init}(1^\lambda)$ returns $\mathbf{Q}$ sampled from HawkKeyGen, μ_K as in Section 6.1.1, $L = 2\sqrt{2n} \cdot \sigma_{\text{verify}}$ and $\sigma = 2\sigma_{\text{sign}}$. If, for all $t \geq 1$ there is no (t, ε) -SAMPLE adversary against ac-omSVP that makes at most $q_o = \kappa \cdot q_s$ queries to **samp** and has $t/\varepsilon < 2^{\lambda+10}$, then there is no (t, ε) -(Q)ROM-SUF-CMA adversary against $\Pi_{\text{HAWK}}^{\text{tables}}$ that makes (q_s, q_h) queries to (OSign, H) such that $t/\varepsilon < 2^\lambda$.*

Proof. This follows from the discussion above. $\square$

6.6 Search module lattice isomorphism problem

The search module lattice isomorphism problem was formalised in [DPPvW22a, Sec. 6.1] building upon the general lattice isomorphism formalism for cryptography proposed in [DvW22]. Similar problems were considered before during the cryptanalytic effort against early NTRU based signature schemes [GS02, Szy03].

Intuitively it is a problem where one is given two matrices over a field K from the same equivalence class under some relation, and must find an invertible matrix over the ring of integers R that relates them.

Recall, for a CM-field K and $r \in \mathbb{Z}_{\geq 1}$ we define $\mathcal{H}_r^{>0}(K) \subset K^{r \times r}$ as the set of all $\mathbf{Q} \in K^{r \times r}$ such that $\mathbf{Q} = \mathbf{Q}^*$ and $\text{Tr}(\mathbf{v}^* \mathbf{Q} \mathbf{v}) > 0$ for all $\mathbf{v} \in K^r \setminus \{\mathbf{0}\}$, i.e. matrices that represent Hermitian positive definite forms. We then define an equivalence relation over $\mathcal{H}_r^{>0}(K)$ where $\mathbf{Q} \sim \mathbf{Q}'$ if and only if there exists $\mathbf{U} \in \text{GL}_r(R)$ such that $\mathbf{Q}' = \mathbf{U}^* \mathbf{Q} \mathbf{U}$ and write $[\mathbf{Q}]$ for the equivalence class of $\mathbf{Q}$. This is a well defined equivalence relation as $\text{GL}_r(K)$ is a group and therefore has an identity, inverses and is closed under composition.

Note that in [DPPvW22a] a different but equivalent definition of $\sim$ is given that requires $\mathbf{U}$ to be in the special linear group. This simplifies proofs regarding average case instances of module lattice isomorphism problems. The simplification of key generation in HAWK compared to HAWK-AC22 no longer allows these average case distributions so we simplify our presentation here, see Section 6.7 for more details. We now give the definition of worst case smLIP.

Definition 7 (Worst case smLIP). Given a CM-field K , its ring of integers R and $\mathbf{Q} \in \mathcal{H}_r^{>0}(K)$, an instance of the worst case search module lattice isomorphism problem $\text{wc-smLIP}_{K,r}^{\mathbf{Q}}$ is given by any $\mathbf{Q}' \in [\mathbf{Q}]$. A solution is $\mathbf{U} \in \text{GL}_r(R)$ such that $\mathbf{Q}' = \mathbf{U}^* \mathbf{Q} \mathbf{U}$.

Note that K and r alone do not define the problem, instances are within an equivalence class determined by $\mathbf{Q}$. The specific equivalence class we work in for HAWK is $[\mathbf{I}_2(K)]$ for K some power of two cyclotomic, i.e. $\mathbf{Q} = \mathbf{I}_2(K)$ in the above definition. Furthermore, the $\mathbf{Q}'$ we are given in HAWK are not worst case, they follow the distribution of public keys output by key generation. While this is in some formal sense an ‘average case’ distribution, it differs from the average case distributions defined in [DvW22, DPPvW22a]. Recall that (after passing to the unstructured setting) finding $\mathbf{U} \in \text{GL}_{2n}(\mathbb{Z})$ that solves the corresponding search lattice isomorphism problem instance is precisely the cryptanalytic experiment we perform in Section 5.1.

6.7 Discussion

Changes to key generation and smLIP. An algorithmic difference between HAWK as specified in Section 3 and HAWK-AC22 is the change in sampling during key generation. In HAWK-AC22 the entries of f, g are sampled from a discrete Gaussian over $\mathbb{Z}$ with a given width, whereas in HAWK this is replaced by a centred binomial distribution of similar width. The experiments of Section 5.1 suggest this simplification makes no difference to the practical hardness of secret key recovery via lattice reduction. However, it does not allow us to appeal to the definitions of average case (module) search lattice isomorphism instances given in [DvW22, Sec. 3.1] and [DPPvW22a, Sec. 6.1]. In the below we follow the notation of [DPPvW22a] which specialises to that of [DvW22] when $K = \mathbb{Q}$ and their $s = \sqrt{2\pi}\sigma$.

The average case distributions are defined algorithmically within an equivalence class $[\mathbf{Q}]$ for some $\mathbf{Q} \in \mathcal{H}_r^{>0}(K)$ and a width parameter σ . We denote an average case distribution by $\text{AC}_\sigma([\mathbf{Q}])$ and the algorithm that samples it by ac_σ .

Distributions $\text{AC}_\sigma([\mathbf{Q}])$ have the useful property that, given any $\mathbf{Q}' \in [\mathbf{Q}]$ and if σ is large enough, then on input $\mathbf{Q}'$ the algorithm ac_σ outputs a sample according to $\text{AC}_\sigma([\mathbf{Q}])$, along with an isomorphism between the sample and $\mathbf{Q}'$. Put simply, provided σ is chosen large enough one can use any representative element of the equivalence class $[\mathbf{Q}]$ to sample from $\text{AC}_\sigma([\mathbf{Q}])$ via ac_σ .

This effectively allows a rerandomisation of worst case instances to average case instances, and therefore worst case to average case reductions when σ is large, see [DvW22, Lem. 3.9] and [DPPvW22a, Lem. 5]. A seemingly necessary property of the distributions sampled internally in ac_σ to produce an output according to $\text{AC}_\sigma([\mathbf{Q}])$ is that they are radial; the probability mass of a sampled element depends only on its ℓ^2 norm. This is not true of the centred binomial distribution, for example if $X \sim \text{Bin}(\eta)^4$ for $\eta \geq 2$ then $(2, 0, 0, 0)$ has the same length as $(1, 1, 1, 1)$ but a different probability mass.

While we therefore cannot claim any worst case to average case reduction for the problem of recovering the secret key of HAWK from the public key, we note that this was already the case for the concrete parameters of HAWK-AC22, where the discrete Gaussian was used in key generation. This is because the width σ_{keygen} used in HAWK-AC22 was too small and did not satisfy [DPPvW22a, Lem. 5].

Discussion of omSVP parametrisation. The ac-omSVP instances implicit in our reduction from HAWK to omSVP are trivially solvable. In particular, without making any queries to the `samp` oracle, an adversary can submit $\mathbf{w}^* = (1 \ 0)$. In this case $\|\mathbf{w}^*\|_{\mathbf{Q}} = \|(f, g)\| \approx \sqrt{2n}\sigma_{\text{krsec}} \leq 2\sqrt{2n}\sigma_{\text{verify}} = L$. This is a consequence of us choosing σ_{krsec} as small as our practical cryptanalysis will allow us, see Section 5.1, and not being able to choose σ_{verify} small enough to mitigate this without unacceptably increasing the restart probability. This

is an instance of us choosing to follow practical cryptanalysis for setting parameters, and using our theoretical reduction to convince us of the soundness of our design; we could easily fix this by increasing η but this would also increase the size of our public keys. We note that, while this decision makes our reduction vacuous, we do not know practically how to use these public key vectors to create forgeries against HAWK. More generally, one can consider an adversary that performs lattice reduction on the form $\mathbf{Q}$ and possibly combines the result with samples from the **samp** oracle.

Before we discuss further, we introduce a lemma that gives a concentration result (from above and below) on the lengths of vectors sampled from a discrete Gaussian distribution whenever ε is a negligible function of the dimension of the lattice. We specialise it to $\mathbb{Z}^n$ and to the width σ which is a factor $\sqrt{2\pi}$ smaller than in [BF11].

Lemma 5 ([BF11, Prop. 7]). *Let $\sigma \geq \eta_\varepsilon(\mathbb{Z}^n)$ for ε a negligible function of n . For any constant $\alpha > 0$ and $\mathbf{w} \leftarrow D_{\mathbb{Z}^n, \sigma}$*

$$\Pr \left[(1 - \alpha)\sigma\sqrt{n} \leq \|\mathbf{w}\| \leq (1 + \alpha)\sigma\sqrt{n} \right] \geq 1 - \text{negl}(n)$$

In particular, $\|\mathbf{w}\|^2$ for $\mathbf{w} \leftarrow D_{\mathbb{Z}^n, \sigma}$ is concentrated around $n\sigma^2$. We could rewrite the above as $\|\mathbf{w}\|_{\mathbf{Q}}^2$ for $\mathbf{w} \leftarrow D_{\mathbf{Q}, 2\sigma_{\text{sign}}}$ is concentrated around $8n\sigma_{\text{sign}}^2$ to make it more relevant to HAWK.

Asymptotically, whenever we satisfy the conditions of Lemma 5 if we are able to parametrise such that $2\sigma_{\text{verify}} < \sigma_{\text{krsec}}$ and our threshold behaviour on lengths during lattice reduction is accurate, then no vectors found during lattice reduction will be short enough to act as omSVP solutions until vectors of length one begin being discovered. One can also consider combining vectors discovered during lattice reduction on $\mathbf{Q}$ with $\mathbf{w}$ received from **samp**. This was conceptualised as a weak forgery attack in [DPPvW22a, Sec. 4.3].

Another parametrisation to note is $\sigma_{\text{verify}} > \sqrt{2}\sigma_{\text{sign}}$. In this instance $\mathbf{w}_1, \mathbf{w}_2$ from **samp** are such that $\|\mathbf{w}_1 + \mathbf{w}_2\|_{\mathbf{Q}}$ will be short enough to be a omSVP solution with overwhelming probability. We assume that $\mathbf{w}_1 + \mathbf{w}_2 \notin \mathcal{L}_{\text{samples}}$. This again does not immediately lead to a forgery for HAWK, but if $\mathbf{h}_1, \mathbf{h}_2 \in (R_n/2R_n)^2$ are such that $\mathbf{w}_i \in \mathbf{h}_i + 2R_n$ then finding a message and salt such that $H(\mathbf{m} \parallel \text{salt}) \in \{\mathbf{h}_1, \mathbf{h}_2, \mathbf{h}_1 + \mathbf{h}_2\}$ will lead to a forgery. Querying many $\mathbf{w}_i$ from **samp** may ease forgery, as one effectively has quadratically many targets $\mathbf{h}_i + \mathbf{h}_j$ with $i \neq j$ to find preimages of. Our parameter sets are such that $\sigma_{\text{verify}} < \sqrt{2}\sigma_{\text{sign}}$ by some margin.

In a more idealised setting, we are effectively asking a omSVP adversary to perform a relaxed notion of lattice sieving for just one vector. The standard heuristic from that literature is that the vectors $\mathbf{w}$ from **samp** are i.i.d. uniform points on a sphere of radius $2\sqrt{2n}\sigma_{\text{sign}}$ [NV08]. Rather than asking for enough distinct shorter combinations of such $\mathbf{w}$ to recurse a sieving operation, the SAMPLE game asks for just one combination of $\mathbf{w}$, provided it does not fall into $\mathcal{L}_{\text{samples}}$, that can actually be slightly longer, by a factor of $\sigma_{\text{verify}}/\sigma_{\text{sign}} \geq 1$. Optimising k -sieving techniques [BLS16, HK17] for this setting where a single relaxed combination is sought may be an interesting approach. Also, whether having access to (lattice reduction on) $\mathbf{Q}$ can help an adversary in this setting, or whether one can reduce to a variant of omSVP where the adversary is not given $\mathbf{Q}$ by showing something equivalent can be obtained efficiently by using the **samp** oracle, are other interesting avenues for further work.

Finally, the applicability of learning style attacks [NR09, DN12] to the particular ac-omSVP instances relevant to HAWK should be considered. Lemma 1 tells us that $\tilde{D}_{\mathbf{Q}, 2\sigma_{\text{sign}}}$ and $D_{\mathbf{Q}, 2\sigma_{\text{sign}}}$ are close, which relate to $\mathbf{w}$ sampled internally during HAWK signing and the output of the **samp** oracle in the SAMPLE game. Recall that $\mathbf{w}$ is public in HAWK signing since it can be computed from $\mathbf{s}$ and $\mathbf{h}$. Therefore, if one can mount such a learning style attack against HAWK signatures, then one can mount such a learning attack against the SAMPLE game using $\mathbf{w}$ from **samp** with a very similar probability.

Altogether now, parameter selection. We now describe our rationale for parameter selection, which one can see algorithmically as the function `find_params`.¹⁰

We begin with the selection of σ_{sign} . We need $\sigma_{\text{sign}} \geq \eta_\varepsilon(\mathbb{Z}^{2n})$ for a suitable ε to appeal to Lemma 2, and also to ensure the Rényi divergence of Lemma 1 is small enough. Ultimately “small enough” is quantified by what are acceptable sizes for the respective losses in the reductions of Sections 6.3 to 6.5 and the effect on the practical cryptanalysis of Section 5, but we find setting $\varepsilon = 1/\sqrt{q_s} \cdot \lambda$ as in FALCON to be a reasonable value. Increasing ε , and therefore decreasing σ_{sign} , decreases the signature sizes.

From σ_{sign} we must select σ_{verify} that is small enough that forgery is not too easy, but also large enough that not too many signatures are rejected due to $\|\mathbf{w}\|_{\mathbf{Q}}$ being too long. In particular we take $\sigma_{\text{verify}} \in (\sigma_{\text{sign}}, \sigma_{\text{krsec}}]$. If $\sigma_{\text{verify}} > \sigma_{\text{krsec}}$ then in our simulation model, e.g. Table 7, the block sizes required for forgery would become smaller than those for secret key recovery. Though it is always true formally that forgery is no harder than secret key recovery, here we are trying to practically ensure that one must recover the secret key to forge. The value we choose for σ_{verify} in this range is maximal such that the weak forgery attack considered in [DPPvW22a, Sec. 4.3] is sufficiently hard. Determining more accurately the value of that attack, and in particular showing that it is ineffective, would allow us to increase σ_{verify} . However, we note that in practice the vast majority of restarts in signing are caused because signatures cannot be properly encoded, so *decreasing* σ_{verify} , which can only improve security, would not penalise us much.

The value σ_{krsec} is not chosen by us, but determined by simulation and our practical cryptanalysis. To ensure that HawkKeyGen does not resample f, g too many times, the distribution that the entries of these polynomials come from must be wide enough to satisfy $\|(f, g)\|^2 > 2n\sigma_{\text{krsec}}^2$ often enough. Thankfully, the centred binomials with $\eta = 4$ and $\eta = 8$ are sufficient. In particular, they have standard deviations $\sqrt{2}$ and 2 respectively which are either close to or above σ_{krsec} for HAWK-512 and HAWK-1024.

We now discuss our choice of $\text{saltlen}_{\text{bits}}$ and the codomain of the hash $M \leftarrow H(\mathbf{m} \parallel \text{hpub})$ in Algorithm 15. We choose $\text{saltlen}_{\text{bits}} = \lambda + \log_2(q_s)$ for the practical reason that the probability of a collision in $H(\mathbf{m} \parallel \text{salt})$ over q_s queries is at most $2^{-\lambda}$. However, in light of Eqs. (49) and (56), this is not sufficient either classically or quantumly to make this leap in the reduction non vacuous. Increasing $\text{saltlen}_{\text{bits}}$ by b bits to account for this would require b bits more randomness per signature attempt and increase the signature size by b bits. Similarly, we realise $M \leftarrow \text{SHAKE256}(\mathbf{m} \parallel \text{hpub})[0 : 512]$. Depending on the number of quantum queries to H allowed, this may make (55) vacuous.

We note that $\text{saltlen}_{\text{bits}}$ is chosen as in FALCON, and that the codomain of the initial hash is chosen as in DILITHIUM. Both are easy to change if required, with little effect on the efficiency of HAWK.

Finally, in Theorem 4 we concretised the restart probability pr defined in (54) as at most one half. It seems that this value may be possible to upper bound pr by considering the restrictions put on f, g, F, G in HawkKeyGen and distribution on lengths of $\mathbf{x}$ in HawkSign. The smaller we are able to make this upper bound, the smaller the parameter κ in our reduction may be taken.

¹⁰https://github.com/hawk-sign/aux/blob/main/code/find_params.sage

References

- [ADM⁺24] Thomas Aulbach, Samed DüzlÜ, Michael Meyer, Patrick Struck, and Maximiliane Weishäupl, *Hash your keys before signing - BUFF security of the additional NIST PQC signatures*, Post-Quantum Cryptography - 15th International Workshop, PQCrypto 2024, Part II (Markku-Juhani Saarinen and Daniel Smith-Tone, eds.), Springer, Cham, June 2024, pp. 301–335.
- [ADPS16] Erdem Alkim, Léo Ducas, Thomas Pöppelmann, and Peter Schwabe, *Post-quantum key exchange - A new hope*, USENIX Security 2016 (Thorsten Holz and Stefan Savage, eds.), USENIX Association, August 2016, pp. 327–343.
- [AGPS20] Martin R. Albrecht, Vlad Gheorghiu, Eamonn W. Postlethwaite, and John M. Schanck, *Estimating quantum speedups for lattice sieves*, ASIACRYPT 2020, Part II (Shiho Moriai and Huaxiong Wang, eds.), LNCS, vol. 12492, Springer, Cham, December 2020, pp. 583–613.
- [AGVW17] Martin R. Albrecht, Florian Göpfert, Fernando Virdia, and Thomas Wunderer, *Revisiting the expected cost of solving uSVP and applications to LWE*, ASIACRYPT 2017, Part I (Tsuyoshi Takagi and Thomas Peyrin, eds.), LNCS, vol. 10624, Springer, Cham, December 2017, pp. 297–322.
- [APvW25] Bill Allombert, Alice Pellet-Mary, and Wessel P. J. van Woerden, *Cryptanalysis of rank-2 module-lp: a single real embedding is all it takes*, EUROCRYPT 2025 (Serge Fehr and Pierre-Alain Fouque, eds.), LNCS, Springer, 2025, to appear.
- [AS04] Scott Aaronson and Yaoyun Shi, *Quantum lower bounds for the collision and the element distinctness problems*, Journal of the ACM (JACM) **51** (2004), no. 4, 595–605.
- [AWHT16] Yoshinori Aono, Yuntao Wang, Takuya Hayashi, and Tsuyoshi Takagi, *Improved progressive BKZ algorithms and their precise cost estimation by sharp simulator*, EUROCRYPT 2016, Part I (Marc Fischlin and Jean-Sébastien Coron, eds.), LNCS, vol. 9665, Springer, Berlin, Heidelberg, May 2016, pp. 789–819.
- [BDGL16] Anja Becker, Léo Ducas, Nicolas Gama, and Thijs Laarhoven, *New directions in nearest neighbor searching with applications to lattice sieving*, 27th SODA (Robert Krauthgamer, ed.), ACM-SIAM, January 2016, pp. 10–24.
- [BDH⁺23] Guido Bertoni, Joan Daemen, Seth Hoffert, Michaël Peeters, Gilles Van Assche, Ronny Van Keer, and Benoît Viguier, *TurboSHAKE*, Cryptology ePrint Archive, Report 2023/342, 2023.
- [BF11] Dan Boneh and David Mandell Freeman, *Linearly homomorphic signatures over binary fields and new tools for lattice-based signatures*, PKC 2011 (Dario Catalano, Nelly Fazio, Rosario Gennaro, and Antonio Nicolosi, eds.), LNCS, vol. 6571, Springer, Berlin, Heidelberg, March 2011, pp. 1–16.
- [BGPS23] Huck Bennett, Atul Ganju, Pura Peetathawatchai, and Noah Stephens-Davidowitz, *Just how hard are rotations of $\mathbb{Z}^n$? algorithms and cryptography with the simplest lattice*, EUROCRYPT 2023, Part V (Carmit Hazay and Martijn Stam, eds.), LNCS, vol. 14008, Springer, Cham, April 2023, pp. 252–281.

- [BLL⁺15] Shi Bai, Adeline Langlois, Tancrede Lepoint, Damien Stehlé, and Ron Steinfeld, *Improved security proofs in lattice-based cryptography: Using the Rényi divergence rather than the statistical distance*, ASIACRYPT 2015, Part I (Tetsu Iwata and Jung Hee Cheon, eds.), LNCS, vol. 9452, Springer, Berlin, Heidelberg, November / December 2015, pp. 3–24.
- [BLS16] Shi Bai, Thijs Laarhoven, and Damien Stehlé, *Tuple lattice sieving*, LMS Journal of Computation and Mathematics **19** (2016), no. A, 146–162.
- [BM21] Tamar Lichter Blanks and Stephen D. Miller, *Generating cryptographically-strong random lattice bases and recognizing rotations of $\mathbb{Z}^n$* , Post-Quantum Cryptography - 12th International Workshop, PQCrypto 2021 (Jung Hee Cheon and Jean-Pierre Tillich, eds.), Springer, Cham, 2021, pp. 319–338.
- [BN24] Henry Bambury and Phong Q. Nguyen, *Improved provable reduction of NTRU and hypercubic lattices*, Post-Quantum Cryptography - 15th International Workshop, PQCrypto 2024, Part I (Markku-Juhani Saariinen and Daniel Smith-Tone, eds.), Springer, Cham, June 2024, pp. 343–370.
- [CDF⁺20] Cas Cremers, Samed Düzlül, Rune Fiedler, Marc Fischlin, and Christian Janson, *BUFFing signature schemes beyond unforgeability and the case of post-quantum signatures*, Cryptology ePrint Archive, Report 2020/1525, 2020.
- [CDF⁺21] ———, *BUFFing signature schemes beyond unforgeability and the case of post-quantum signatures*, 2021 IEEE Symposium on Security and Privacy, IEEE Computer Society Press, May 2021, pp. 1696–1714.
- [CFM⁺24] Clémence Cheviguard, Pierre-Alain Fouque, Guilhem Mureau, Alice Pellet-Mary, and Alexandre Wallet, *A reduction from hawk to the principal ideal problem in a quaternion algebra*, Cryptology ePrint Archive, Report 2024/1147, 2024.
- [CN11] Yuanmi Chen and Phong Q. Nguyen, *BKZ 2.0: Better lattice security estimates*, ASIACRYPT 2011 (Dong Hoon Lee and Xiaoyun Wang, eds.), LNCS, vol. 7073, Springer, Berlin, Heidelberg, December 2011, pp. 1–20.
- [DDGR20] Dana Dachman-Soled, Léo Ducas, Huijing Gong, and Mélissa Rossi, *LWE with side information: Attacks and concrete security estimation*, CRYPTO 2020, Part II (Daniele Micciancio and Thomas Ristenpart, eds.), LNCS, vol. 12171, Springer, Cham, August 2020, pp. 329–358.
- [DEP23] Léo Ducas, Thomas Espitau, and Eamonn W. Postlethwaite, *Finding short integer solutions when the modulus is small*, CRYPTO 2023, Part III (Helena Handschuh and Anna Lysyanskaya, eds.), LNCS, vol. 14083, Springer, Cham, August 2023, pp. 150–176.
- [DFHS24] Jelle Don, Serge Fehr, Yu-Hsuan Huang, and Patrick Struck, *On the (in)security of the BUFF transform*, CRYPTO 2024, Part I (Leonid Reyzin and Douglas Stebila, eds.), LNCS, vol. 14920, Springer, Cham, August 2024, pp. 246–275.
- [DN12] Léo Ducas and Phong Q. Nguyen, *Learning a zonotope and more: Cryptanalysis of NTRUSign countermeasures*, ASIACRYPT 2012 (Xiaoyun Wang and Kazue Sako, eds.), LNCS, vol. 7658, Springer, Berlin, Heidelberg, December 2012, pp. 433–450.

- [DPPvW22a] Léo Ducas, Eamonn W. Postlethwaite, Ludo N. Pulles, and Wessel P. J. van Woerden, *Hawk: Module LIP makes lattice signatures fast, compact and simple*, ASIACRYPT 2022, Part IV (Shweta Agrawal and Dongdai Lin, eds.), LNCS, vol. 13794, Springer, Cham, December 2022, pp. 65–94.
- [DPPvW22b] ———, *Hawk: Module LIP makes lattice signatures fast, compact and simple*, Cryptology ePrint Archive, Report 2022/1155, 2022.
- [Duc18] Léo Ducas, *Shortest vector from lattice sieving: A few dimensions for free*, EUROCRYPT 2018, Part I (Jesper Buus Nielsen and Vincent Rijmen, eds.), LNCS, vol. 10820, Springer, Cham, April / May 2018, pp. 125–145.
- [Duc22] ———, *Estimating the hidden overheads in the BDGL lattice sieving algorithm*, Post-Quantum Cryptography - 13th International Workshop, PQCrypto 2022 (Jung Hee Cheon and Thomas Johansson, eds.), Springer, Cham, September 2022, pp. 480–497.
- [Duc24] ———, *Provable lattice reduction of $\mathbb{Z}^n$ with blocksize $n/2$* , DCC **92** (2024), no. 4, 909–916.
- [DvW22] Léo Ducas and Wessel P. J. van Woerden, *On the lattice isomorphism problem, quadratic forms, remarkable lattices, and cryptography*, EUROCRYPT 2022, Part III (Orr Dunkelman and Stefan Dziembowski, eds.), LNCS, vol. 13277, Springer, Cham, May / June 2022, pp. 643–673.
- [EK20] Thomas Espitau and Paul Kirchner, *The nearest-colattice algorithm: time-approximation tradeoff for approx-CVP*, The Open Book Series **4** (2020), no. 1, 251–266, ANTS XIV.
- [EP24] Thomas Espitau and Heorhii Pliatsok, *On hermitian decomposition lattices and the module-LIP problem in rank 2*, Cryptology ePrint Archive, Report 2024/1148, 2024.
- [ETWY22] Thomas Espitau, Mehdi Tibouchi, Alexandre Wallet, and Yang Yu, *Shorter hash-and-sign lattice-based signatures*, CRYPTO 2022, Part II (Yevgeniy Dodis and Thomas Shrimpton, eds.), LNCS, vol. 13508, Springer, Cham, August 2022, pp. 245–275.
- [FH23] Serge Fehr and Yu-Hsuan Huang, *On the quantum security of HAWK*, Post-Quantum Cryptography - 14th International Workshop, PQCrypto 2023 (Thomas Johansson and Daniel Smith-Tone, eds.), Springer, Cham, August 2023, pp. 405–416.
- [FIP15] *SHA-3 standard: Permutation-based hash and extendable-output functions*, National Institute of Standards and Technology, NIST FIPS PUB 202, U.S. Department of Commerce, August 2015.
- [GHHM21] Alex B. Grilo, Kathrin Hövelmanns, Andreas Hülsing, and Christian Majenz, *Tight adaptive reprogramming in the QROM*, ASIACRYPT 2021, Part I (Mehdi Tibouchi and Huaxiong Wang, eds.), LNCS, vol. 13090, Springer, Cham, December 2021, pp. 637–667.
- [GPV08] Craig Gentry, Chris Peikert, and Vinod Vaikuntanathan, *Trapdoors for hard lattices and new cryptographic constructions*, 40th ACM STOC (Richard E. Ladner and Cynthia Dwork, eds.), ACM Press, May 2008, pp. 197–206.

- [GR24] Morgane Guereau and Mélissa Rossi, *A not so discrete sampler: Power analysis attacks on hawk signature scheme*, IACR Transactions on Cryptographic Hardware and Embedded Systems **2024** (2024), no. 4, 156–178.
- [GS02] Craig Gentry and Michael Szydlo, *Cryptanalysis of the revised NTRU signature scheme*, EUROCRYPT 2002 (Lars R. Knudsen, ed.), LNCS, vol. 2332, Springer, Berlin, Heidelberg, April / May 2002, pp. 299–320.
- [HK17] Gottfried Herold and Elena Kirshanova, *Improved algorithms for the approximate k -list problem in euclidean norm*, PKC 2017, Part I (Serge Fehr, ed.), LNCS, vol. 10174, Springer, Berlin, Heidelberg, March 2017, pp. 16–40.
- [Kec11] *Keccak implementation overview*, Submitted to NIST’s SHA-3 competition, September 2011, <https://keccak.team/obsolete/Keccak-implementation-3.1.pdf>.
- [KO62] Anatoly Karatsuba and Yuri Ofman, *Multiplication of many-digital numbers by automatic computers*, Proceedings of the USSR Academy of Sciences **145** (1962), no. 2, 293–294.
- [LDK⁺22] Vadim Lyubashevsky, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Peter Schwabe, Gregor Seiler, Damien Stehlé, and Shi Bai, *CRYSTALS-DILITHIUM*, Tech. report, National Institute of Standards and Technology, 2022, available at <https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022>.
- [LJPW24] Hengyi Luo, Kaijie Jiang, Yanbin Pan, and Anyu Wang, *Cryptanalysis of rank-2 module-LIP with symplectic automorphisms*, ASIACRYPT 2024, Part IV (Kai-Min Chung and Yu Sasaki, eds.), LNCS, vol. 15487, Springer, Singapore, December 2024, pp. 359–385.
- [LS17] Hendrik W. Lenstra, Jr. and Alice Silverberg, *Lattices with symmetry*, Journal of Cryptology **30** (2017), no. 3, 760–804.
- [LS19] Hendrik W. Lenstra Jr. and Alice Silverberg, *Testing isomorphism of lattices over CM-Orders*, SIAM Journal on Computing **48** (2019), no. 4, 1300–1334.
- [MPPW24] Guilhem Mureau, Alice Pellet-Mary, Georgii Pliatsok, and Alexandre Wallet, *Cryptanalysis of rank-2 module-LIP in totally real number fields*, EUROCRYPT 2024, Part VII (Marc Joye and Gregor Leander, eds.), LNCS, vol. 14657, Springer, Cham, May 2024, pp. 226–255.
- [MR04] Daniele Micciancio and Oded Regev, *Worst-case to average-case reductions based on Gaussian measures*, 45th FOCS, IEEE Computer Society Press, October 2004, pp. 372–381.
- [MW16] Daniele Micciancio and Michael Walter, *Practical, predictable lattice basis reduction*, EUROCRYPT 2016, Part I (Marc Fischlin and Jean-Sébastien Coron, eds.), LNCS, vol. 9665, Springer, Berlin, Heidelberg, May 2016, pp. 820–849.
- [NR09] Phong Q. Nguyen and Oded Regev, *Learning a parallelepiped: Cryptanalysis of GGH and NTRU signatures*, Journal of Cryptology **22** (2009), no. 2, 139–160.
- [NV08] Phong Q. Nguyen and Thomas Vidick, *Sieve algorithms for the shortest vector problem are practical*, J. Mathematical Cryptology **2** (2008), no. 2, 181–207.

- [PFH⁺22] Thomas Prest, Pierre-Alain Fouque, Jeffrey Hoffstein, Paul Kirchner, Vadim Lyubashevsky, Thomas Pornin, Thomas Ricosset, Gregor Seiler, William Whyte, and Zhenfei Zhang, *FALCON*, Tech. report, National Institute of Standards and Technology, 2022, available at <https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022>.
- [Por23] Thomas Pornin, *Improved key pair generation for falcon, BAT and hawk*, Cryptology ePrint Archive, Report 2023/290, 2023.
- [PP19] Thomas Pornin and Thomas Prest, *More efficient algorithms for the NTRU key generation using the field norm*, PKC 2019, Part II (Dongdai Lin and Kazue Sako, eds.), LNCS, vol. 11443, Springer, Cham, April 2019, pp. 504–533.
- [Pre17] Thomas Prest, *Sharper bounds in lattice-based cryptography using the Rényi divergence*, ASIACRYPT 2017, Part I (Tsuyoshi Takagi and Thomas Peyrin, eds.), LNCS, vol. 10624, Springer, Cham, December 2017, pp. 347–374.
- [SAB⁺22] Peter Schwabe, Roberto Avanzi, Joppe Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M. Schanck, Gregor Seiler, Damien Stehlé, and Jintai Ding, *CRYSTALS-KYBER*, Tech. report, National Institute of Standards and Technology, 2022, available at <https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022>.
- [Szy03] Michael Szydło, *Hypercubic lattice reduction and analysis of GGH and NTRU signatures*, EUROCRYPT 2003 (Eli Biham, ed.), LNCS, vol. 2656, Springer, Berlin, Heidelberg, May 2003, pp. 433–448.
- [Zha15] Mark Zhandry, *A note on the quantum collision and set equality problems*, Quantum Information and Computation **15** (2015), no. 7-8, 557–567.